Plan de Migración: Marcador Outbound (Java) → IPcom Dialer (.NET con Asterisk.Sdk)

Versión: 6.0
Fecha: 2026-03-07
Autor: Arquitectura de Software
Estado: Borrador para revisión
Clasificación: Documento técnico interno
Producto origen: DyalogoCBXMarcador v1.5.2 (Java, legacy Dyalogo — ahora propiedad de ipcom.ai)
Producto destino: IPcom Dialer v1.0 (.NET 10 Native AOT)

Tabla de Contenidos

1. Resumen Ejecutivo
2. Entendimiento del Sistema Actual
3. Supuestos Técnicos
4. Evaluación Inicial del Proyecto Java
5. Arquitectura Objetivo en .NET
6. Estrategia de Migración
7. Inventario de Componentes a Migrar
8. Plan por Fases
9. Mapeo Tecnológico Java → .NET
10. Diseño de Integración con Asterisk
11. Riesgos y Mitigaciones
12. Plan de Pruebas
13. Plan de Despliegue
14. Recomendaciones de Observabilidad
15. Backlog Inicial de Trabajo
16. Cronograma Tentativo
17. Conclusión
18. Anexos

---

1. Resumen Ejecutivo

Objetivo General

Reemplazar completamente la aplicación **DyalogoCBXMarcador** (v1.5.2, legacy Dyalogo — ahora propiedad de ipcom.ai) con un nuevo producto: **IPcom Dialer v1.0**, construido en **.NET 10 con Native AOT** sobre la librería **Asterisk.Sdk**. El marcador es un componente del ecosistema más amplio de IPcom (antes Dyalogo), por lo que **el esquema de base de datos MySQL existente no se modifica** — el nuevo sistema opera directamente sobre las mismas tablas. El resultado será un sistema más mantenible, performante y alineado con la plataforma tecnológica

objetivo de la organización, libre de la marca Dyalogo.

Beneficios Esperados

Beneficio	Descripción
Rendimiento	Native AOT elimina JIT cold-start; System.IO.Pipelines ofrece parsing zero-copy vs. reflection de asterisk-java
Escalabilidad	Asterisk.Sdk validado para 100K+ agentes con `AsteriskServerPool` multi-servidor
Mantenibilidad	Código tipado, inyección de dependencias nativa, source generators vs. reflection
Observabilidad	`System.Diagnostics.Metrics` integrado (contadores, histogramas, gauges) vs. log4j manual
Unificación tecnológica	Consolida stack en .NET 10, eliminando dependencia de JRE y Hibernate
Independencia de marca	Producto propio IPcom Dialer, sin dependencia del código base Dyalogo
Compatibilidad de datos	Opera sobre el esquema MySQL existente sin alteraciones — coexiste con el resto del ecosistema
Testabilidad	Interfaces (`IAmiConnection`, `IAsteriskServer`) facilitan unit testing con NSubstitute
Seguridad	Eliminación de credenciales hardcodeadas, integración con secret managers
Operación	Binario self-contained (~1.3 MB AOT) vs. uber-jar con 15+ dependencias

Riesgos Principales

1. **Paridad funcional incompleta:** Comportamientos no documentados en el código Java que se pierdan en la migración
2. **Continuidad operativa:** El marcador es crítico para la operación diaria de campañas
3. **Integración con CRM:** La integración CRM existente (base de datos directa + REST) debe replicarse exactamente
4. **Timing de marcación:** La lógica de throttling, retries y delays es sensible al rendimiento y debe preservarse
5. **Esquema de base de datos compartido:** Tablas MySQL compartidas entre múltiples servicios — el esquema NO se modifica

Restricciones de Diseño

Restricción	Razón
No modificar esquema de BD	El marcador es una pieza de un sistema más grande (CRM, AgiAmi, Web) que comparte las mismas tablas. Cualquier cambio de esquema afectaría a todos los módulos
Eliminar marca Dyalogo	El código fuente original fue adquirido por ipcom.ai. El nuevo producto se denomina **IPcom Dialer** — namespaces, nombres de servicio y documentación no deben referenciar "Dyalogo"
Reemplazo total	No se busca coexistencia gradual. El objetivo es un reemplazo completo del marcador Java. No aplica Strangler Fig
Dockerizado desde día 1	El despliegue es 100% containerizado. No se instala en bare-metal ni con systemd. Imagen Docker multi-stage con AOT publish

Restricción	Razón
Kubernetes-ready	Aunque el despliegue inicial es Docker Compose, el diseño debe ser compatible con Kubernetes: stateless, config via env vars/ConfigMaps, health/readiness probes, graceful shutdown, logs a stdout
Multi-tenant	IPcom alquila la plataforma a clientes. Cada cliente (`id_huesped`/`id_proyecto`) ejecuta 1+ campañas. Todas las queries deben filtrar por tenant
Asterisk 18 → latest	Producción actual es Asterisk 18. Se planea migrar a versiones recientes. IPcom Dialer debe ser compatible con Asterisk 18, 20, 21+

Estrategia Recomendada

Reemplazo completo (Big Bang controlado) con validación exhaustiva previa al corte. A diferencia de un Strangler Fig, el objetivo no es coexistencia gradual sino construir el nuevo sistema completo, validarlo en shadow mode contra el sistema Java, y realizar un corte único cuando haya paridad funcional demostrada. Se mantiene el Java como fallback durante un período de estabilización corto.

Justificación del cambio respecto a Strangler Fig:

- El marcador es una unidad funcional atómica — no tiene sentido migrar "medio marcador" porque las campañas dependen de todos los subsistemas (originate, callbacks, AGI, CRM sync) funcionando juntos
- La complejidad de coexistencia (flags por campaña, dos AGI servers, dos procesadores de abandonadas) introduce más riesgo del que evita
- El esquema de BD no se modifica, por lo que no hay migración de datos — solo cambio de consumidor

---

2. Entendimiento del Sistema Actual

Tipo de Aplicación

DyalogoCBXMarcador es un servicio standalone de Java (ejecutable como `java -jar`) que funciona como **marcador automático de campañas outbound** (dialer). No es una aplicación web; es un servicio daemon multi-hilo que se conecta a Asterisk vía AMI y a MySQL para persistencia.

Responsabilidades Funcionales

```
■■■
```

Módulo	Paquete Java	Clases Clave	Responsabilidad
AMI	`modelo.asterisk`	`ConexionAMI`, `AccionesAsteriskAMI`, `ManejadorEventos`	Conexión dual AMI, originate async, event handling
Callbacks	`modelo.asterisk`	`DyalogoOriginateCallback`, `DYOriginateCallback`	Manejo de resultados: Success/NoAnswer/Busy/Failure
Campañas	`modelo`	`ManejaCampanasMarcacion`, `HCampanaMarcacion`	Loop principal (20s), scheduling por hora/día
PDS/Dinámico	`modelo.dinamico`	`HMarcacionPDSDinamicoPrincipal`, `HMarcacionDinamicoCampanaMaster`	Marcación predictiva, robótica, PDS
Contactos	`modelo`	`HEjecutaMarcacionContacto`, `HProcesadorActualizadorContactos`	Ejecución por contacto, hasta 10 números, retries
Persistencia	`dao`, `conexionbd`	`DaoMarcadorCampanas`, `DaoContactos`, `DaoMuestras`	Hibernate JPA + C3P0, MySQL
Entidades	`tablas`	26 entities (campañas, contactos, CDR, troncales, etc.)	Modelo de datos JPA
CRM	`integracioncrm`	`OperacionesDatosCRM`, `ConsumeREST`	INSERT/UPDATE en BD CRM + REST POST
Thread Pools	`modelo`	`PoolHilosMarcador`	50 hilos llamadas + 20 hilos eventos
Utilidades	`modelo`	`UtilidadMarcador`, `AnalizadorRutaSaliente`	Traducción de eventos, análisis de rutas

Relación con Asterisk

```
DyalogoCBXMarcador ■■AMI■■■ Asterisk PBX
        ■                   ■
        ■  OriginateAction  ■  HangupEvent
        ■  (async + callback) ■  QueueCallerAbandonEvent
        ■                   ■  NewStateEvent
        ■  CoreShowChannels ■
        ■                   ■
        ▼                   ▼
   MySQL (estado)      CDR/QueueLog
```

Conexión dual AMI:

- Conexión primaria: eventos (ManejadorEventos implementa ManagerEventListener)
- Conexión secundaria: AsteriskServerImpl para `originateAsync()` con callbacks

Flujo de originate:

1. Se selecciona contacto de la muestra activa
2. Se itera por hasta 10 teléfonos del contacto
3. Se construye `OriginateAction` con channel, context, exten, callerID, timeout, variables
4. Se envía `originateAsync()` con `DyalogoOriginateCallback`
5. Callback reporta: Dialing(1) → Success(2) / NoAnswer(3) / Busy(4) / Failure(5)
6. Se actualiza estado del contacto y se registra en log

Tipos de Campaña

Tipo	Código	Descripción
PDS	6	Preview Dialing System — agente revisa antes de marcar
Predictivo	7	Marca automáticamente anticipando disponibilidad de agentes
Robótico	8	Marcación masiva sin agentes (mensajes pregrabados)

Proyectos Hermanos: DyalogoCBXLib y DyalogoCBXAgiAmi

El marcador **no es un sistema aislado**. Depende directamente de dos proyectos hermanos que amplían significativamente el alcance de la migración:

DyalogoCBXLib — Librería Compartida de Plataforma

Naturaleza: Librería core compartida por todos los módulos de Dyalogo (Marcador, Web, AgiAmi, etc.).

Dimensión	Cantidad
Entidades JPA	113+ (74 core + 31 canales electrónicos + 8 marcador)
DAOs	128+ (58 core + 30 canales electrónicos + 6 marcador + otros)
Caches en memoria	16 implementaciones (`ICache<T>` + `ObjetoCache<T>`)
Utilidades	29 clases (fechas, archivos, HTTP, Base64, Excel, etc.)
DTOs/Domain	25+ objetos de transferencia para WS/API
Integraciones externas	Microsoft 365 (Graph API), Infobip (SMS), SendGrid (email)
Persistencia	2 unidades JPA: `dyalogocbxbd` (RESOURCE_LOCAL) + `dyalogocrmbd` (JTA)

Componentes usados directamente por el Marcador:

Componente de CBXLib	Uso en Marcador	Acción de Migración
`AnalizadorRutaSaliente`	Análisis de ruta por número destino	Migrar a .NET (ya considerado)
`ConfiguracionCBX` (singleton)	Config global del sistema	Reemplazar con `IOptions<T>`
`ConstantesCBX`	Constantes de rutas y paths	Convertir a `static class`
`FuncionesFecha`	Operaciones de fecha/hora	Reemplazar con `DateTimeOffset` nativo
`HTTPRequest`	Cliente HTTP para REST CRM	Reemplazar con `IHttpClientFactory`
`Conversiones`, `ConversionesNumericas`	Formateo de teléfonos	Extension methods en .NET
`EncriptadorPropio` (seguridad)	Cifrado de claves de agentes	Replicar algoritmo o migrar a bcrypt
`ManejaEncripcion`	Hash de claves de salida	Replicar o reemplazar
`EnumTecnologiasVOIP`	SIP, IAX, DAHDI, etc.	`enum` en .NET
`IOperacionesTelefoniaAMI`	Interfaz de operaciones AMI	Reemplazar con `IAmiConnection`
`OperacionesTelefonia`	Wrapper de comandos Asterisk	Reemplazar con Asterisk.Sdk Actions
`DAOImpOperaciones<T>` (base DAO)	Base de todos los DAOs	Reemplazar con Dapper repositories
`ICache<T>` + `ObjetoCache<T>`	Framework de cache	`IMemoryCache` / `ConcurrentDictionary`
Entidades de marcador (8)	`DyMarcadorCampanas`, `DyMarcadorContactos`, etc.	POCOs + Dapper

Componente de CBXLib	Uso en Marcador	Acción de Migración
Entidades core compartidas	`DyAgentes`, `DyCampanas`, `DyExtensiones`, `DyTroncales`, `DyRutasSalientes`, `DyVariablesGlobales`	POCOs + Dapper (solo lectura)
`CodigosError` (enum)	Códigos de error estándar	`enum` en .NET

Componentes de CBXLib NO necesarios para el Marcador:

Módulo	Razón de exclusión
Canales Electrónicos (email/chat) — 30 DAOs, 31 entidades	No relacionado con marcación
Microsoft 365 / Infobip / SendGrid	Integraciones de otros módulos
Bot/AI (`ia/bot/`)	Módulo de chatbot, no de telefonía
Journey management	Customer journey, no dialer
Video integration	Video calls, no outbound dialer
Multi-tenancy (`DyHospedadores`)	Evaluar si producción es multi-tenant
SOAP WS (59+ clases)	Web services para la interfaz web

> ***Implicación clave:*** *No se migra DyalogoCBXLib completa. Se extraen* ***solo los componentes que usa el Marcador*** *(~25% de la librería). Esto simplifica enormemente el alcance pero requiere un análisis fino de dependencias transitivas.*

DyalogoCBXAgiAmi — Servicio AGI/AMI Bridge

Naturaleza: Servicio standalone (v2.0.24) que funciona como puente entre Asterisk AGI/AMI y la plataforma Dyalogo. Corre como proceso independiente junto al Marcador.

Dimensión	Cantidad
AGI Scripts	17 clases (extienden `BaseAgiScript`)
Hilos AMI	8 event processors
DAOs	31 específicos
Entidades JPA	77+
Clases totales	115+

AGI Scripts críticos para el Marcador:

Script AGI	Función	Invocado por
`EventoMarcadorContestaHumano`	Actualiza contacto a "Contestada" cuando humano contesta	Dialplan de Asterisk (contexto de campaña)
`EventoMarcadorContestaMaquina`	Actualiza contacto cuando detecta contestadora	Dialplan de Asterisk (AMD)
`InsertaRegistroMuestraMarcador`	Inserta registro en muestra del marcador	Dialplan de Asterisk
`EnviaRespuestaAMDAPI`	Respuesta de Answering Machine Detection	Dialplan de Asterisk

Flujo bidireccional Marcador ↔ AgiAmi:

```
■■■■■■■■■■■■■■■■■■■■■■■■                    ■■■■■■■■■■■■■■■■■■
■  DyalogoCBXMarcador■ ■■ originate ■■■   ■  Asterisk    ■
■  (marca contactos) ■                   ■   PBX       ■
■■■■■■■■■■■■■■■■■■■■■■■■                    ■■■■■■■■■■■■■■■■■■
          ▲                                  ■
          ■                                  ■ AGI call
          ■                                  ▼
```

```
■                                  ■■■■■■■■■■■■■■■■■■■■■■■■
■  ■■■ UPDATE contacto ■■■          ■  DyalogoCBXAgiAmi  ■
■      (via MySQL)                ■  (FastAGI server)  ■
■                                  ■                    ■
■  ■■■ INSERT contacto ■■■          ■  EventoMarcador*   ■
■      (abandonadas→dialer)        ■  HiloProcesa*      ■
■                                  ■■■■■■■■■■■■■■■■■■■■■■■■
■                                           ■
■■■■■■■■■■■ MySQL compartido ■■■■■■■■■■■■■■
```

Flujos de AgiAmi que afectan al Marcador:

Flujo	Descripción	Impacto
Abandoned → Dialer	`QueueCallerAbandonEvent` → `HiloProcesaLlamadasAbandonadas` (cada 60s) → agrupa por campaña → INSERT en `DyMarcadorContactos` si la campaña tiene `idMuestraMarcador`	El marcador recoge estos contactos y los marca como outbound callback
Rejected → Dialer	`AgentRingNoAnswerEvent` → `HiloProcesaLlamadasRechazadas` → misma lógica de re-inyección	Contactos rechazados vuelven a la cola de marcación
AGI status update	`EventoMarcadorContestaHumano/Maquina` → actualiza `DyMarcadorContactos.estado`	El marcador lee este estado para decidir reintentos
Agent monitoring	`HEvaluaIngresoSalidaAgente` → login/logout/pause tracking → email alerts	Marcador predictivo necesita saber agentes disponibles
License enforcement	`ManejadorAgentesConectados` → limita agentes vía `QueueRemoveAction`	Afecta capacidad de recepción de llamadas marcadas

AGI Scripts NO relacionados con Marcador (pero que podrían migrarse después):

Script	Función	Migración
`AGIUsuarioRegistrado`	Carga perfil de usuario/extensión (15+ variables de canal)	Fase futura
`VerificaClaveLlamada`	Valida código de autorización para llamadas salientes manuales	Fase futura
`VerificaDiaFestivo`	Detección de día festivo	Fase futura
`TTSGoogle`	Text-to-Speech vía Google Translate + ffmpeg	Fase futura
`ExcedioLimiteAgentes`	Enforcement de límite de agentes	Fase futura
`DesbordeParaExtensiones`	Overflow routing	Fase futura
`RegistraLogSalasConferencia`	Logging de conferencias	Fase futura

Configuración compartida:

- Los tres proyectos leen de `/Dyalogo/conf/servicios_asterisk.properties` (mismas credenciales AMI y MySQL)
- AgiAmi también lee `/Dyalogo/conf/dyalogo_config_smtp.properties` para alertas por email
- AgiAmi usa una licencia en `/usr/local/lib/DLCAMIAGI.properties`

Diagrama de Dependencias Completo

```
■■■■■■■■■■■■■■■■■■■
```

```
■  DyalogoCBXMarcador   ■   ■  DyalogoCBXAgiAmi           ■
■  (JAR standalone)     ■   ■  (JAR standalone)           ■
■  v1.5.2               ■   ■  v2.0.24                    ■
■                       ■   ■                             ■
■  Marcación outbound   ■   ■  FastAGI server (port 4573) ■
■  Campañas PDS/Pred/Rob■   ■  AMI event listener         ■
■  Originate + Callbacks■   ■  17 AGI scripts             ■
■  CRM integration      ■   ■  Abandoned/Rejected handler ■
■                       ■   ■  Agent monitoring           ■
■■■■■■■■■■■■■■■■■■■■■■■■■■   ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
            ■                           ■
            ■    MySQL (compartido)     ■
            ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
                        ■
                        ▼
              ■■■■■■■■■■■■■■■■■
              ■   Asterisk    ■
              ■   PBX         ■
              ■   (AMI+AGI)   ■
              ■■■■■■■■■■■■■■■■■
```

Decisión Arquitectónica: Alcance de Migración de Proyectos Hermanos

Proyecto	Decisión	Justificación
DyalogoCBXLib	**Extracción parcial** — solo componentes usados por Marcador	La librería sirve a múltiples módulos; migrar todo es innecesario y riesgoso
DyalogoCBXAgiAmi (AGI scripts del marcador)	**Migrar a Asterisk.Sdk.Agi** (FastAgiServer)	Los 4 scripts de marcador deben correr en .NET para eliminar dependencia Java
DyalogoCBXAgiAmi (Abandoned/Rejected)	**Migrar al servicio .NET**	La lógica de re-inyección de contactos abandonados es parte del flujo del marcador
DyalogoCBXAgiAmi (Agent monitoring)	**Reemplazar con Asterisk.Sdk.Live**	`AgentManager` ya provee tracking de login/logout/pause
DyalogoCBXAgiAmi (otros AGI scripts)	**No migrar ahora**	Los 13 scripts restantes no son del marcador; migración futura independiente

---

3. Supuestos Técnicos

Supuestos Asumidos

#	Supuesto	Impacto si es incorrecto
S1	La base de datos MySQL (`dyalogo_telefonia`) seguirá disponible durante la transición	Requeriría migración de datos adicional
S2	El esquema de BD CRM (`DYALOGOCRM_SISTEMA`) es estable y no cambia frecuentemente	Podría requerir adapter pattern
S3	~~La versión de Asterisk en producción es compatible con AMI estándar~~	**CONFIRMADO** — Asterisk 18 en producción, AMI estándar. Plan de migrar a últimas versiones
S4	No existen otros servicios que escriban en las mismas tablas de campañas/contactos de forma concurrente	Condiciones de carrera en la migración paralela
S5	El endpoint REST `/bi/gestion/pdsprerob` es propiedad de la organización y su API es estable	Requeriría wrapper/adapter

#	Supuesto	Impacto si es incorrecto
S6	Las propiedades de configuración (`servicios_asterisk.properties`) representan el contrato completo de configuración	Configuraciones ocultas en variables de entorno o JVM flags
S7	El pool de 50 hilos para llamadas y 20 para eventos es el dimensionamiento óptimo actual	Requeriría benchmarking en .NET para equivalencia
S8	~~La aplicación corre como un solo proceso por instancia de Asterisk~~	**CONFIRMADO** — Un servidor Asterisk actualmente. Futuro: multi-servidor + Kubernetes. IPcom Dialer debe diseñarse stateless para escalar horizontalmente
S9	**DyalogoCBXAgiAmi seguirá corriendo en Java** para los 13 AGI scripts no relacionados con marcador después del corte	Si se apaga AgiAmi, se pierden funcionalidades de otros módulos
S10	**DyalogoCBXLib no será modificada** durante la migración del marcador (otros módulos Java la siguen usando)	Cambios en CBXLib podrían romper compatibilidad
S11	~~El algoritmo de `EncriptadorPropio` es reproducible en .NET~~	**RESUELTO** — AES-ECB con key hardcodeada `D7@l0g0*S.A.S109`, reproducible trivialmente con `System.Security.Cryptography.Aes`
S12	Los **4 AGI scripts del marcador** pueden migrarse al `FastAgiServer` de Asterisk.Sdk sin cambiar el dialplan de Asterisk	Si el dialplan hardcodea IP:puerto del AGI Java, hay que actualizarlo
S13	El **flujo de abandonadas→marcador** (`HiloProcesaLlamadasAbandonadas`) puede replicarse en .NET sin afectar el AgiAmi Java que sigue corriendo	Ambos sistemas no deben re-inyectar el mismo contacto

Información Faltante por Levantar

#	Información Necesaria	Fuente
I1	~~Versión exacta de Asterisk en producción~~	**RESUELTO** — Asterisk 18 en producción. Plan de migrar a últimas versiones. Asterisk.Sdk basado en AMI estándar, compatible con 18+
I2	~~Esquema completo de tablas MySQL~~	**RESUELTO** — 4 DDLs analizados en `/dyalogo/db/Esquemas/`: `ddl_telefonia.sql` (241KB), `ddl_crm.sql` (343KB), `ddl_general.sql` (28KB), `ddl_asterisk.sql` (18KB). Ver sección 4.5
I3	~~Documentación funcional de reglas de negocio de reintentos~~	**RESUELTO** — Algoritmo completo documentado en sección 4.6. 2 reintentos inmediatos/teléfono (hardcoded), max 3 reintentos globales/contacto (configurable), 2,875 reglas por campaña×respuesta en `dy_marcador_respuestas_reintentos`. Callbacks agendados con prioridad y reset de intentos
I4	~~Volúmenes de producción~~	**RESUELTO** — 10-30 campañas simultáneas en producción inicialmente. Modelo SaaS: IPcom alquila la plataforma a clientes que ejecutan una o varias campañas. Se espera crecimiento

#	Información Necesaria	Fuente
I5	~~API contract del endpoint REST CRM~~	**RESUELTO** — POST JSON a `http://{ipServicioCore}:{puerto}/dyalogocore/api/bi/gestion/pdsprerob`. Auth: usuario="local", token="local". Request: campaignId, contactId, agentId (-10), resultCode (1-5), phone, callId. Response: `{strEstado_t: "OK"
I6	~~Otros servicios que escriben en mismas tablas~~	**RESUELTO** — 7 servicios identificados. Tabla `dy_marcador_contactos` tiene 4 escritores concurrentes (Marcador, AgiAmi, CBXLib, dyalogocore). `CONDIA` tiene 3 escritores. `dy_llamadas*` tiene 4 escritores. Sin locking entre servicios. Ver sección 4.10
I7	Requisitos de alta disponibilidad y RPO/RTO actuales	Operaciones
I8	~~Proceso de despliegue~~	**RESUELTO** — IPcom Dialer será **dockerizado** desde el día 1. Futuro: migración a **Kubernetes**. Diseño cloud-native requerido
I9	~~Integración con `DyalogoCBXLib` y `DyalogoCBXAgiAmi` (proyectos hermanos)~~	**RESUELTO** — análisis completo incorporado en sección 2
I10	~~Contextos y extensiones de Asterisk~~	**RESUELTO** — El marcador usa contexto dinámico `DyCampanaMarcador_<campaignId>`, extensión `s`, prioridad `1`. PDS usa contexto de `dy_extensiones.context` (ej: `salida_huesped_20`). Ver sección 4.11
I11	~~**Dialplan de Asterisk**~~	**RESUELTO** — AGI scripts se invocan desde dialplan dinámico generado por dyalogocore. FastAGI Java en puerto 4573. Scripts registrados via `AgiMappingStrategy`. Contextos por campaña: `DyCampanaMarcador_<id>`. Ver sección 4.11
I12	~~**Algoritmo de EncriptadorPropio**~~	**RESUELTO** — AES-ECB, key=`D7@l0g0*S.A.S109`, fallback key2=`{1..16}`, Base64 encoding. Ver sección 4 "Evaluación de Seguridad"
I13	~~**¿Multi-tenant activo?**~~	**RESUELTO** — Confirmado multi-tenant. IPcom alquila la plataforma a múltiples clientes. Cada cliente ejecuta 1+ campañas. Queries deben filtrar por `id_proyecto`/`id_huesped`
I14	**Configuración de campañas con manejo de abandono:** qué campañas tienen `manejoAbandono` y `idMuestraMarcador` activos	DBA / query a producción
I15	~~**LibreriaPersistencia.jar**~~	**RESUELTO** — 9 clases, clase principal `DAOImpOperaciones` (969 líneas). Mayormente boilerplate Hibernate que Dapper resuelve nativamente. Lógica a replicar: audit trail opcional (`IAuditoriaBD`), bulk save, INSERT + LAST_INSERT_ID. Ver sección 4.7

Dependencias Críticas

1. **asterisk-java 3.41.0** → Reemplazado por **Asterisk.Sdk** (basado en asterisk-java 3.42.0-SNAPSHOT)
2. **Hibernate 5.4.4 + C3P0** → Reemplazado por **Dapper + Npgsql** (o MySqlConnector para MySQL)
3. **MySQL Connector 5.1.49** → **MySqlConnector** (ADO.NET, AOT-compatible)
4. **Log4j 1.2.16** → **Serilog** + `System.Diagnostics.Metrics`
5. **Guava 33.3.1** → Funcionalidad nativa de .NET (colecciones concurrentes, caching)
6. **Gson 2.8.6** → **System.Text.Json** (source-generated para AOT)

---

4. Evaluación Inicial del Proyecto Java

Qué Revisar en el Código Fuente

4.1 Artefactos y Estructura

Elemento	Ubicación	Estado
Build system	`build.xml` (Ant)	Legacy — no Maven/Gradle
Configuración JPA	`META-INF/persistence.xml`	Credenciales hardcodeadas
Pool de conexiones	`META-INF/c3p0-config.xml`	min=5, max=100, sobredimensionado
Logging	`log4j.properties`	7 appenders, MDC con campanaId/contactoId/llamadaId
Propiedades externas	`/Dyalogo/conf/servicios_asterisk.properties`	Ruta absoluta en filesystem

4.2 Patrones y Anti-patrones Detectados

Patrones positivos:

- Singletons para estado compartido (`SingletonHilosCampana`, `SingletonGestionesOriginateActivas`)
- Thread naming consistente (`H_` prefix para hilos)
- MDC logging con contexto de campaña/contacto/llamada
- Separación clara DAO/Entidad/Modelo
- Callback pattern para originates asíncronos

Deuda técnica identificada:

#	Deuda	Severidad	Impacto en Migración
DT1	**Credenciales hardcodeadas** en persistence.xml y properties	Alta	Implementar secret management
DT2	**Singletons con estado mutable** compartido entre hilos	Alta	Reemplazar con DI + scoped services
DT3	**Thread management manual** (extends Thread, pools fijos)	Media	Usar `Task`/`Channel<T>`/`BackgroundService`
DT4	**Build con Ant** — sin gestión de dependencias moderno	Media	Eliminado con .NET (NuGet + MSBuild)
DT5	**Hibernate queries** mezcladas con lógica de negocio en DAOs	Media	Separar con repository pattern
DT6	**Delay random (100-800ms)** para evitar thundering herd	Baja	Implementar rate limiter formal
DT7	**Log4j 1.2.16** — versión obsoleta con vulnerabilidades conocidas	Alta	Serilog en .NET

#	Deuda	Severidad	Impacto en Migración
DT8	**MySQL Connector 5.1.49** — versión EOL	Media	MySqlConnector moderno
DT9	**Conexión dual AMI** — complejidad innecesaria en Asterisk.Sdk	Media	Una sola `IAmiConnection`
DT10	**Campos de teléfono 1-10** en contacto (columnas, no normalizado)	Baja	Mantener por compatibilidad de esquema

4.3 Evaluación de Seguridad y Cifrado

El análisis del paquete `dyalogo.cbx.seguridad` de CBXLib revela **5 mecanismos de cifrado**, de los cuales **3 afectan directamente al marcador**:

Clases de cifrado identificadas:

Clase	Algoritmo	Estado	Relevancia para Marcador
`EncriptadorPropio`	AES-ECB, key hardcodeada	**Activo**	**Crítica** — cifra PINs de agente y credenciales CRM
`ManejaEncripcion`	Wrapper: AES (delegado) + XOR custom	**Activo**	**Crítica** — wrapper usado por AGI scripts
`EncriptaSHA`	SHA-256 (hash irreversible)	**Activo**	Baja — autenticación web, no usada por el marcador
`TCSeguridad`	PBKDF2 + AES-CBC	Sin uso	Ninguna
`AcdTEA`	TEA (Tiny Encryption Algorithm)	Sin uso	Ninguna

Detalle de `EncriptadorPropio` (el único que debe migrarse):

```
Algoritmo:  AES (Advanced Encryption Standard)
Modo:       ECB (Electronic Codebook) — sin IV
Key 1:      "D7@l0g0*S.A.S109" (16 bytes, hardcodeada en fuente)
Key 2:      {1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16} (fallback)
Encoding:   Base64
Lógica:     Intenta decrypt con key1; si falla, usa key2
```

Datos cifrados que afectan al marcador:

Dato Cifrado	Tabla MySQL	Columna	Quién lo lee	Uso en Marcador
PIN de salida de llamadas	`dy_agentes`	`clave_salida_llamadas`	AGI `VerificaClaveLlamada`, `ValidadorClaveLlamada`	Los AGI scripts del marcador validan PIN del agente antes de marcar
PIN de salida (extensión)	`dy_extensiones`	`clave_salida_llamadas`	AGI `VerificaClaveLlamada`	Idem
Contraseña CRM	`dy_configuracion_crm`	`contrasena`	`ConexionCRM` del Marcador	El marcador descifra para abrir conexión JDBC al CRM
Contraseña SMTP	`dy_email_smtp_imap`	`password`	`Correo.enviar()` de AgiAmi	Envío de alertas por email de llamadas abandonadas

Equivalente .NET (reproducción exacta):

```
public sealed class LegacyEncryptor
{
    private static readonly byte[] Key = "D7@l0g0*S.A.S109"u8.ToArray();

    public static string Decrypt(string base64Encrypted)
    {
        var encrypted = Convert.FromBase64String(base64Encrypted);
```

```
        using var aes = Aes.Create();
        aes.Key = Key;
        aes.Mode = CipherMode.ECB;
        aes.Padding = PaddingMode.PKCS7;
        using var decryptor = aes.CreateDecryptor();
        var decrypted = decryptor.TransformFinalBlock(encrypted, 0, encrypted.Length);
        return Encoding.UTF8.GetString(decrypted);
    }
}
```

Hallazgos de seguridad:

#	Hallazgo	Severidad	Recomendación para .NET
SEC-1	Key de cifrado hardcodeada en código fuente	**Crítica**	Mover a secret manager (`IOptions<EncryptionOptions>` + env vars)
SEC-2	AES-ECB sin IV — mismo plaintext produce mismo ciphertext	**Alta**	Mantener para compatibilidad; a futuro migrar a AES-CBC/GCM
SEC-3	Credenciales de test en `main()` de `EncriptadorPropio`	**Alta**	No migrar el `main()`; ya eliminado en .NET
SEC-4	XOR custom en `ManejaEncripcion.Encriptar()` — criptográficamente débil	**Media**	No migrar; solo el wrapper AES se usa para el marcador
SEC-5	Contraseña AMI en plaintext en properties	**Media**	Usar secret manager desde el día 1 en .NET
SEC-6	Contraseña SIP por defecto "extdyalogo" en extensiones	**Baja**	No afecta al marcador

Decisión de migración:

- **Fase inmediata:** Replicar AES-ECB con misma key para leer datos existentes en producción
- **Fase de estabilización:** Mover key a secret manager; las credenciales CRM/SMTP migrar a `appsettings.json` cifrado o env vars, eliminando la necesidad de cifrado reversible en BD
- **No migrar:** `EncriptaSHA` (no lo usa el marcador), `TCSeguridad` y `AcdTEA` (sin uso), XOR custom de `ManejaEncripcion`

4.4 Evaluación del Esquema de Base de Datos

Fuente: 4 DDLs en `/dyalogo/db/Esquemas/` — esquemas completos con datos de referencia.

Panorama general:

Base de Datos	Archivo DDL	Tamaño	Tablas	Filas (referencia)	Rol
`dyalogo_telefonia`	`ddl_telefonia.sql`	241 KB	50+	dy_llamadas: 101K, dy_campanas: 2,278	BD principal de telefonía y marcador
`DYALOGOCRM_SISTEMA`	`ddl_crm.sql`	343 KB	100+	CONDIA: 14,554, CAMPAN: 2,551	CRM, gestiones, formularios
`dyalogo_general`	`ddl_general.sql`	28 KB	20+	huespedes: 279, actividad_actual: 1,893	Multi-tenant, config global
`asterisk`	`ddl_asterisk.sql`	18 KB	2 + 8 views	cdr: 57,694, queue_log: 66,498	CDR y queue log de Asterisk

Tablas del marcador (7 tablas core en `dyalogo_telefonia`):

Tabla	Filas	Columnas clave	Operación del marcador
dy_marcador_campanas	42	estado, cantidad_llamadas_simultaneas, timeout, hora_inicial/final, scheduling por día, accion_contesta_humano/maquina, detectar_maquina_contestadora, CRM fields id_campana_crm, id_campo_telefono_crm[1-10]	READ/WRITE — config de campaña
dy_marcador_contactos	(variable)	telefono1-10, telefono_marcado, telefono_efectivo, estado, exitoso, reintentar, intentos, fecha_ultimo_intento, dato_adicional_1-5, url[1-5], agendado, agenda_fecha_hora	READ/WRITE — contactos a marcar
dy_marcador_muestras_campanas	42	nombre_muestra, tipo_origen (File/CRM/FTP), columnas_csv, FTP sync config, prioridad	READ — muestras activas
dy_marcador_log	78,383	id_contacto, telefono_marcado, respuesta, razon, unique_id, fecha_hora	WRITE — logging de cada intento
dy_marcador_efectividad_campanas	42	id_campana_marcador, id_tipificacion_efectiva	READ — tipificaciones exitosas
dy_marcador_respuestas_reintentos	2,875	id_campana_marcador, id_respuesta_originate, tipo_reintento (1=Auto, 2=No Retry)	READ — reglas de reintento
dy_marcador_estados_agentes_externos	—	identificacion_agente, id_campana, estado (1=Available, 2=Paused)	READ — disponibilidad de agentes

Tablas compartidas que el marcador lee (en dyalogo_telefonia):

Tabla	Filas	Para qué la usa el marcador
dy_campanas	2,278	Config base de campaña, manejoAbandono, tipo (sub_tipo_campana: OutPDS/OutPredictivo/OutProgresivo), SLA, audio
dy_agentes	352	Lookup de agente, clave_salida_llamadas (cifrada AES)
dy_extensiones	364	Extensiones SIP/IAX, contexto, tipo_grabacion
dy_troncales	33	Troncales para originate, pbx_distribuido, codigo_antepuesto
dy_rutas_salientes	—	Rutas por patrón, digito_excluir, prefijo, numero_antepuesto, costo_minuto
pasos_troncales	113	Selección de troncal por id_tipos_destino con desborde

Tabla	Filas	Para qué la usa el marcador
`tipos_destino`	372	Patrones de destino: `codigo_antepuesto`, `patron`, `patron_validacion`
`dy_variables_globales`	4	`cantidad_maxima_canales_marcador`, timeouts
`dy_configuracion_crm`	—	Credenciales CRM cifradas (AES)
`dy_respuestas_originate`	—	Mapeo de códigos de respuesta Asterisk
`dy_festivos`	472	Calendario de festivos
`dy_turnos`	—	Definición de turnos con ventanas horarias por día
`dy_llamadas`	101,537	Registro de llamadas (verificación de duplicados)
`dy_llamadas_salientes`	—	Registro de salientes (insert por marcador)

Tablas CRM que el marcador toca (en `DYALOGOCRM_SISTEMA`):

Tabla	Operación	Qué hace el marcador
`CONDIA`	**WRITE**	INSERT de registro de gestión (duración, efectividad, agente, teléfono, unique_id)
`CAMPAN`	READ	Config CRM de la campaña (llamadas simultáneas, aceleración, AMD, scheduling)
`MUESTR`	READ	Muestra/lista de contactos CRM
`GUION_`	READ	Formulario CRM (para obtener campo de tipificación `PREGUN_Tip_b` y campo de reintento `PREGUN_Rep_b`)
`PREGUN`	READ	Definición de campos del formulario CRM
`CAMINC`	READ	Mapeo campo población ↔ campo formulario
`MONOEF`	READ	Tipificaciones de efectividad

Tablas de Asterisk:

Tabla/Vista	Operación	Qué hace el marcador
`cdr`	READ	CDR para verificar resultado de llamadas
`queue_log`	READ	Verificar si una llamada fue abandonada (`SELECT COUNT(1) FROM asterisk.queue_log WHERE callid=...`)
`v_queue_log`	READ	Vista con datos concatenados
`v_queue_log_abandonadas`	READ	Vista de llamadas abandonadas

Hallazgos relevantes del esquema:

#	Hallazgo	Impacto en migración
DB-1	`dy_marcador_contactos` tiene **10 columnas de teléfono** (`telefono1-10`) desnormalizadas + `telefono_marcado` + `telefono_efectivo`	Mantener estructura para compatibilidad; el POCO tendrá 12 campos de teléfono
DB-2	`dy_marcador_campanas` tiene **scheduling granular por día** (lunes_hora_inicial/final, martes_*, etc.) además del horario general	El `CampaignSchedule` debe soportar horarios por día individual

#	Hallazgo	Impacto en migración
DB-3	`dy_marcador_campanas` soporta **festivos con horario propio** (`ejecuta_festivos`, `festivo_hora_inicial/final`)	Agregar lógica de festivos al scheduler
DB-4	`dy_marcador_muestras_campanas` soporta **3 orígenes de datos**: File, CRM, FTP con sync periódico	**RESUELTO** — El marcador **NO implementa FTP sync**. Solo lee contactos ya insertados en `dy_marcador_contactos`. La carga de archivos/FTP la hace la web UI u otro servicio. IPcom Dialer no necesita FTP sync
DB-5	`dy_marcador_contactos` tiene **5 campos** `dato_adicional` y **5 campos** `url` para datos custom	Mantener en POCO
DB-6	`dy_marcador_contactos` tiene campo `agendado` + `agenda_fecha_hora` para **callbacks agendados**	**RESUELTO** — El marcador tiene un hilo `HLlamadasAgenda` que cada 40s busca contactos con `agendado=true AND agenda_fecha_hora <= now()`. Los agendados se priorizan en la cola (se agregan primero) y su contador de intentos se resetea a -1. Después de marcar, se limpia `agendado=false`. IPcom Dialer debe implementar `ScheduledCallbackService`
DB-7	`CAMPAN` en CRM tiene **su propia configuración de marcador** (llamadas simultáneas, aceleración, AMD, scheduling)	**RESUELTO** — Modelo de dos niveles: `dy_marcador_campanas` es la config primaria (scheduling, estado, acciones). `CAMPAN` del CRM se usa **solo para campañas dinámicas** (tipo 6=PDS, 7=Predictivo, 8=Robótico) y aporta aceleración, AMD, llamadas simultáneas. No se mezclan — el path legacy usa `dy_marcador_campanas`, el path dinámico usa `CAMPAN`. IPcom Dialer debe leer ambas tablas según el tipo de campaña
DB-8	`dy_marcador_respuestas_reintentos` tiene **2,875 reglas** (tipo_reintento: 1=Auto, 2=No Retry)	**RESUELTO** — Algoritmo completo documentado: por cada respuesta de originate, se busca la regla por campaña. tipo=1 → reintentar, tipo=2 → no reintentar. Si no hay regla → reintentar por defecto. 2 reintentos inmediatos por teléfono (hardcoded), max 3 reintentos globales por contacto (configurable). Ver sección 4.6
DB-9	`pasos_troncales` + `tipos_destino` implementan un **motor de routing multi-paso** con desborde	El `OutboundRouteResolver` debe replicar esta lógica de pasos
DB-10	Sistema es **multi-tenant** (`id_proyecto`/`id_huesped` en la mayoría de tablas)	**RESUELTO** — Confirmado multi-tenant activo. IPcom alquila la plataforma a clientes. Todas las queries deben filtrar por `id_proyecto`/`id_huesped`

4.6 Algoritmo de Reintentos (Investigación de Código)

Fuente: Análisis directo del código Java de `HEjecutaMarcacionContacto`, `ActualizaIntentosContactoLog`, `HMarcacionDinamicoCampanaMaster`, `DYOriginateCallback`.

Flujo completo de marcación de un contacto:

```
Contacto encolado para marcación
  ■
  ■■ ¿Llamado en últimos 10 minutos? → SÍ: Saltar
  ■ NO
  ■■ Espera random 100-800ms (anti-thundering-herd)
```

```
■
■■ FOR cada teléfono (telefono1 a telefono10):
■   ■■ FOR 2 reintentos inmediatos (hardcoded):
■   ■   ■■ ORIGINATE → Asterisk AMI
■   ■   ■■ Espera callback (polling 1.5s)
■   ■   ■■ Código 2 (CONTESTADA) → Fin, cerrar contacto ✓
■   ■   ■■ Código 5 (FALLIDA) → Intentar troncal de respaldo
■   ■   ■■ Otros (3/4/101/102) → Consultar dy_marcador_respuestas_reintentos:
■   ■   ■   ■■ tipo_reintento=1 → Reintentar (continuar loop)
■   ■   ■   ■■ tipo_reintento=2 → No reintentar, pasar al siguiente teléfono
■   ■   ■   ■■ Sin regla → Reintentar por defecto
■   ■   ■■ Sleep 2s entre reintentos inmediatos
■   ■■ Sleep 3s entre teléfonos
■   ■■ Avanzar al siguiente teléfono
■
■■ Después de agotar todos los teléfonos:
■   ■■ contacto.intentos++
■   ■■ ¿contacto.intentos &gt; campaña.LimiReinb (default 3)?
■   ■   ■■ SÍ → Marcar contacto como "gestionado", excluir de futuros intentos ✗
■   ■   ■■ NO → Contacto elegible para futuro reintento
■   ■■ Cerrar hilo del contacto
```

Códigos de respuesta de originate:

Código	Respuesta	Comportamiento por defecto
1	MARCANDO	En progreso (no aplica)
2	CONTESTADA	NO reintentar (exitoso)
3	NO_CONTESTADA	Reintentar
4	OCUPADA	Reintentar
5	FALLIDA	Reintentar (con troncal de respaldo)
101	Rechazada por troncal	Reintentar
102	Sin canales libres	Suspender marcación de la campaña

Parámetros de configuración:

Parámetro	Tabla	Default	Descripción
`max reintentos por teléfono`	hardcoded en `HEjecutaMarcacionContacto`	**2**	Reintentos inmediatos por número
`delay entre reintentos`	hardcoded	**2s**	Sleep entre reintentos del mismo número
`delay entre teléfonos`	hardcoded	**3s**	Sleep al avanzar al siguiente teléfono
`delay inicial random`	hardcoded	**100-800ms**	Anti-thundering-herd
`LimiReinb`	`dy_marcador_campanas` / `CAMPAN`	**3**	Max reintentos globales por contacto
`TieLimReib`	`dy_marcador_campanas` / `CAMPAN`	**-1**	-1 = aplicar límite, otro = sin límite
`tipo_reintento`	`dy_marcador_respuestas_reintentos`	—	1=Auto, 2=No Retry, por campaña × respuesta

Callbacks agendados (HLlamadasAgenda):

Aspecto	Valor
Polling interval	cada **40 segundos**
Query	`agendado=true AND agenda_fecha_hora <= NOW()`

Aspecto	Valor
Prioridad	Agendados se agregan **al inicio** de la cola (antes que contactos regulares)
Reset de intentos	`contacto.intentos = -1` (reset completo, máximos reintentos disponibles)
Post-marcación	`agendado=false`, `agenda_fecha_hora=null`

Modelo de configuración de campaña (dos niveles):

```
■■■
```

Decisión: La mayoría de la funcionalidad del base DAO es Hibernate boilerplate que Dapper resuelve nativamente. Solo el **audit trail** y el **bulk save** requieren implementación explícita.

4.9 API REST CRM — Contract Completo (Investigación de Código)

Fuente: Análisis directo de `ConsumeREST.java`, `HTTPRequest.java`, `JAXRSGestionaComunicacion.java`.

Endpoint: `POST http://{ipServicioCore}:{puerto}/dyalogocore/api/bi/gestion/pdsprerob`

Configuración de URL base:

- Propiedades: `/etc/dyalogo/cbx/conf/parametros_generales.properties`
- Keys: `ipServicioCore` (default: `127.0.0.1`), `intPuertoCore` (default: `8080`)
- Template: `http://{ip}:{port}/dyalogocore/api`

Request (JSON POST):

```
{
  "strUsuario_t": "local",
  "strToken_t": "local",
  "intConsInteCAMPAN_t": 42,
  "intConsInte_t": 1234,
  "intIdUsuarioAgente_t": -10,
  "intResultadoMarcacion_t": 3,
  "strTelefono_t": "573001234567",
  "strIdLlamada_t": "abc-123-def"
}
```

Campo	Tipo	Descripción	Valor típico
`strUsuario_t`	string	Usuario autenticación	`"local"` (hardcoded)
`strToken_t`	string	Token autenticación	`"local"` (hardcoded)
`intConsInteCAMPAN_t`	int	ID campaña CRM (`CAMPAN.CAMPAN_ConsInte__b`)	FK de `dy_marcador_campanas.id_campana_crm`
`intConsInte_t`	int	ID contacto/registro en muestra CRM	ID del contacto
`intIdUsuarioAgente_t`	int	ID agente	`-10` (sistema, no agente humano)
`intResultadoMarcacion_t`	int	Código resultado	1=Marcando, 2=Contestada, 3=No contestada, 4=Ocupada, 5=Fallida
`strTelefono_t`	string	Teléfono marcado	Número completo
`strIdLlamada_t`	string	UniqueID de la llamada (Asterisk)	UUID del originate

Response (JSON):

```
{
  "strEstado_t": "OK",
  "strMensaje_t": "Comunicacion procesada",
  "objSerializar_t": null
}
```

Campo	Valores	Descripción
`strEstado_t`	`"OK"` / `"FALLO"`	Resultado de la operación
`strMensaje_t`	string	Mensaje descriptivo
`objSerializar_t`	null/object	Datos adicionales (normalmente null)

Cuándo se invoca:

1. Resultado > 2 (No Contestada/Ocupada/Fallida): inmediatamente

2. Resultado = 2 (Contestada) + campaña robótica: inmediatamente
3. Resultado = 2 sin robótica: después de 1 minuto (verifica CDR para determinar si llegó a agente)

Manejo de errores: Fallos se loguean pero **no se reintentan**. HTTP >= 400 logueado como ERROR.

Equivalente .NET:

```
public interface ICrmRestClient
{
    Task&lt;CrmResponse&gt; UpdateContactResultAsync(CrmContactUpdate update, CancellationToken ct);
}

// Implementación con IHttpClientFactory + typed client
// Retry policy con Polly si se desea mejorar respecto al Java
```

4.10 Servicios Concurrentes en Tablas Compartidas (Investigación de Código)

Fuente: Análisis de todos los proyectos en `/dyalogo/java/dy_jee/`.

Matriz de escrituras cruzadas:

Tabla	Marcador	AgiAmi	ColectorDatos	CBXLib	Web (cbx)	Core	BI
`dy_marcador_contactos`	**UPDATE**	**INSERT**	—	**DELETE**	—	**UPDATE**	—
`dy_marcador_log`	**INSERT**	—	—	—	—	—	—
`dy_marcador_muestras`	INSERT	—	—	**UPDATE**	—	—	—
`dy_llamadas`	—	—	**INSERT/UPDATE/DELETE**	—	—	—	—
`dy_llamadas_salientes`	**INSERT**	—	**INSERT/UPDATE/DELETE**	—	**SP**	—	—
`CONDIA` (CRM)	**INSERT/UPDATE**	—	—	—	—	**INSERT/UPDATE**	**UPDATE**

Riesgos de concurrencia identificados:

#	Riesgo	Severidad	Impacto en IPcom Dialer
CC-1	`dy_marcador_contactos` tiene **4 escritores** sin locking: Marcador (UPDATE estado), AgiAmi (INSERT abandonadas), CBXLib (DELETE al eliminar muestra), Core (UPDATE sync CRM)	**Crítico**	IPcom Dialer debe usar transacciones con `SELECT ... FOR UPDATE` al actualizar contactos para evitar condiciones de carrera
CC-2	`CONDIA` tiene **3 escritores** con SQL dinámico (riesgo de SQL injection en Java original)	**Alto**	Usar Dapper parameterizado (ya seguro). Verificar que no hay conflictos de ID
CC-3	`dy_llamadas_salientes` tiene **4 escritores** incluyendo un stored procedure (`sp_inserta_llamada_prev_prog`)	**Medio**	IPcom Dialer solo INSERT; el SP de la web y el ColectorDatos son independientes
CC-4	Sin control de concurrencia optimista (no hay campos `version` o `updated_at` en las tablas)	**Alto**	Considerar agregar idempotencia basada en `unique_id` de Asterisk para evitar duplicados

Servicios que siguen corriendo en Java después del corte:

Servicio	Tablas que toca	Impacto
DyalogoCBXAgiAmi	`dy_marcador_contactos` (INSERT abandonadas)	IPcom Dialer asume esta función — **desactivar en AgiAmi**
DyalogoCBXColectorDatos	`dy_llamadas*` (INSERT/UPDATE/DELETE), limpieza periódica	Sigue corriendo sin cambios — no interfiere
dyalogocbx (Web)	`dy_llamadas_salientes` (SP), gestión de campañas/muestras	Sigue corriendo — coordinar con IPcom Dialer
dyalogocore	`dy_marcador_contactos` (sync CRM), `CONDIA`	Sigue corriendo — **riesgo de conflicto en contactos**
DyalogoCBXBI	`CONDIA` (UPDATE analytics)	Sigue corriendo sin cambios

Recomendación para IPcom Dialer:

- Usar `SELECT ... FOR UPDATE` en queries críticas de `dy_marcador_contactos`
- Implementar idempotencia basada en `unique_id` para evitar duplicados
- Verificar con `WHERE estado_cod = @expected` antes de UPDATE (optimistic locking informal)

4.11 Dialplan de Asterisk y AGI (Investigación de Código)

Fuente: Análisis de `extensions_llamadasapi.conf`, `HCampanaMarcacion.java`, `HAccionesAsteriskAMI.java`, `CBXAgiAmi.java`.

Contextos utilizados por el marcador:

Contexto	Formato	Origen	Uso
`DyCampanaMarcador_<id>`	Dinámico por campaña	`HCampanaMarcacion.java` línea 391	Contexto principal de originate — el dialplan de producción debe tener una sección por cada campaña activa
`salida_huesped_<id>`	Dinámico por tenant	`dy_extensiones.context`	PDS — contexto de la extensión del agente
`DyLlamadaPDS`	Estático	`extensions_llamadasapi.conf`	Contexto para llamadas PDS con validación de extensión

Parámetros de OriginateAction:

```
// HAccionesAsteriskAMI.java líneas 60-66
originateAction.setChannel(objDatEnvLla_t.getChannel());       // "SIP/troncal/numero"
originateAction.setContext("DyCampanaMarcador_" + campaignId);  // Contexto dinámico
originateAction.setExten("s");                                   // Extensión "s" (system)
originateAction.setPriority(1);                                  // Prioridad 1
originateAction.setCallerId(name + " - " + number);            // CallerID
```

Estructura del dialplan por campaña (inferida del código):

```
[DyCampanaMarcador_42]
; Contexto generado dinámicamente para campaña 42
exten =&gt; s,1,NoOp(Campaña marcador 42)
 same =&gt; n,Answer()
 ; Si contesta humano:
 same =&gt; n,AGI(agi://localhost:4573/EventoMarcadorContestaHumano.agi?...)
 ; Si detecta máquina (AMD):
 ; same =&gt; n,AGI(agi://localhost:4573/EventoMarcadorContestaMaquina.agi?...)
 same =&gt; n,Queue(cola_campana_42,...)
 same =&gt; n,Hangup()
```

FastAGI — Scripts del marcador:

Script Java	Path AGI	Acción	Variables de canal
`EventoMarcadorContestaHumano`	`EventoMarcadorContestaHumano.agi`	UPDATE contacto a "Contestada"	Lee `DY_AGI_*`

Script Java	Path AGI	Acción	Variables de canal
`EventoMarcadorContestaMaquina`	`EventoMarcadorContestaMaquina.agi`	UPDATE contacto estado_cod=7	Lee `DY_AGI_*`
`InsertaRegistroMuestraMarcador`	`inserta_muestra_marcador.agi?tel=&cont=&ori=&idm=`	INSERT en muestra	Params en URL
`EnviaRespuestaAMDAPI`	`EnviaRespuestaAMDAPI.agi`	Procesa resultado AMD	Lee `AMDSTATUS`

Configuración de FastAGI:

- Puerto Java: **4573** (default de `DefaultAgiServer` de asterisk-java)
- Puerto IPcom Dialer: **4574** (configurado en Docker)
- Scripts registrados via `AgiMappingStrategy` (mapeo nombre → clase)

Hallazgo crítico: Los contextos `DyCampanaMarcador_<id>` son **dinámicos** — se generan en el dialplan de Asterisk por `dyalogocore`. Esto significa que el cambio de AGI en el dialplan no es un simple find-replace de puerto, sino que hay que modificar el **generador de dialplan** en `dyalogocore` para que las nuevas campañas apunten al puerto 4574 del IPcom Dialer.

Implicación para el corte:

```
Antes del corte:
[DyCampanaMarcador_42]
exten =&gt; s,n,AGI(agi://localhost:4573/EventoMarcadorContestaHumano.agi)

Después del corte:
[DyCampanaMarcador_42]
exten =&gt; s,n,AGI(agi://localhost:4574/EventoMarcadorContestaHumano.agi)
```

El cambio debe hacerse en el **generador de contextos** de `dyalogocore` (servicio Java que sigue corriendo) o directamente en la configuración de Asterisk si los contextos son estáticos en un archivo `.conf`.

4.12 Configuración de Producción de Asterisk (Análisis de `/etc/asterisk/`)

Fuente: Configuración real de producción en `/dyalogo/asterisk/etc/asterisk/`.

Hallazgo principal: La configuración de producción confirma y amplía significativamente lo inferido del código Java. Se identifican 57 contextos de marcador activos, configuración AMD real, credenciales AMI, networking NAT/TLS, y el servidor AGI real en `172.18.0.2:5000` (no `localhost:4573` como se asumía).

a) Usuarios AMI de producción (`manager.conf`):

Usuario	Secret	Permisos Read	Permisos Write	Uso
`admin`	`dyalogo`	system,call,log,verbose,agent,user,config,dtmf,reporting,cdr,dialplan	system,call,agent,user,config,command,reporting,originate	Administración general
`dyalogoami`	`dyalogo*`	system,call,log,verbose,command,agent,user,config,cdr,originate	system,log,verbose,command,agent,user,config,cdr,call,originate	**Marcador** (originate)
`dyamievt`	`dy4l0g0`	all	(solo lectura)	Monitor de eventos
`dyalogoamilan`	`dyalogo*lan`	(igual a dyalogoami)	(igual a dyalogoami)	Acceso LAN

Implicación: IPcom Dialer debe conectarse como `dyalogoami` (o crear un usuario dedicado `ipcomdialer`). El writetimeout=5000 de `admin` es relevante para operaciones lentas. Todas las credenciales deben migrar a secrets.

b) Dialplan de producción (`extensions.conf`):

```
#include /etc/asterisk/marcador/*    &lt;-- 57 archivos DyCampanaMarcador_*.conf
```

Variables globales relevantes para el marcador:

- `DY_IP_AGI_SERVER=127.0.0.1` — IP del servidor AGI (pero los contextos reales usan `172.18.0.2:5000`)

- `DY_NOMBRE_CLIENTE=CBXv3`
- `DY_CANTIDAD_MAXIMA_GANALES_MARCADOR_G=20` — Limite global de canales del marcador

c) Contextos de marcador reales — 2 patrones identificados:

Patrón 1: Sin AMD (35 campañas) — Directo a cola ACD:

```
[DyCampanaMarcador_10]
  exten =&gt; s,1,Answer()
  exten =&gt; s,n,NOOP(Llamada marcador V1 ...)
  exten =&gt; s,n,SIPAddHeader(Call-InfoCAM: sip:\;id_campana=10)
  exten =&gt; s,n,SIPAddHeader(Call-InfoURL1-5: ...)        ; URLs de CRM
  exten =&gt; s,n,SIPAddHeader(Call-InfoUID: ...${UNIQUEID})
  exten =&gt; s,n,SIPAddHeader(Call-AccDef: ...accion=cola_acd_2178)
  exten =&gt; s,n,SIPAddHeader(Call-AccHum: ...accion=null)  ; Sin accion humano
  exten =&gt; s,n,SIPAddHeader(Call-AccMaq: ...accion=null)  ; Sin accion maquina
  exten =&gt; s,n,SIPAddHeader(Call-InfoCID: ...contacto_id=...)
  exten =&gt; s,n,SIPAddHeader(Call-Autocontestar: ...autocontestado=true)
  exten =&gt; s,n,Set(CANTIDAD_AGENTES_LISTOS=${QUEUE_MEMBER(cola_acd_2178,ready)})
  exten =&gt; s,n,ExecIf($["${CANTIDAD_AGENTES_LISTOS}"="0"]?Hangup(2000))
  exten =&gt; s,n,Macro(Grabacion)
  exten =&gt; s,n,Set(__DatoAdicionalCIDName=...)
  exten =&gt; s,n,Set(CALLERID(num)=${DY_MARCADOR_TELEFONO_MARCADO})
  exten =&gt; s,n,Goto(DyPMCampana_2178,s,1)
  exten =&gt; h,1,Hangup(${HANGUPCAUSE})
```

Patrón 2: Con AMD (22 campañas) — Detección de contestadora + AGI:

```
[DyCampanaMarcador_2]
  exten =&gt; s,1,Answer()
  exten =&gt; s,n,Playback(s200ms, m(10))                   ; Silencio breve pre-AMD
  exten =&gt; s,n,NOOP(Llamada marcador V1 ...)
  exten =&gt; s,n,SIPAddHeader(...)                          ; Mismos headers
  exten =&gt; s,n,SIPAddHeader(Call-AccHum: ...accion=cola_acd_2138)  ; Accion si humano
  exten =&gt; s,n,Macro(Grabacion)
  exten =&gt; s,n,NoOp(Iniciando AMD para la campana ...)
  exten =&gt; s,n,AMD()                                      ; Deteccion de maquina
  exten =&gt; s,n,Noop(AMD RES =&gt; ${AMDSTATUS},${AMDCAUSE})
  exten =&gt; s,n,GotoIf($["${AMDSTATUS}" = "HUMAN"]?humano,1)
  exten =&gt; s,n,GotoIf($["${AMDSTATUS}" = "NOTSURE"]?humano,1)   ; NOTSURE = humano
  exten =&gt; s,n,GotoIf($["${AMDSTATUS}" = "UNKNOWN"]?humano,1)   ; UNKNOWN = humano
  ; Si llega aqui = MACHINE:
  exten =&gt; s,n,Agi(agi://172.18.0.2:5000/AGIClasificaAM.agi?idContacto=...&amp;idCampan=...&amp;accam=
-1&amp;valaccam=null)
  exten =&gt; s,n,Hangup(${HANGUPCAUSE})

  exten =&gt; humano,1,NoOp(Llamada contestada por un humano)
  exten =&gt; humano,n,Set(CALLERID(num/name)=...)
  exten =&gt; humano,n,Goto(DyPMCampana_2138,s,1)            ; Va a cola ACD
  exten =&gt; h,1,Hangup(${HANGUPCAUSE})
```

d) Variables de canal del marcador (12 variables):

Variable	Origen	Uso en dialplan
`DY_MARCADOR_OPCION`	Originate	Tipo de operación
`DY_MARCADOR_IDCLIENTE`	Originate	ID del cliente/tenant
`DY_MARCADOR_TELEFONO_MARCADO`	Originate	Número marcado (se usa como CallerID)
`DY_MARCADOR_ID_CONTACTO`	Originate	ID del contacto en BD
`DY_MARCADOR_URL`	Originate	URL principal CRM
`DY_MARCADOR_URL1` a `URL5`	Originate	URLs adicionales CRM (headers SIP)
`DY_MARCADOR_CONSINTE`	Originate	Consecutivo de intento
`DY_MARCADOR_CONSINTE_CAMPAN_CRM`	Originate	Consecutivo CRM de campaña

Implicacion para IPcom Dialer: El `OriginateAction` debe setear estas 12 variables como channel variables para que el dialplan funcione correctamente. Esto se hace via `setVariable()` en el originate.

e) Configuración AMD de producción (`amd.conf`):

Parametro	Valor produccion	Backup (20260217)	Descripcion
`total_analysis_time`	8000ms	5000ms	Tiempo maximo analisis
`silence_threshold`	256	256	Umbral de silencio
`initial_silence`	2500ms	2500ms	Silencio inicial max
`after_greeting_silence`	800ms	800ms	Silencio post-saludo
`greeting`	1300ms	1500ms	Duracion max saludo
`min_word_length`	90ms	100ms	Duracion min palabra
`between_words_silence`	40ms	50ms	Silencio entre palabras
`maximum_number_of_words`	2	3	Max palabras antes de MACHINE
`maximum_word_length`	5000ms	(no definido)	Duracion max palabra

Hallazgo: Existe un archivo backup `amd.conf-20260217` con parametros mas conservadores. Esto indica ajuste activo de AMD en produccion. IPcom Dialer debe exponer estos parametros como configuracion por campaña (no global).

f) Servidor AGI real:

- **IP produccion:** `172.18.0.2:5000` (NO `localhost:4573` como indica el default de asterisk-java)
- **Script unico del marcador:** `AGIClasificaAM.agi` (clasificacion de contestadora automatica)
- Solo se invoca en campañas con AMD activo (22 de 57)
- Los scripts `EventoMarcadorContestaHumano.agi` y `EventoMarcadorContestaMaquina.agi` **no aparecen** en los contextos de produccion — la logica se resuelve directamente en dialplan con `GotoIf` + headers SIP

Correccion critica: La seccion 4.11 asumia 4 scripts AGI del marcador basandose en el codigo Java. En produccion real, **solo se usa 1 script AGI** (`AGIClasificaAM.agi`) y la logica de humano/maquina se maneja en el dialplan con Goto. IPcom Dialer necesita implementar solo este script en FastAGI.

Servidor AGI real: El script `AGIClasificaAM.agi` NO corre en `DyalogoCBXAgiAmi` (puerto 4573) sino en `dy_gwivrstrx` (Gateway IVR Transaccional, puerto 5000). Es un EJB `@Singleton` `@Startup` que inicia `DefaultAgiServer` en un WAR JavaEE. El mapping está en `fastagi-mapping.properties`:

```
AGIGW.agi=com.dyalogo.gwivr.agi.AGIGW
AGIClasificaAM.agi=com.dyalogo.gwivr.agi.AGIClasificaMaquinaContestadora
```

Logica del script `AGIClasificaMaquinaContestadora` (93 lineas):

1. Lee parametros del request AGI: `idContacto`, `idCampan`, `accam`, `valaccam`
2. Log del evento: contacto, campaña, CallerID, UniqueID
3. **Llama al CRM REST API** (`/bi/gestion/pdsprerob`) con `intResultadoMarcacion_t=6` (código "máquina contestadora"):

   ```
   {
     "strUsuario_t": "local",
     "strToken_t": "local",
     "intConsInteCAMPAN_t": "&lt;idCampan&gt;",
     "intConsInte_t": "&lt;idContacto&gt;",
     "intIdUsuarioAgente_t": "-10",
     "intResultadoMarcacion_t": "6",
     "strTelefono_t": "&lt;callerIdNumber&gt;",
     "strIdLlamada_t": "&lt;uniqueId&gt;"
   }
   ```

4. Si `accam=2`: reproduce audio de la máquina (3x silencio + audio del parámetro)

5. Si `accam=1` o `-1`: no hace nada adicional (solo la actualización CRM)

Configuracion via `/etc/dyalogo/cbx/conf/parametros_generales.properties`:

Propiedad	Uso	Default
`direccionIpAmi`	IP servidor Asterisk AMI	-
`usuario` / `contrasena`	Credenciales AMI	-
`direccionIpBd` / `usuarioBd` / `contrasenaBd`	Conexion MySQL	-
`ipServicioCore`	IP de dyalogocore (para REST API)	`127.0.0.1`
`intPuertoCore`	Puerto de dyalogocore	`8080`
`ipServiciosAdicionales`	IP servicios adicionales (usada en dialplan)	`127.0.0.1`
`usarHTTPS`	Usar HTTPS para API calls	`false`
`blendActivo`	Blending inbound/outbound activo	`false`
`tecnologiaUsuarios`	`SIP` o `PJSIP`	`SIP`
`usarSubrutinas`	Gosub vs Macro en dialplan	`false`

Implicacion para IPcom Dialer: El script AGI es trivial — solo hace un HTTP POST al CRM y opcionalmente reproduce un audio. En .NET se implementa como un handler de ~20 lineas registrado en `FastAgiServer`.

g) Networking y seguridad (`sip.conf`, `http.conf`, `rtp.conf`):

Aspecto	Configuracion	Implicacion IPcom Dialer
Protocolo SIP	`chan_sip` (NO pjsip — deshabilitado en modules.conf)	Compatible con Asterisk 18. Migrar a pjsip para Asterisk 20+
NAT	`externaddr=34.63.181.35`, `localnet=172.18.0.0/24`, `nat=force_rport,comedia`	Infraestructura en GCP (IP 34.x.x.x)
TLS	Habilitado en SIP (5061) y HTTP (8089), certs en `/etc/asterisk/keys/`	IPcom Dialer debe usar WSS/TLS para ARI si se usa
WebRTC	`websocket_enabled=true`, `avpf=yes`, `icesupport=yes`	Webphone activo en produccion
STUN	`stunaddr=stun.dyalogo.cloud:3478`	Necesita actualizarse a dominio IPcom
Extern host	`aipcom360.ai`	Ya migrado a dominio IPcom
RTP ports	`10000-65000`	Rango amplio para alto volumen
Codecs	Solo `ulaw` (G.711u)	Codec unico simplifica transcoding
HTTP	Puerto 8088 (HTTP) + 8089 (HTTPS/TLS), `sessionlimit=10000`	ARI disponible en ambos puertos
ARI user	`dyalogoari` / `Dy4l0g04Ry`	Disponible si IPcom Dialer necesita ARI
CDR	CSV + queue_log habilitados, `unanswered=yes`	CDRs registran todas las llamadas incluyendo no contestadas
Grabacion	`recordingformat=wav`, `Macro(Grabacion)` en todos los contextos	Todas las llamadas del marcador se graban

h) Estadisticas de produccion (57 contextos):

Metrica	Valor
Total contextos marcador	**57**
Campañas con AMD	**22** (39%)

Metrica	Valor
Campañas sin AMD (directo a cola)	**35** (61%)
Campañas con `Playback` pre-AMD	**22** (todas las de AMD)
Campañas que verifican agentes disponibles (`QUEUE_MEMBER`)	**30** (53%)
Autocontestado activo	**57** (100%) — todas
Accion maquina (`AccMaq`) distinta de null	**0** — ninguna usa accion especial para maquina
Accion humano (`AccHum`) con cola ACD	**22** — las que tienen AMD redirigen a cola si humano
Scripts AGI invocados	**1** (`AGIClasificaAM.agi` en `172.18.0.2:5000`)
Limite global canales marcador	**20** (`DY_CANTIDAD_MAXIMA_GANALES_MARCADOR_G`)

i) Implicaciones para la migracion:

1. **Solo 1 script AGI necesario** (no 4): `AGIClasificaAM.agi` — clasifica resultado AMD en BD
2. **El dialplan no necesita modificarse** para el marcador — IPcom Dialer genera los originates con las mismas 12 variables de canal y el dialplan existente funciona transparentemente
3. **El AGI se invoca desde Asterisk a** `172.18.0.2:5000` — para la migracion, IPcom Dialer debe escuchar en esa IP:puerto (o configurar una IP/puerto alternativa y actualizar los contextos)
4. **AMD es nativo de Asterisk** (`AMD()` en dialplan) — IPcom Dialer no necesita implementar AMD, solo generar el dialplan correcto y manejar el resultado via AGI
5. `QUEUE_MEMBER(cola,ready)` **check** — 30 campañas verifican agentes disponibles antes de conectar; el marcador hace lo mismo via AMI antes de originar, pero el dialplan tiene una segunda verificacion como safety net
6. **Grabacion universal** — `Macro(Grabacion)` en todos los contextos, no requiere logica en IPcom Dialer
7. **Headers SIP custom** — 13 headers `SIPAddHeader` por llamada transportan metadata (campaña, URLs, contacto, autocontestado). Estos headers llegan al softphone/webphone para mostrar informacion al agente

4.13 Mapa Completo del Ecosistema Dyalogo/IPcom (Análisis Exhaustivo)

Fuente: Análisis exhaustivo de `/media/Data/Source/OrionSoft/dyalogo/` — 23 proyectos Java, 7 servicios Node.js, aplicación PHP/CRM, infraestructura Terraform/GCP, 368 tablas MySQL, 110+ archivos de configuración Asterisk.

a) Inventario de Servicios del Ecosistema

Servicios Java (23 proyectos en `/java/dy_jee/`):

Servicio	Tipo	Puerto	Función	Interacción Asterisk	Interacción Marcador
DyalogoCBXMarcador	JAR standalone	-	Marcador outbound (PDS/estático)	AMI: Originate, QueueStatus	**ES el marcador**
DyalogoCBXAgiAmi	JAR standalone	4573 (AGI)	FastAGI server + AMI listener, abandonadas	AMI+AGI: 50+ scripts	Lee/escribe tablas llamadas
dy_gwivrstrx	WAR	5000 (AGI)	Gateway IVR, `AGIClasificaAM.agi`	AGI: 2 scripts	Clasifica AMD via CRM API
dyalogocore	WAR	8080	Hub central: AMI, config, chat, VoIP	AMI directo (EJB)	Genera dialplan marcador
dyalogocbx	WAR	8080/8181	Desktop agente (JSF/PrimeFaces)	Indirecto (via core)	Lee estados agente
DyalogoCBXColectorDatos	JAR standalone	-	ETL de CDR, serializa llamadas	Consume CDR	Lee `dy_llamadas_*`

Servicio	Tipo	Puerto	Función	Interacción Asterisk	Interacción Marcador
DyalogoCBXBI	EJB	-	Business Intelligence, reportes auto	No	Reportes marcador
dy_distribuidor_trabajo	WAR	8080	Distribucion de trabajo a agentes	No	No
dy_servicios_adicionales	WAR	8080	Erlang, leads, actividad	No	No
dy_procesador_ce	WAR	8080	Procesador correo electronico	No	No
dy_cbx_detallado_llamadas	WAR	8080	Detalle/export CSV llamadas	No	Lee CDR
dy_soporte	WAR	8080	Portal soporte/admin	No	No
dy_public_front	WAR	8080	Portal publico, Click-to-Call	No	No
DyImportaExcel	JAR CLI	-	Import masivo Excel→BD	No	Carga contactos
DyalogoCBXLib	JAR lib	-	Libreria compartida (config, crypto, WS)	-	-
LibreriaPersistencia	JAR lib	-	DAO base, ORM Hibernate	-	-
dy-asterisk-java	JAR lib	-	Fork asterisk-java 3.41.0	-	-
DyLibEJBRemote	JAR lib	-	Interfaces EJB remotas	-	-
dyisolib	JAR lib	-	Objetos dominio aislados	-	-
DyalogoCBXLicencia	JAR lib	-	Gestion licencias	-	-
dyjmsq	JAR/WAR	-	Cola mensajes JMS	-	-
cloudrun/emails	Spring Boot	8080	Microservicio email (SendGrid)	No	No
cloudrun/infobip	Spring Boot	8080	Microservicio SMS (Infobip)	No	No

Servicios Node.js (7 proyectos en `/nodejs/`):

Servicio	Puerto	Función	BD
ccaas_mdw_gupshup_sms	8080	Middleware WhatsApp/SMS/FB/Instagram	Cloud SQL `bd-ccaas-singleton`
dysingleton	8080	Config centralizada (key-value)	Cloud SQL `bd-ccaas-singleton`
dysmsmw	8080	Agregador SMS (Masiv, Twilio, iaTech, Handy)	Cloud SQL `bd-ccaas-singleton`
dy_msintegration	3000	Integracion Microsoft 365 email	Cloud SQL `bd-ccaas-singleton`
dy_journey	80	Customer journey tracking	Cloud SQL
api_reportes	3000	Generacion reportes Excel	MySQL dinamico
dy_ia	-	(vacio/placeholder)	-

Aplicacion PHP/CRM:

Modulo	Ruta	Función
Manager	`/manager/`	Panel admin: colas, metricas, agentes, config
Backoffice (CRM)	`/crm_php/`	Interface agente: gestiones, formularios, scripts
Admin	`/admin/`	Laravel: tenants, troncales, canales, webhooks
QA	`/QA/`	Monitoreo calidad
WhatsApp	`/whatsapp/`	Integracion plantillas WhatsApp
AGIs	`/agis/`	Scripts AGI PHP (inserta_muestra_marcador, etc.)

b) Infraestructura de Produccion (GCP)

Recurso	Detalle
Cloud	Google Cloud Platform, region `us-central1-b`
VPC	`dyalogo-net`, CIDR `172.18.0.0/20`, routing global
MySQL principal	`bd-ccaas-hermes`: 2 CPU, 8GB RAM, 10GB SSD, MySQL 8.0, timezone -05:00
MySQL singleton	`bd-ccaas-singleton`: 2 CPU, 8GB RAM, 10GB SSD, MySQL 8.0
Acceso	VPN Ipcom (`185.32.76.153/32`), SSH via IAP (`35.235.240.0/20`)
Cloud Run	`us-east1`: emails, infobip, dy_msintegration, ccaas_mdw
IP publica	`34.63.181.35` (Asterisk), dominio `aipcom360.ai`
Backups	7 dias retencion, transaction log habilitado
Grabaciones	`/mnt/disks/dy_grabaciones/` (disco montado)

c) Base de Datos — 368 Tablas en 4+ Esquemas

Esquema	Tablas	Servicios que acceden	Uso principal
`dyalogo_telefonia`	120	Marcador, AgiAmi, Colector, Core, CBX, BI, PHP	Agentes, campañas, colas, rutas, IVRs, marcador
`DYALOGOCRM_SISTEMA`	224	CBX, PHP CRM, Core	Contactos, gestiones, estrategias, scripts
`dyalogo_general`	22	Core, PHP Manager, BI	Tenants, alertas, config, reportes
`asterisk`	2	Colector, Asterisk CDR	CDR (57K+), queue_log (66K+)
`bd-ccaas-singleton`	~20	Node.js (5 servicios)	Canales, SMS, config singleton
`dy_mw_emails`	~5	cloudrun/emails	Correos SendGrid
`dy_mw_sms`	~5	cloudrun/infobip	SMS Infobip

Tablas del marcador (7 tablas dedicadas): `dy_marcador_campanas`, `dy_marcador_contactos`, `dy_marcador_muestras_campanas`, `dy_marcador_efectividad_campanas`, `dy_marcador_estados_agentes_externos`, `dy_marcador_log`, `dy_marcador_respuestas_reintentos`

d) Grafo de Dependencias Inter-Servicios

```
■■■
```

```
      ■  SIP:5060 ■ AMI:5038 ■ HTTP:8088/8089       ■
      ■  120+ ext ■ 155+ colas ■ 57 ctx marcador    ■
      ■■■
```

Propiedad	Valor Produccion	Servicios que la leen	Relevancia IPcom Dialer
`cantidadMaximaConexiones`	`10`	Todos los Java	IPcom Dialer usa `MaxPoolSize=50` (mayor)
`direccionIpApp`	`127.0.0.1:8080`	CBXLib	App server (Payara)
`ipExterna`	`aipcom360.ai`	CBXLib, Core	Dominio publico
`ipServicioCore`	`127.0.0.1`	CBXLib, gwivrstrx, PHP	**ALTA** — IPcom Dialer llama core REST API
`intPuertoCore`	`8080`	CBXLib, gwivrstrx	Mapea a `CRM__RESTBASEURL`
`ipServicioReportes`	`127.0.0.1`	CBXLib	Servicio reportes Node.js
`puertoServicioReportes`	`3000`	CBXLib	Puerto api_reportes
`tokenAPIInterno`	`lRjdmcwSiOVxr512...`	CBXLib	**ALTA** — token para REST interno
`blendActivo`	`true`	CBXLib, Core	**ALTA** — habilita inbound+outbound blend
`eventoUUIDActivo`	`true`	CBXLib	Eventos AMI con UUID
`usarHTTPS`	`true`	CBXLib	HTTPS habilitado en produccion
`ipServidorEJB`	`172.18.0.2`	CBXLib	Servidor EJB (Docker network)
`ipServidorADMIN`	`127.0.0.1`	CBXLib	Panel admin
`ipServicioDistribucion`	`172.18.0.2`	CBXLib	Distribuidor de trabajo
`ipServiciosAdicionales`	`172.18.0.2`	CBXLib, gwivrstrx	Servicios adicionales
`publicIp`	`34.63.181.35`	CBXLib, Core	IP publica del Asterisk
`tecnologiaUsuarios`	`SIP`	CBXLib, Core	chan_sip (no pjsip)
`dominioDescargaArchivosSubidosG`	`https://34.63.181.35`	CBXLib	Dominio descarga archivos
`tokenAPIPagos`	`BCNaUJEvHwGhs4X...`	CBXLib	Token API pagos
`URLAPIPagos`	`http://10.142.0.1:8081/dy_paytrx`	CBXLib	Endpoint pagos
`strURLAPIMiddleware`	`https://ccaasmdw...run.app/dymdw/api/`	CBXLib	Middleware WhatsApp (Cloud Run)
`urlMeetMiddlewareAPI`	`http://10.142.0.3:8082/dy_om/`	CBXLib	Meet middleware
`urlMeetAPI`	`https://meet.aip360.cloud:5443/dy`	CBXLib	Videollamada Meet
`strDominioInfobip`	`infobip.aiipcom.ai`	CBXLib	SMS Infobip
`strDominioMdwSMS_t`	`https://aipcom-smsmw...run.app`	CBXLib	SMS middleware (Cloud Run)
`tokenInfobip`	`A5A1E3CC39FAE...`	CBXLib	Token Infobip
`strDominioEmailDySendgrid`	`emailsendgir.aiipcom.ai`	CBXLib	Email SendGrid
`MSIntegrationDomain`	`https://dymsintegration...run.app/`	CBXLib	Microsoft 365 integration
`AIIntegrationDomain`	`ai.aiipcom.ai`	CBXLib	AI integration

Propiedades criticas para IPcom Dialer: Solo 8 de las 36 propiedades son necesarias para el marcador (AMI, BD, Core REST, blend, token interno). Las demas son de servicios no relacionados.

Archivo legado: `/Dyalogo/conf/servicios_asterisk.properties` (usado por Marcador y AgiAmi)

Observaciones de produccion:

- `blendActivo=true` — el blend inbound+outbound esta ACTIVO. IPcom Dialer debe respetar esto (pausar marcacion cuando agente toma llamada entrante)
- `eventoUUIDActivo=true` — los eventos AMI incluyen UUID. Asterisk.Sdk ya maneja esto nativamente
- `cantidadMaximaConexiones=10` — pool de conexiones MySQL MUY bajo para Java. IPcom Dialer configura `MaxPoolSize=50` (5x mas)
- IPs internas `172.18.0.2` — todos los servicios Java corren en Docker network. IPcom Dialer debe estar en la misma red Docker o tener conectividad

f) Credenciales Hardcodeadas (Riesgo de Seguridad)

Servicio	Usuario	Password	BD/Servicio
Marcador	`dymarcador`	`oF3s}jOK#L_06XO4`	`dyalogo_telefonia`
Colector	`dycolector`	`C0l3cT03BD*`	`dyalogo_telefonia`
Soporte	`soporte`	`S0p0rt3DY`	`dy_support`
PHP deploy	`dyhttpd`	`svr4app12*`	MySQL admin
PHP deploy	`dyalogoadm`	`dyalogoadm*bd`	MySQL admin
Emails	`dyemailmwd`	`4ppCRDy1987MwEma1l`	Cloud SQL
Infobip	`dyinfobip`	`4ppCRDy19871nfobip`	Cloud SQL
AMI admin	`admin`	`dyalogo`	Asterisk AMI
AMI marcador	`dyalogoami`	`dyalogo*`	Asterisk AMI
AMI eventos	`dyamievt`	`dy4l0g0`	Asterisk AMI
ARI	`dyalogoari`	`Dy4l0g04Ry`	Asterisk ARI
CRM API	`local`	`local`	REST API auth
PHP API	-	`PGbtywunzaCwCLGSo7zj9CGLV9QxiVgJ`	Admin token
PHP→Core	-	`D43dasd321`	REST token
Node→Chat	-	`EAD268922A462ACA978F98BD4B9E7`	REST token

IPcom Dialer debe: Mover TODAS las credenciales a secret manager (K8s Secrets / env vars).

g) Tareas Programadas (Cron/Schedule)

Tarea	Intervalo	Servicio	Funcion
`procesadorReportes`	1 min	BI	Genera reportes automatizados
`sincronizarSMSs`	5 min	BI	Sincroniza SMS entrantes
`revisarSesionesAgentesInactivos`	3 min	BI	Detecta agentes inactivos
`procesaDatosEstrategiasPasos`	15 min	BI	Procesa estrategias
`refrescarCache`	15 min	Distribuidor	Refresca cache trabajo
`limpiarUltimos`	10 min	Distribuidor	Limpia registros antiguos
`procesadorTimeOutAsignacion`	30 min	BI	Timeout de asignaciones
`evaluaUsuariosInactivos`	23:59 diario	BI	Evaluacion diaria inactividad
`HLlamadasAgenda`	40s	Marcador	Polling callbacks agendados

Tarea	Intervalo	Servicio	Funcion
`HiloSincronizadorCRM`	continuo	Marcador	Sync contactos con CRM

h) Logging de Produccion

Servicio	Ruta log	Tamaño max	Backups
Core	`/var/log/dyalogo/core/info.log`	40 MB	20
CBX	`/var/log/dyalogo/cbx/app/info.log`	20 MB	20
Marcador	`/var/log/dyalogo/marcador/info.log`	~20 MB	20
AgiAmi	`/var/log/dyalogo/agiami/info.log`	~20 MB	20
Colector	`/var/log/dyalogo/colector/info.log`	~20 MB	20
Asterisk	`/var/log/asterisk/messages`	Rotacion logrotate	-

Patron log4j: `%d{yyyy-MM-dd HH:mm:ss} [%5p] [%F:%M:%L] %c %x - %m%n`

IPcom Dialer: stdout JSON (Serilog `JsonFormatter`) — Docker/K8s captura automaticamente.

i) Escala de Produccion Actual

Metrica	Valor
Tenants (huespedes)	**279**
Extensiones SIP	**120+**
Colas ACD	**155+** (queue_2135 a queue_2425)
Rutas entrantes	**60+**
IVRs	**23**
Contextos marcador	**57**
Tablas MySQL totales	**368**
CDR records	**57,694+**
Queue log records	**66,498+**
Alertas sistema	**293,522+**
Servicios Java	**15** (desplegados)
Servicios Node.js	**6** (activos)
Cloud Run services	**4**

j) Implicaciones para IPcom Dialer

1. **IPcom Dialer reemplaza SOLO** `DyalogoCBXMarcador` — los otros 22+ servicios Java siguen operando sin cambios
2. **Debe mantener compatibilidad** con `dyalogocore` (REST API `/bi/gestion/pdsprerob`), `DyalogoCBXAgiAmi` (comparten tablas `dy_llamadas_*`), y `DyalogoCBXColectorDatos` (lee CDR)
3. **Las 7 tablas del marcador** son exclusivas — solo el marcador escribe en `dy_marcador_campanas`, `dy_marcador_contactos`, etc. El riesgo de conflicto es bajo
4. **La config debe migrar** de `servicios_asterisk.properties` a env vars (Anexo E)

5. **El servidor AGI en** `dy_gwivrstrx:5000` sigue operando — IPcom Dialer no necesita replicar `AGIClasificaAM.agi` a menos que se quiera consolidar
6. **Los servicios Node.js y PHP NO son afectados** — no interactuan con el marcador
7. **El hub central** `dyalogocore` sigue generando los contextos `DyCampanaMarcador_*` del dialplan — el unico cambio es que las variables de canal ahora las setea IPcom Dialer

4.14 Configuracion Operacional de Produccion (Análisis de `/etc/dyalogo/`)

Fuente: Análisis completo de `/media/Data/Source/OrionSoft/dyalogo/dyaloconf/etc/dyalogo/` — scripts operacionales, logging, apps auxiliares, deployment Payara.

a) Estructura del Directorio `/etc/dyalogo/`

```
/etc/dyalogo/
■■■ cbx/conf/
■   ■■■ parametros_generales.properties    ← Archivo maestro (analizado en §4.13e)
■■■ conf/
■   ■■■ log4j_core.properties              ← Logging dyalogocore
■   ■■■ log4j_colector.properties          ← Logging colector datos
■   ■■■ log4j_agent.properties             ← Logging CBX agente
■   ■■■ passwordspayara.file               ← Credenciales Payara admin
■■■ bin/
■   ■■■ ColectorDatos.sh                   ← Launcher del colector ETL
■   ■■■ generadorVistas.sh                 ← Genera vistas BI por tenant
■   ■■■ MantenimientoPayara.sh             ← Restart Payara (3 apps)
■   ■■■ trans_gcp_storage_public.sh        ← Upload a GCS bucket
■■■ apps/
■   ■■■ colector/
■   ■   ■■■ DyalogoCBXColectorDatos.jar    ← JAR standalone (55 MB)
■   ■■■ procesadorFlechas/
■       ■■■ procesarFlechasCargue.js        ← Node.js: routing de estrategias
■       ■■■ package.json                   ← Deps: mysql2, moment, winston, yargs
■       ■■■ src/models/                    ← Modelos: Estcon, Gxxxx, Campan, Asitar, etc.
■■■ adjuntos_agente/                       ← Archivos adjuntos de agentes
■■■ clientes/img_huespedes/                ← Imagenes de tenants
■■■ info_inbound.info                      ← Contiene "05" (flag de version inbound)
```

b) Aplicaciones Desplegadas en Payara 5

El script `MantenimientoPayara.sh` confirma las 3 aplicaciones WAR desplegadas:

Aplicacion	Tipo	Funcion
`dyalogocore`	WAR	Hub central: AMI, config, chat, VoIP, dialplan
`dyalogocbx`	WAR	Desktop agente JSF/PrimeFaces
`dy_public_front`	WAR	Portal publico, Click-to-Call

Credenciales Payara: `AS_ADMIN_PASSWORD=dyalogo` (archivo `passwordspayara.file`)

Ciclo de mantenimiento:

1. `asadmin disable` las 3 apps
2. `systemctl stop payara`
3. Limpia `/opt/payara5/glassfish/domains/production/{logs,generated,osgi-cache}/`
4. `systemctl start payara`
5. `asadmin enable` las 3 apps

Implicacion IPcom Dialer: El marcador NO esta en Payara — es un JAR standalone. IPcom Dialer tambien es standalone (Worker Service), asi que el ciclo de vida de Payara es independiente.

c) Colector de Datos (ETL)

Launcher: `bin/ColectorDatos.sh`

```
java -jar -Dlog4j.configuration=File:///etc/dyalogo/conf/log4j_colector.properties \
     /etc/dyalogo/apps/colector/DyalogoCBXColectorDatos.jar callbackAbandonadas=true
```

- **Parametro** `callbackAbandonadas=true`**:** El colector procesa callbacks de llamadas abandonadas
- **Tamaño JAR:** 55 MB (incluye todas las dependencias — fat JAR)
- **Logging:** `log4j_colector.properties` — RollingFileAppender a `/var/log/dyalogo/colector/`, `MaxFileSize=99MB`, 20 backups, nivel `debug`
- **Deteccion de instancia:** Usa `ps aux | grep DyalogoCBXColectorDatos` para evitar ejecucion duplicada

Implicacion IPcom Dialer: El colector consume CDR y tablas `dy_llamadas_*` que el marcador escribe. IPcom Dialer debe escribir en las mismas tablas con el mismo formato para que el colector funcione sin cambios.

d) Procesador de Flechas (Node.js)

App: `procesadorFlechas` — procesamiento de "flechas" (transiciones) en estrategias de campañas.

Aspecto	Detalle
Runtime	Node.js 16.13.0 (via NVM)
Dependencias	mysql2, moment-timezone, winston, yargs, dotenv
Funcion	Procesa las flechas (condiciones SQL) entre pasos de una estrategia de campañas
Modelos	`Estcon` (flechas), `Gxxxx` (tablas G), `Campan` (campañas), `Asitar` (asignaciones), `TablaTemporal`, `Usuari`, `Estpas` (pasos)
Invocacion	CLI: `node procesarFlechasCargue.js -p <paso> -b <base> [-c campo] [-t tabla_temporal]`
Logging	Winston

Logica clave: Lee las flechas (`ESTCON_Consulta_sql_b`) de un paso, extrae el `WHERE`, evalua la condicion, y mueve contactos al paso destino. Tipos de paso: `1` y `6` son campañas con muestra.

Implicacion IPcom Dialer: Este procesador alimenta las muestras (`dy_marcador_muestras_campanas`) que el marcador consume. IPcom Dialer lee muestras de la misma tabla — no requiere cambios en el procesador.

e) Generador de Vistas BI

Script: `bin/generadorVistas.sh`

```
# Consulta todos los tenants activos
IDS=$(mysql -h "$DB_HOST" -u "$DB_USER" -p"$DB_PASS" -D "dyalogo_general" \
      -se "SELECT id FROM huespedes WHERE activo=true;")

# Itera y genera vistas BI por tenant
for IDTENNANT in $IDS; do
    curl 'http://127.0.0.1:8080/dyalogocore/api/bi/generator/views/generateByTennant' \
         --data '{"strUsuario_t":"crm","strToken_t":"D43dasd321","intIdGeneralTennant_t":$IDTENNANT}'
    sleep 10
done
```

- **BD:** `dyalogo_general`, usuario `payara` / `p4y4r4*bd`
- **API Core:** `POST /dyalogocore/api/bi/generator/views/generateByTennant`
- **Token:** `D43dasd321` (hardcodeado)
- **Frecuencia:** Manual o cron — genera vistas SQL por cada tenant para reportes BI

Implicacion IPcom Dialer: No interactua directamente. Las vistas BI se generan sobre datos que el marcador escribe. Si IPcom Dialer mantiene el mismo esquema de datos, el generador funciona sin cambios.

f) Almacenamiento GCS

Script: `bin/trans_gcp_storage_public.sh`

```
gsutil -q cp $1 gs://ipcom360-comunicaciones/$2
echo "https://storage.googleapis.com/ipcom360-comunicaciones/$2"
```

- **Bucket:** `gs://ipcom360-comunicaciones/`

- **URL publica:** `https://storage.googleapis.com/ipcom360-comunicaciones/`
- **Uso:** Subida de archivos (grabaciones, adjuntos) a GCS

Implicacion IPcom Dialer: Si IPcom Dialer necesita subir grabaciones o archivos, debe usar el mismo bucket o el patrón GCS.

g) Configuracion de Logging (Log4j)

Servicio	Archivo config	Ruta log	Max Size	Backups	Nivel
Core	`log4j_core.properties`	`/var/log/dyalogo/core/debug.log`	80 MB	40	debug
Core (info)	`log4j_core.properties`	`/var/log/dyalogo/core/info.log`	20 MB	20	info
Colector	`log4j_colector.properties`	`/var/log/dyalogo/colector/debug.log`	99 MB	20	debug
Colector	`log4j_colector.properties`	`/var/log/dyalogo/colector/{all,info,warn,error,fatal}.log`	1-20 MB	20	multi-level
CBX Agente	`log4j_agent.properties`	`/var/log/dyalogo/cbx/app/debug.log`	80 MB	100	debug

Patron comun: `%d{yyyy-MM-dd HH:mm:ss} [%5p] [%F:%M:%L] %c %x - %m%n`

Nota critica: Los servicios Java usan Log4j 1.x (legacy) con `RollingFileAppender` a archivos locales. Esto es INCOMPATIBLE con contenedores — los logs se pierden al recrear el contenedor.

IPcom Dialer: Usa Serilog con `JsonFormatter` a stdout (ADR-016). Docker/K8s captura automaticamente. Patron de output:

```
{"Timestamp":"2026-03-07T14:30:00.000Z","Level":"Information","MessageTemplate":"Originate sent","Propert
ies":{"campaignId":1,"contactId":12345}}
```

h) Resumen de Hallazgos Operacionales

Hallazgo	Impacto IPcom Dialer	Accion
`parametros_generales.properties` centraliza TODO	Solo 8/36 propiedades son relevantes	Mapear a env vars (Anexo E)
`blendActivo=true` en produccion	IPcom Dialer debe pausar marcacion cuando agente en inbound	Implementar `BlendMode` en `CampaignWorker`
Payara 5 con 3 WARs (core, cbx, public)	Marcador es independiente de Payara	Sin impacto — deployment separado
Colector ETL con `callbackAbandonadas=true`	Debe escribir tablas `dy_llamadas_*` en formato compatible	Mantener esquema BD intacto (ADR-014)
`procesadorFlechas` alimenta muestras	Lee mismas tablas que IPcom Dialer	Sin cambios requeridos
Log4j 1.x a archivos locales	Patron legacy incompatible con containers	IPcom Dialer usa stdout JSON (ya resuelto)
`cantidadMaximaConexiones=10` (Java)	Pool muy bajo	IPcom Dialer: `MaxPoolSize=50`
Docker network `172.18.0.2`	Servicios en red interna Docker	IPcom Dialer debe estar en misma red
GCS bucket `ipcom360-comunicaciones`	Grabaciones y archivos publicos	Reutilizar si IPcom Dialer sube archivos
Generador vistas BI por tenant	Depende del esquema de datos	Funciona sin cambios si se mantiene esquema

4.15 Análisis Exhaustivo de `dyalogocore` — El Hub Central del Ecosistema

Fuente: Análisis exhaustivo de 104 clases Java en `/media/Data/Source/OrionSoft/dyalogo/java/dy_jee/dyalogocore/` — 50+ endpoints REST, 5 EJBs singleton, 8 acciones AMI, 11 eventos AMI, 20+ entidades JPA.

a) ¿Qué es dyalogocore?

`dyalogocore` es una aplicación WAR desplegada en Payara 5 (Jakarta EE) que actúa como **hub central** del ecosistema Dyalogo/IPcom. Es el único componente que tiene acceso directo tanto a AMI como a la base de datos y a los servicios externos.

Dimensión	Valor
Tipo	WAR (Jakarta EE 3.1)
Servidor	Payara 5, puerto 8080
Clases Java	104
Endpoints REST	50+
EJBs Singleton	5 (3 con @Startup)
Tareas programadas	3 activas + 2 deshabilitadas
Acciones AMI	8 tipos (Queue*, Hangup, Ping, DbPut, DbDel, Command, CoreShowChannels, SipShowPeer)
Eventos AMI	11 tipos procesados
Entidades BD	20+ referenciadas
Paquetes	20+

b) ¿Qué Hace? — Las 7 Responsabilidades de dyalogocore

1. Generación Dinámica de Dialplan (CRÍTICA para el marcador)

dyalogocore genera TODOS los archivos `.conf` del dialplan de Asterisk dinámicamente:

Generador	Archivo generado	Trigger
`ConfiguracionMarcador`	`DyCampanaMarcador_<id>.conf`	REST `POST /campanas/voip/persistir`
`ConfiguracionColas`	`queue_<id>.conf` + `DyPMC_<id>.conf`	Creación/modificación de cola ACD
`ConfiguracionExtensiones`	`ext<extension>.conf` + `park_<extension>.conf`	Alta/baja de extensiones SIP
`ConfiguracionRutasEntrantes`	Rutas entrantes `.conf`	REST `POST /voip/re/persistir`
`ConfiguradorIVRs`	IVR `.conf`	REST `POST /voip/ivrs/persistir`
`ConfiguracionTroncales`	Troncales `.conf`	REST `/voip/troncales/persistir`
`UtilidadCampanasVOIPSaliente`	`paso<id>.conf` (salida)	Creación campaña outbound

Flujo de generación del dialplan del marcador:

```
CRM/Admin → POST /campanas/voip/persistir
  → JAXRSCampanasVOIP.persistir()
    → UtilidadCampanasVOIPSaliente.crearCampanaMarcacionAutomatica()
      → Crea DyMarcadorCampanas en BD
      → ConfiguracionMarcador.configurar()
        → creaPlanMarcadorPredictivo()
          → Escribe DyCampanaMarcador_&lt;id&gt;.conf
          → OperacionesTelefonia.recargarPlanMarcacion()
            → AMI CommandAction("dialplan reload")
```

Códigos de acción en el dialplan generado:

Código	Significado	Valor ejemplo	Dialplan generado
1	Ninguna	-	(no acción)
2	Pasar a extensión	`101`	`Goto(ext101,s,1)`
3	Cola ACD	`cola_acd_<id>`	`Goto(cola_acd_<id>,s,1)`
4	Audio	`archivo.wav`	`Playback(archivo)`
5	IVR	`ivr_<nombre>`	`Goto(ivr_<nombre>,s,1)`
6	Externo	contexto	`Goto(contexto,s,1)`
7	Encuestas	encuesta	`Goto(encuesta,s,1)`

Configuración de AMD por campaña:

Tipo campaña	AMD habilitado	AccDef	AccHum	AccMaq
6 (PDS)	No	3 (cola)	-	-
7 (Predictivo) sin AMD	No	3 (cola)	-	-
7 (Predictivo) con AMD	Si	1 (ninguna)	3 (cola)	-1 (AGI clasifica)
8 (Robot) sin AMD	No	5 (IVR)	-	-
8 (Robot) con AMD	Si	1 (ninguna)	5 (IVR humano)	5 (IVR máquina)

2. Gestión de Agentes via AMI (CRÍTICA para operación)

dyalogocore controla el ciclo de vida completo de los agentes:

Login de agente:

1. REST `POST /agentes/acciones/ejecutar` (acción=1)
2. Ejecuta stored procedure `sp_login_agente(extension, identification)`
3. Escribe en ASTDB: `DbPutAction("dy_cbx_act", extension, agent_id)`
4. Agrega agente a cada cola: `QueueAddAction` con delay de 6500ms
5. Evento `QueueMemberAddedEvent` → actualiza `ActividadActual` en BD

Logout de agente:

1. REST `POST /agentes/acciones/ejecutar` (acción=2)
2. Ejecuta stored procedure `sp_logout_agente(extension, identification)`
3. Elimina de ASTDB: `DbDelAction("dy_cbx_act", extension)`
4. Cuelga llamadas activas: `HangupAction` para cada canal
5. Remueve de cada cola: `QueueRemoveAction`
6. Evento `QueueMemberRemovedEvent` → actualiza `ActividadActual`

Pausa de agente:

1. REST `POST /agentes/acciones/ejecutar` (acción=3)
2. Inserta registro en `dy_descansos` (break)
3. `QueuePauseAction` con delay 1000-3500ms según tipo pausa
4. Evento `QueueMemberPausedEvent` → actualiza estado
5. Códigos de pausa: CRM, DESCANSO, ALMUERZO, etc.

3. Event Listener AMI — Monitoreo en Tiempo Real

dyalogocore mantiene una conexión AMI dedicada (usuario `dyamievt`) que escucha 11 tipos de eventos:

Evento	Handler	Acción
`QueueMemberPausedEvent`	`HAsignarEstadoCola`	Actualiza estado pausa en BD
`QueueMemberRemovedEvent`	`HAsignarEstadoCola`	Registra remoción de cola
`QueueMemberAddedEvent`	`HAsignarEstadoCola`	Registra adición a cola
`AbstractQueueMemberEvent`	`HAMIGestionaEventosOperativo`	Sync estado agente con BD
`ExtensionStatusEvent`	`HAMIGestionaEventosOperativo`	"Timbrando"/"No disponible"
`NewChannelEvent`	`HEnviaNotificacionUniqueIdReal`	Tracking nuevo canal
`DialEvent`	`HGgestionaSingletonAgentesBlend`	Tracking llamada saliente
`HangupEvent`	`HGgestionaSingletonAgentesBlend`	Tracking fin de llamada
`JoinEvent`	`HEscuchaEventoACDBlend`	Cliente entra a cola
`NewExtenEvent`	`HEscuchaEventoACDBlend`	Extensión invocada
`AgentConnectEvent`	`HAsignarEstadoCola`	Agente contesta llamada

Conexión dual AMI:

- **Conexión 1 (acciones):** Usuario `dyalogoami` / `dyalogo*` — via `ConfiguracionCBX`
- **Conexión 2 (eventos):** Usuario `dyamievt` / `dy4l0g0` — hardcodeado en `EJBEscuchaEventosTelefonia:137`

4. Sistema de Blend (Inbound + Outbound)

5 clases dedicadas al blend (`com.dyalogo.core.modelo.blend.*`):

- `SingletonAgentesACDConLlamadasSaliente` — registro de agentes con llamadas outbound activas
- `HGgestionaSingletonAgentesBlend` — procesa `DialEvent`/`HangupEvent` para tracking
- `HEscuchaEventoACDBlend` — procesa `JoinEvent` para detectar llamadas entrantes
- `DatoAgenteAsignadoACDLlamadaSaliente` — modelo de agente en blend
- `ComparadorListaAgentesACDLlamadaSaliente` — comparador para ordenar agentes

5. Caché Centralizado

`EJBSingletonCacheDatos` mantiene en memoria:

- Agentes (`AgentesCache`)
- Campañas (`CampanCache`, `CamordCache`, `CamconCache`)
- Usuarios (`UsuariCache`, `UsuariosCache`)
- Extensiones (`ExtensionesCache`)
- Actividad actual (`ActividadActualCache`)
- Tokens API (`CacheDyApiTokens`)
- Config email, SMS, pasos CRM (`EstpasCache`)
- Respuestas originate (`DyRespuestasOriginate`)

Refresco: cada 10+ minutos, o forzado via REST `POST /cache/refrescarCache`.

6. REST API para Operaciones del Marcador

Endpoints que el marcador llama directamente:

Endpoint	Método	Propósito	IPcom Dialer
`/voip/manager/acd/agregarMiembroCola`	POST	Agregar agente a cola	Puede llamar directamente a AMI
`/voip/manager/acd/removerMiembroCola`	POST	Remover agente de cola	Puede llamar directamente a AMI
`/voip/manager/acd/pausaMiembroCola`	POST	Pausar/despausar agente	Puede llamar directamente a AMI

Endpoint	Método	Propósito	IPcom Dialer
`/agentes/acciones/ejecutar`	POST	Login/logout/pausa de agente	**DEBE seguir llamando a dyalogocore**
`/campanas/voip/persistir`	POST	Crear/modificar campaña	Generado desde CRM, no desde marcador
`/notificaciones/notificar`	POST	Enviar notificación	Puede llamar directamente
`/cache/refrescarCache`	POST	Refrescar caché	Puede llamar directamente
`/bi/gestion/pdsprerob`	POST	Gestión PDS/Preview/Robot	**DEBE seguir llamando a dyalogocore**

7. Tareas Programadas

Tarea	Frecuencia	Función
`actualizaEstadosNoDisponible()`	Cada 15 min	Sync estados agentes no disponibles
`revisarAgentesDisponiblesACD()`	Cada 5 min	Verificar agentes disponibles en colas
`revisarConexionAMI()`	Cada 5 min	Heartbeat — reconecta si no hay eventos en 15s

c) ¿Por Qué Lo Hace Así? — Decisiones de Diseño

Decisión	Razón	Deuda técnica
Archivo .conf por campaña	Asterisk requiere archivos estáticos; `#include marcador/*` en `extensions.conf`	No hay API de dialplan dinámico en Asterisk (excepto AEL o ARI)
Conexión AMI dual	Separar acciones (lectura/escritura) de eventos (solo lectura)	Hardcode de credenciales del listener
Async con Thread.sleep()	Delays para evitar race conditions en Asterisk	Debería usar event confirmation, no delays fijos
Caché en memoria (Singleton)	Reducir carga a MySQL	No hay invalidación reactiva — solo refresh periódico
EJB @Singleton	Jakarta EE provee lifecycle, concurrencia, scheduling	Acoplamiento fuerte al contenedor Payara
Generación de dialplan centralizada	Un solo punto de verdad para la config de Asterisk	dyalogocore es SPOF para cambios de configuración

d) ¿Qué Implicaciones Tiene Que Siga Como Está?

Si dyalogocore SIGUE operando (recomendación actual):

Aspecto	Implicación	Riesgo
Generación de dialplan	Sigue generando `DyCampanaMarcador_*.conf` — IPcom Dialer lee el contexto generado	**Bajo** — sin cambios necesarios
Gestión de agentes	Sigue controlando login/logout/pausa via AMI	**Bajo** — IPcom Dialer no duplica esta función
Event listener	Sigue sincronizando estados de agentes en BD	**Bajo** — IPcom Dialer lee `ActividadActual` de BD
Blend	Sigue trackingendo llamadas inbound/outbound	**Medio** — IPcom Dialer genera `DialEvent`/`HangupEvent` que dyalogocore procesa

Aspecto	Implicación	Riesgo
Caché	Sigue en memoria — no tiene invalidación reactiva	**Medio** — si IPcom Dialer modifica datos, el caché puede estar desactualizado hasta 10 min
REST API para CRM	Sigue siendo el gateway para crear campañas	**Bajo** — flujo no cambia
Payara como runtime	Java EE legacy, Payara 5 (fin de soporte próximo)	**Alto a largo plazo** — migración de Payara será necesaria eventualmente
Conexión AMI dual	Consume 2 de los 4 usuarios AMI	**Bajo** — IPcom Dialer usa un 3er usuario (`marcador`)

Riesgos de mantenerlo:

1. **Payara 5 end-of-life** — requiere migración a Payara 6 o Jakarta EE 10+ eventualmente
2. **Java EE legacy** — Log4j 1.x (vulnerable), EJB 3.x, thread spawning manual
3. **SPOF para dialplan** — si dyalogocore cae, no se pueden crear/modificar campañas
4. **Caché stale** — hasta 10 min de desactualización en datos de agentes/campañas

e) ¿Es Primordial Migrarlo? — Análisis Costo/Beneficio

Factor	Migrar dyalogocore	Dejarlo como está
Esfuerzo	**ENORME** — 104 clases, 50+ endpoints, 11 eventos AMI, blend, chat, email, SMS, BI, generación de vistas, notifications	**Mínimo** — solo asegurar compatibilidad con IPcom Dialer
Riesgo	**Muy alto** — afecta a TODOS los servicios del ecosistema (CRM, CBX, BI, PHP, chat)	**Bajo** — el marcador es el único componente que cambia
Beneficio	Eliminar Payara + Java legacy, consolidar en .NET	Solo el marcador se moderniza
Timeline	+6-12 meses adicionales (mínimo)	0 semanas adicionales
Dependencias	Requiere migrar también PHP CRM, dyalogocbx, BI, chat	Sin dependencias adicionales

VEREDICTO: NO migrar dyalogocore en esta fase.

Justificación:

1. dyalogocore tiene **22 responsabilidades** además del marcador (chat, email, SMS, BI, vistas, notificaciones, troncales, IVRs, rutas entrantes, extensiones, etc.)
2. IPcom Dialer solo necesita **3 interacciones** con dyalogocore: (a) leer el dialplan generado, (b) leer estados de agentes de BD, (c) opcionalmente llamar REST para operaciones PDS
3. El costo de migrar dyalogocore es **10x mayor** que migrar solo el marcador
4. La coexistencia es **viable** — IPcom Dialer y dyalogocore comparten BD y AMI sin conflictos
5. La migración de dyalogocore debe ser un **proyecto separado** si se decide modernizar todo el ecosistema

f) Acciones Concretas para la Coexistencia

#	Acción	Descripción	Prioridad
1	**Respetar dialplan generado**	IPcom Dialer NO genera `.conf` — usa los contextos que dyalogocore ya genera	P0
2	**Leer `ActividadActual` de BD**	Para conocer estado de agentes sin duplicar event listener	P0
3	**Llamar REST para PDS/Preview**	`POST /bi/gestion/pdsprerob` si se requiere interacción con workflow CRM	P1

#	Acción	Descripción	Prioridad
4	**Notificar via REST**	`POST /notificaciones/notificar` para notificaciones a agentes	P1
5	**Refrescar caché**	`POST /cache/refrescarCache` después de modificar datos que dyalogocore cachea	P2
6	**NO duplicar operaciones de agentes**	Login/logout/pausa son responsabilidad EXCLUSIVA de dyalogocore	P0
7	**Coordinar blend**	`blendActivo=true` — IPcom Dialer debe pausar marcación cuando `ActividadActual` indica agente en llamada inbound	P1
8	**Tercer usuario AMI**	IPcom Dialer usa usuario `marcador` (diferente a `dyalogoami`/`dyamievt` de dyalogocore)	P0

4.16 Algoritmo PDS, Flujo de Originate y Tablas Compartidas (Análisis P0)

Fuente: Análisis exhaustivo de `DyalogoCBXMarcador` (algoritmo PDS + originate) y `DyalogoCBXAgiAmi` (tablas compartidas y race conditions).

a) Algoritmo PDS — Cómo Decide Cuándo y Cuántas Llamadas Hacer

Jerarquía de hilos:

```
HMarcacionPDSDinamicoPrincipal (cada 60s)
  ■■ Consulta BD: campañas activas tipo 6/7/8 con CAMPAN_ConfDinam_b=-1
     ■■ Por cada campaña activa:
        ■■ HMarcacionDinamicoCampanaMaster.pds() (loop continuo)
           ■■ cantidadCanalesMarcar() → calcula cuántas llamadas enviar
           ■■ asignarContactos() → obtiene contactos del CRM via REST
           ■■ estadoMuestraBatch() → filtra contactos ya gestionados
           ■■ HEjecutaMarcacionContacto (1 thread por contacto)
           ■   ■■ originateAsync() + CallBackLlamadaPDSD
           ■■ tiempoEsperaLoteEnviado() → espera basada en AHT
```

Fórmula de pacing por tipo de campaña:

Tipo	Fórmula	Wait entre lotes	Descripción
6 (PDS)	`(agentes_disponibles + aceleración) - canales_ocupados`	`AHT - 2s` (min 3s, max 10s)	Predictivo estándar
7 (Predictivo)	`(agentes_disponibles + min(aceleración, agentes×3)) - llamadas_en_vuelo`	`AHT - 5s` (min 3s, max 10s)	Predictivo agresivo
8 (Robot)	`cantidad_llamadas_simultaneas - (al_aire + timbrando)`	Sin espera	IVR automatizado, sin agentes

Parámetros de la fórmula:

Parámetro	Fuente	Cómo se obtiene
`agentes_disponibles`	AMI `queue show <cola>`	Parseo de output: líneas con "SIP" y "(Not in use)", excluyendo pausados/timbrando
`aceleración`	`CAMPAN_Aceleracion_b`	BD CRM, rango 1-80 (tipo 7 cap)

Parámetro	Fuente	Cómo se obtiene
`llamadas_en_vuelo`	`SingletonLlamadasProcesadasCampana`	ConcurrentHashMap incrementado en originateAsync, decrementado en callback
`AHT`	`dy_informacion_actual_campanas.aht`	SQL `WHERE fecha=DATE(now()) AND id_campana=?`, cache 2 min
`canales_ocupados`	AMI `queue show`	Agentes en estado Busy/Ringing

6 mejoras implementadas en el código (comentadas como "Mejora N"):

1. Descontar llamadas en vuelo (tipo 7)
2. Batch query de estado de contactos
3. No esperar lote completo si hay contactos disponibles
4. `removeAll()` en vez de removes individuales
5. Cache de AHT por 2 minutos
6. Tope relativo: máximo 3 llamadas por agente disponible (tipo 7)

Selección de contactos:

- Llama REST `POST /agentes/tareas/trabajoCampana` (primero a `dy_distribuidor_trabajo`, fallback a `dyalogocore`)
- Credenciales: `usuario=local, token=local`
- Máximo 3 intentos de llenado, espera 5s entre intentos
- Filtros: `estado > 2` → excluir, `activo == 0` → excluir, `intentos > límite` → excluir, `estado == -2000` → ya gestionado

Scheduling de campañas:

- Campos `ejecutaLunes..ejecutaDomingo` + `horaInicial/horaFinal` en tabla CAMPAN
- Validación: `SingletonHilosCampana.horarioValido()` — si fuera de horario, libera recursos y sale
- Si 20+ iteraciones sin canales disponibles → sale del loop

b) Flujo Completo de un Originate — Del Contacto al Resultado en BD

Paso 1: Construcción del OriginateAction

Campo	Valor	Línea
Channel	`SIP/<troncal>/<numero>` (de `AnalizadorRutaSaliente`)	1226
Context	`DyCampanaMarcador_<id>`	1202
Exten	`"s"`	1203
Priority	`1`	1204
Timeout	`max(11000, campaign.timeout × 1000)` ms	1211-1224
CallerId	Número marcado (teléfono del contacto)	1199
ActionId	`"<contactId>	<campaignId>"`
Account	`"<contactId>	<campaignId>"`

7 Variables de canal seteadas por el marcador (NO 12 — las otras 5 las setea el dialplan):

Variable	Valor	Propósito
`DY_MARCADOR_OPCION`	`"<acciónHumano>	<acciónMáquina>"`
`DY_MARCADOR_IDCLIENTE`	ID contacto CRM	Identificación
`DY_MARCADOR_TELEFONO_MARCADO`	Teléfono marcado	Display

Variable	Valor	Propósito
`DY_MARCADOR_ID_CONTACTO`	ID contacto	Tracking
`DY_MARCADOR_ADICIONAL1`	`"PDSD"`	Identifica origen PDS
`DY_MARCADOR_CONSINTE`	ID contacto (redundante)	CRM integration
`DY_MARCADOR_CONSINTE_CAMPAN_CRM`	ID campaña CRM	CRM integration

Corrección crítica: El dialplan generado por `dyalogocore` agrega las otras 5 variables via `SIPAddHeader` (URL1-5, UID, AccDef, AccHum, AccMaq, etc.). El marcador solo setea 7 variables de canal en el `Originate`.

Paso 2: Selección de troncal (`AnalizadorRutaSaliente`)

1. Obtiene contextos de campaña de `SingletonContextoMarcacionPDS`
2. Matching de patrón: número contra `strPatron_t` (e.g., `"5760XXXXXXXX"`)
3. Si match: usa `objTroncal_t`, si desborde: usa `objTroncalDesborde_t`
4. Genera dial string: `SIP/<nombre_troncal>/<teléfono>` o `IAX2/...` o `DAHDI/r<grupo>/...`
5. Si no match: `"_SINRUTA"` → llamada rechazada

Paso 3: Envío asíncrono

- `svrAsterisk_t.originateAsync(originateAction, callback)`
- Antes: verifica que el teléfono no esté ya en marcación (`SingletonGestionesOriginateActivas`)
- Registra llamada en vuelo para conteo

Paso 4: Callback con 5 outcomes posibles

Código	Evento	Acción	Reintento
1	`onDialing`	Log, update BD estado="Marcando"	-
2	`onSuccess`	Update BD estado="Contestada", `exitoso=true`, serializar llamada	**NO** — éxito terminal
3	`onNoAnswer`	Update BD estado="No contestada"	Si hay reintentos disponibles
4	`onBusy`	Update BD estado="Ocupada"	Si hay reintentos disponibles
5	`onFailure`	Update BD estado="Fallida"	**Si** — intenta troncal de desborde

Códigos especiales de hangup:

- `101` — "call reject" en `hangupCauseText` → rechazada por troncal
- `102` — "Network out of order" → sin canales libres en troncal

Paso 5: Escrituras en BD (3 tablas)

Tabla	Cuándo	Columnas clave
`dy_marcador_contactos`	En cada callback final (no en MARCANDO)	`intentos++`, `estado`, `estadoCod`, `fecha_ultimo_intento`, `uniqueId`, `telefono_marcado`, `reintentar`, `exitoso`
`dy_marcador_log`	En cada callback final	`fecha_hora`, `uniqueId`, `razon`, `respuesta`, `canal`, `contexto`, `id_contacto`, `id_troncal`, `telefono_marcado`, `traza_completa`
`dy_llamadas_salientes`	En cada callback final	`fecha_hora`, `marcador=true`, `id_troncal`, `numero_marcado`, `id_campana`, `id_proyecto`, `uniqueId`

Tabla `dy_respuestas_originate` — mapeo de códigos a acciones:

Código	Descripción	Campo `id_monoef`
1	Marcando	-
2	Contestada	Efectividad positiva
3	No contestada	Efectividad negativa
4	Ocupada	Efectividad negativa
5	Fallida	Efectividad negativa
101	Rechazada por troncal	-
102	Sin canales libres	-

c) Tablas Compartidas entre Marcador y AgiAmi — Race Conditions

Tabla de acceso concurrente:

Tabla	Marcador ESCRIBE	AgiAmi ESCRIBE	Columnas en conflicto	Riesgo
`dy_marcador_contactos`	`intentos`, `estado`, `estadoCod`, `fecha_ultimo_intento`, `uniqueId`, `exitoso`, `reintentar`	`estado` (via EventoMarcadorContestaHumano/Maquina), INSERT nuevos (abandonadas, muestras)	`estado`, `estadoCod`	**ALTO**
`dy_marcador_log`	INSERT completo	NO	-	Ninguno
`dy_llamadas_salientes`	INSERT	NO (solo lee)	-	Ninguno
`dy_marcador_campanas`	NO (lee)	NO (lee)	-	Ninguno
`dy_llamadas`	NO	INSERT	-	Ninguno
`dy_llamadas_causas_colgado`	NO	INSERT	-	Ninguno
`dy_tiempos_timbrado`	NO	INSERT	-	Ninguno

4 Race Conditions identificadas:

#	Escenario	Tablas	Riesgo	Mitigación para IPcom Dialer
RC-1	AgiAmi inserta contacto abandonado (cada 60s) mientras IPcom Dialer actualiza el mismo contacto	`dy_marcador_contactos`	**ALTO** — duplicados o escritura perdida	Usar `INSERT ... ON DUPLICATE KEY UPDATE` o transacciones con `SELECT FOR UPDATE`
RC-2	AgiAmi actualiza `estado="Contestada"` (EventoMarcadorContestaHumano) mientras IPcom Dialer actualiza `estadoCod`	`dy_marcador_contactos`	**MEDIO** — estado inconsistente	IPcom Dialer es dueño del estado; AgiAmi debería leerse como complementario
RC-3	Dos threads calculan `maximaPrioridad()` simultáneamente → ambos insertan con misma prioridad	`dy_marcador_contactos`	**BAJO** — prioridades duplicadas	Usar `AUTO_INCREMENT` o `MAX(prioridad)+1` atómico

#	Escenario	Tablas	Riesgo	Mitigación para IPcom Dialer
RC-4	AgiAmi falla el INSERT pero marca como procesado y envía email de éxito	`dy_marcador_contactos`	**BAJO** — false positive en reporte	Bug existente en Java; IPcom Dialer no lo replica

Scripts AGI de AgiAmi que modifican tablas del marcador (4 de 17):

Script	Tabla	Operación	Impacto
`InsertaRegistroMuestraMarcador`	`dy_marcador_contactos`	INSERT (callback inbound)	IPcom Dialer verá el contacto nuevo en su próxima iteración
`EventoMarcadorContestaHumano`	`dy_marcador_contactos`	UPDATE `estado="Contestada"`	Puede competir con el callback del marcador
`EventoMarcadorContestaMaquina`	`dy_marcador_contactos`	UPDATE `estado="Maquina detectada"`, `estadoCod=7`	Puede competir con el callback del marcador
`HiloProcesaLlamadasAbandonadas`	`dy_marcador_contactos`	INSERT (abandonadas cada 60s)	Duplicados posibles sin check transaccional

Eventos AMI que AgiAmi escucha (NO conflicto con marcador):

Evento	Handler	Tabla afectada	Conflicto con IPcom Dialer
`AgentConnectEvent`	`HProcesaEvento`	`dy_tiempos_timbrado`	Ninguno
`HangupEvent`	`HProcesaEventoColgado`	`dy_llamadas_causas_colgado`	Ninguno
`QueueCallerAbandonEvent`	Singleton → hilo abandonadas	`dy_marcador_contactos`	**RC-1**
`AgentRingNoAnswerEvent`	Singleton → hilo rechazadas	Solo email	Ninguno
`NewChannelEvent`	`HEvaluaIngresoSalidaAgente`	Estados agente	Ninguno

4.17 Stored Procedures, Callbacks, Blend e Integración CRM (Análisis P1)

a) Stored Procedures — El Marcador NO Usa SPs

Hallazgo clave: El marcador (`DyalogoCBXMarcador`) NO llama stored procedures. Usa queries SQL básicas directamente. Los SPs son responsabilidad de `dyalogocore` y `DyalogoCBXColectorDatos`.

SPs relevantes para el ecosistema del marcador:

Stored Procedure	Quién lo llama	Tablas afectadas	Relevancia IPcom Dialer
`sp_login_agente(ext, ident)`	dyalogocore (login agente)	INSERT: `dy_sesiones`, `dy_actividad_actual`, `dy_sesiones_campanas`, `dy_actividad_actual_campanas`, `dy_logueo_agentes_campanas`. UPDATE: `dy_descansos`	**Ninguna directa** — dyalogocore sigue ejecutándolo
`sp_logout_agente(ext, ident)`	dyalogocore (logout agente)	UPDATE: `dy_sesiones`, `dy_descansos`. DELETE: `dy_logueo_agentes_campanas`, `dy_actividad_actual`	**Ninguna directa** — dyalogocore sigue ejecutándolo
`sp_inserta_llamada_prev_prog(...)`	dyalogocbx (preview/prog)	INSERT: `dy_llamadas_salientes`	**Media** — IPcom Dialer escribe directo a `dy_llamadas_salientes` sin SP

Stored Procedure	Quién lo llama	Tablas afectadas	Relevancia IPcom Dialer
`sp_actualiza_llamada_prev_prog(id, uid)`	dyalogocbx	UPDATE: `dy_llamadas_salientes`	**Media** — same
`sp_elimina_usuario_agente(id)`	dyalogocore	DELETE cascado en 10+ tablas	**Ninguna** — admin operation
`sp_llena_info_act_cam(id)`	Colector	Métricas campaña	**Ninguna** — colector lo ejecuta
`sp_etiquetar_llamada(uid)`	Colector	UPDATE: `dy_llamadas_espejo`	**Ninguna** — colector lo ejecuta

Implicación: IPcom Dialer puede ignorar completamente los SPs. Las escrituras a `dy_marcador_contactos`, `dy_marcador_log`, y `dy_llamadas_salientes` se hacen con SQL directo.

b) Callbacks Agendados — Polling Cada 40 Segundos

Clase: `HLlamadasAgenda` (hilo daemon que corre cada 40s)

Flujo completo:

```
Agente programa callback desde CRM
  → INSERT dy_marcador_contactos SET agendado=true, agendaFechaHora='2026-03-07 15:30:00'

HLlamadasAgenda (cada 40s):
  → SELECT * FROM dy_marcador_contactos
    WHERE agendado=true AND agendaFechaHora &lt;= NOW()
    ORDER BY agendaFechaHora
  → Almacena en lista estática en memoria

PDS Loop (HCampanaMarcacion):
  → Obtiene contactos agendados PRIMERO (prioridad sobre contactos normales)
  → Reset: intentos=-1, reintentar=true (fuerza reintento)
  → Originate normal via AMI
  → En callback: agendado=false, agendaFechaHora=null (limpia agenda)
```

Columnas de `dy_marcador_contactos` para agenda:

Columna	Tipo	Propósito
`agendado`	BOOLEAN	Flag de callback programado
`agendaFechaHora`	TIMESTAMP	Fecha/hora del callback
`fechaHoraMinimaProximaLlamada`	TIMESTAMP	Mínimo delay antes de reintento
`reintentar`	BOOLEAN	Flag de reintento
`intentos`	INT	Contador de intentos

Prioridad: Los callbacks agendados se **anteponen** a los contactos normales en la lista de marcación. Se insertan ANTES (`lstContactos.add(contactoAgenda)` antes de `lstContactos.addAll(normales)`).

Timezone: Usa timezone del JVM del sistema (`SimpleDateFormat` sin timezone explícito). Comparación por string, NO por fecha. **Bug potencial** si timezone del server difiere del cliente.

No-answer flow: Si el callback no se contesta → `ConsumeREST.enviaAgendaNoContestada()` → REST `POST /dateNoAnswer/open` → notifica al agente para reprogramar.

IPcom Dialer debe: Implementar polling de `dy_marcador_contactos WHERE agendado=true AND agendaFechaHora <= NOW()` y anteponer estos contactos al batch normal.

c) Lógica de Blend — Pause a Nivel de Campaña

Arquitectura dual: Toggle global (`blendActivo`) + toggle per-campaña (`priorizarLlamadasEntrantes`).

```
blendActivo=true (global) AND priorizarLlamadasEntrantes=true (campaña) → BLEND ACTIVO
blendActivo=false → SIN BLEND (independiente de campaña)
blendActivo=true AND priorizarLlamadasEntrantes=false → SIN BLEND para esa campaña
```

Cómo el marcador verifica disponibilidad antes de originar:

1. **Fuente primaria: AMI** `queue show <cola>` — parsea output línea por línea:

- Agente disponible: contiene `"SIP"` Y `"(Not in use)"` Y NO `"(paused:"` Y NO `"Ringing"` Y NO `"Unavailable"` Y NO en wrap-up
- Wrap-up detection: si `"last was X secs ago"` y X ≤ wrapuptime → NO disponible
- Se ejecuta cada 6 segundos via `HProcesoActualizacionCanalesDisponiblesACD`

2. **Fuente secundaria: BD** `dy_actividad_actual` — cuando blend trigger se activa:

- Query: `WHERE sentido='out' AND pausa=''` (agentes en outbound sin pausa)
- Detecta agentes pausados en CRM pero sin llamada activa

Flujo de blend cuando llega llamada inbound:

```
JoinEvent (cliente entra a cola)
  → HEscuchaEventoACDBlend thread
    → Verifica blendActivo=true Y priorizarLlamadasEntrantes=true
    → Sleep random 1200-3500ms (anti-thundering herd)
    → Query AMI: agentes disponibles
    → IF (llamadas_en_cola &gt; 0 AND agentes_disponibles == 0):
      → Busca agente con llamada outbound MÁS CORTA
      → Finaliza esa llamada outbound (libera agente para inbound)
    → ELSE: no acción (hay agentes libres)
```

En el marcador: Cuando detecta `llamadasEnEspera > 0`, **pausa la campaña ENTERA** (no por agente individual):

```
if (llamadasEnEspera &gt; 0) {
    esperar(llamadasEnEspera * 2500, 1, "Llamadas encoladas");  // 2.5s por llamada en cola
}
```

IPcom Dialer debe:

1. Leer `blendActivo` del config (env var `DIALER__BLENDMODE`)
2. Leer `priorizarLlamadasEntrantes` de `dy_campanas` por cada campaña
3. Antes de cada batch de originates: query `queue show` para contar llamadas en espera
4. Si hay llamadas inbound en cola → pausar marcación de esa campaña por N×2.5s
5. NO necesita replicar el `SingletonAgentesACDConLlamadasSaliente` — eso es responsabilidad de dyalogocore

d) Integración CRM REST — 7 Endpoints

Autenticación: `usuario=local, token=local` (hardcodeado en todo el marcador)

URLs base (de `ConfiguracionCBX`):

- Core: `http://<ipServicioCore>:<intPuertoCore>/dyalogocore/api`
- Distribuidor: `http://<ipServicioDistribucion>:8080/dy_distribuidor_trabajo/api`

Endpoints que IPcom Dialer DEBE llamar:

#	Endpoint	Método	Cuándo	Datos enviados	Fallback
1	`/agentes/tareas/trabajoCampana`	POST	Cada batch PDS — obtener contactos	`intIdCampanaCRM_t`, `intIdAgenteCBX_t=-10`, `booAgenda_t`	Primero distribuidor, luego core
2	`/bi/gestion/pdsprerob`	POST	Después de CADA llamada completada	`intConsInteCAMPAN_t`, `intConsInte_t`, `intIdUsuarioAgente_t`, `intResultadoMarcacion_t`, `strTelefono_t`, `strIdLlamada_t`	Ninguno (log WARN)

#	Endpoint	Método	Cuándo	Datos enviados	Fallback
3	`/agentes/tareas/listarPDSPre`	POST	Preview campaigns — listar asignaciones	`intIdAgenteCBX_t`	Ninguno

Endpoints opcionales:

#	Endpoint	Método	Cuándo	Propósito
4	`/campanas/voip/sincronizarMarcadorAlCRM`	POST	Manual/on-demand	Sync contactos marcador→CRM
5	`/dateNoAnswer/open`	POST	No-answer callback	Crea agenda para agente
6	`/marcador/gestion`	POST	Call tracking	Agregar/eliminar/limpiar llamadas en proceso
7	`/cache/refrescarCache`	POST	Después de cambios	Fuerza refresco caché dyalogocore

Acceso directo a BD CRM (sin REST): El marcador Java también escribe directamente a `DYALOGOCRM_SISTEMA` via JDBC:

- UPDATE tablas MUESTRA (estado de contacto en CRM)
- INSERT tablas CONDIA (historial de contacto)
- INSERT tablas DY_ESTADO (estado de gestión)

IPcom Dialer debe: Mantener AMBOS canales — REST para operaciones orquestadas + JDBC directo para escrituras de estado. Las connection strings están en Anexo E.

Error handling: El Java NO tiene reintentos automáticos. Solo log WARN si HTTP ≠ 200. IPcom Dialer puede mejorar esto con retry policy (Polly).

4.5 Componentes de Migración Directa vs. Rediseño

Componente	Migración	Justificación
Entidades JPA (26 tablas)	**Directa**	Mapeo 1:1 a POCOs + Dapper
DAOs de consulta	**Directa**	SQL queries se migran a Dapper
Lógica de scheduling (hora/día)	**Directa**	Misma lógica en C#
Originate + Callbacks	**Rediseño**	Asterisk.Sdk ya tiene `OriginateAsync()` con API diferente
Event handling (AMI)	**Rediseño**	Asterisk.Sdk usa `IObservable<T>` y `AsyncEventPump`
Thread pools	**Rediseño**	`System.Threading.Channels` + `Task.Run`
Singletons de estado	**Rediseño**	DI containers + scoped services
Lógica PDS/Predictivo	**Rediseño parcial**	Core logic migra, pero se adapta a Activities framework
Conexión AMI dual	**Eliminado**	Asterisk.Sdk maneja una sola conexión con reconnect
CRM integration	**Adapter**	Preservar contract, abstraer implementación
Configuración	**Rediseño**	`IOptions<T>` + `appsettings.json` + secret manager

4.18 Volúmenes de Datos y Escala de Producción (Análisis P2)

> ***Fuente:*** *Análisis de DDLs (AUTO_INCREMENT),* `parametros_generales.properties`*, configuración de Asterisk, y código Java del marcador.*

a) Tamaños de Tablas (AUTO_INCREMENT = conteo acumulativo aproximado)

Tablas core de llamadas:

Tabla	Registros aprox.	Notas
`dy_llamadas`	~101,537	Acumulado desde inicio del sistema
`dy_llamadas_salientes`	~235,873	Outbound — tabla principal de IPcom Dialer
`dy_llamadas_entrantes`	~32,463	Inbound
`dy_marcador_log`	~78,383	Log transaccional del marcador
`dy_marcador_contactos`	Sin AUTO_INCREMENT	Crecimiento ilimitado — no hay auto-limpieza

Tablas de Asterisk:

Tabla	Registros aprox.
`cdr`	~57,694
`queue_log`	~66,498

Tablas de configuración y campañas:

Tabla	Registros aprox.	Notas
`dy_campanas`	~2,278	Activas + históricas
`CAMPAN` (CRM)	~2,551	Total CRM
`dy_marcador_campanas`	~42 activas	Campañas activas del marcador
`dy_marcador_muestras_campanas`	~42	1:1 con campañas
`dy_marcador_respuestas_reintentos`	~2,875	Reglas de reintento por código de respuesta

Tablas de agentes y extensiones:

Tabla	Registros aprox.	Notas
`dy_agentes`	~352	Agentes activos
`dy_extensiones`	~364	Extensiones SIP
`dy_campanas_agentes`	~13,516	Mapeos agente-campaña
`dy_actividad_actual`	~1,937	Estado en tiempo real
`dy_actividad_actual_campanas`	~5,135	Actividad por campaña

b) Restricciones del Sistema

Parámetro	Valor	Fuente
Canales simultáneos globales	20	`DY_CANTIDAD_MAXIMA_GANALES_MARCADOR_G`
Conexiones MySQL	10 máx.	`cantidadMaximaConexiones` en `parametros_generales`
Colas ACD	168	Archivos de configuración en `/asterisk/etc/asterisk/queue_config/`
Contextos de marcador	57	`DyCampanaMarcador_*` en dialplan

Parámetro	Valor	Fuente
Campañas activas simultáneas	42	`dy_marcador_campanas`
Agentes	352	`dy_agentes`
Tenants activos	279	Script `generadorVistas.sh`

c) Parámetros de Pacing por Campaña

Campo	Default	Rango	Descripción
`CAMPAN_MaxRegDinam_b`	10	10-1000	Batch size de consulta de contactos
`CAMPAN_Aceleracion_b`	1	1-N	Factor de aceleración de marcación
`CAMPAN_LlamadasSimultaneas_b`	1	1-20	Llamadas simultáneas por campaña
`CAMPAN_LimiRein__b`	3	1-10	Límite de reintentos por contacto

d) Tiempos y Delays del Sistema

Operación	Valor	Fuente
Ciclo principal PDS	60s	`INT_S_ESPERA_ITERACION_PRINCIPAL_T`
Espera sync queue_log	10s inicial, 10 iteraciones × 100-500ms	`HRevisarLlamadaContestada`
Retry contacto fallido	3,000ms	`HEjecutaMarcacionContacto`
NoAnswer/Busy delay	1,500-3,000ms	Antes de reintentar
AMI timeout por llamada	11-20s (configurable, default ~65s)	Per-campaña
Colector batch log	Cada 100 llamadas	`HProcesaLlamadasMarcador`

e) Capacidad Diaria Calculada

Métrica	Valor	Cálculo
Capacidad máxima teórica	~57,600 intentos/día	20 canales × (86,400s ÷ 15s AHT) × 50% ASR
Campañas concurrentes reales	4-5 típicas	20 canales ÷ 4-5 canales/campaña
Crecimiento `dy_marcador_log`	~3-5 registros/min pico	78K registros ÷ operación 8h/día
Agentes por cola promedio	~2	352 agentes ÷ 168 colas

f) Implicaciones para IPcom Dialer

Hallazgo	Acción para IPcom Dialer
Pool MySQL = 10 conexiones (MUY bajo)	`MinPoolSize=5; MaxPoolSize=50` — ya definido en Anexo D
`dy_marcador_contactos` sin límite de crecimiento	Considerar cleanup periódico o archivado — fuera de scope del marcador
20 canales globales = cuello de botella	`SemaphoreSlim` global con capacity=20, no solo per-campaña
Batch default de 10 registros	Configurable en IPcom Dialer, default 10 para paridad
42 campañas pero solo 4-5 activas realmente (por canales)	`DIALER__MAXCONCURRENTCAMPAIGNS=30` es holgado
13,516 mapeos agente-campaña	Query de agentes disponibles debe usar índices — Dapper con parámetros

---

4.19 Procedimiento de Validación en Shadow Mode (Análisis P2)

> ***Restricción fundamental:*** *Ambos marcadores NO pueden operar simultáneamente sobre las mismas campañas. Comparten tablas, conexión AMI, y endpoints REST. Operación concurrente produciría Originate duplicados, race conditions amplificadas, y corrupción de auditoría. El diseño usa* ***partición a nivel de campaña*** *con migración progresiva de tráfico.*

a) Fase 0 — Preparación de Infraestructura (1-2 semanas)

Paso 0.1 — Aislamiento AMI:

- IPcom Dialer usa usuario AMI `marcador` (distinto de `dyalogoami` del Java).
- Permisos idénticos verificados. Variable `X-Dialer-Source: ipcom` en cada Originate para atribución en CDR.

Paso 0.2 — Instrumentación de BD:

```
ALTER TABLE dy_marcador_contactos ADD COLUMN dialer_source VARCHAR(10) DEFAULT 'java';
ALTER TABLE dy_marcador_log ADD COLUMN dialer_source VARCHAR(10) DEFAULT 'java';
ALTER TABLE dy_llamadas_salientes ADD COLUMN dialer_source VARCHAR(10) DEFAULT 'java';
```

- Java escribe con default `'java'` (sin cambios de código). IPcom Dialer escribe `'ipcom'` explícitamente.
- Columna indexada para queries de comparación eficientes.

Paso 0.3 — Tabla de Enrutamiento de Campañas:

```
CREATE TABLE dy_marcador_dialer_routing (
    campana_id BIGINT PRIMARY KEY,
    tenant_id INT NOT NULL,
    dialer VARCHAR(10) NOT NULL DEFAULT 'java',  -- 'java' o 'ipcom'
    switched_at TIMESTAMP NULL,
    switched_by VARCHAR(50) NULL
);
```

- Poblada con todas las campañas existentes como `dialer='java'`.
- Ambos marcadores leen esta tabla cada 30 segundos.
- Cada marcador solo procesa campañas asignadas a él.
- **Alternativa si no se puede modificar el JAR:** deshabilitar campañas individuales en `dy_marcador_campanas.activa = 0` y configurar IPcom Dialer para esas campañas via su propia lista. Más tosco pero cero cambios Java.

Paso 0.4 — Pipeline de Métricas Comparativas:

Métrica	Tabla fuente	Agregación
Contactos marcados/hora/campaña	`dy_marcador_contactos`	COUNT por `dialer_source`, campaña, hora
Distribución de estados finales	`dy_marcador_contactos`	GROUP BY `estadoCod`, `dialer_source`
Tasa de conexión	`dy_llamadas_salientes`	contestadas / originadas por `dialer_source`
Tiempo promedio de ring	`dy_llamadas_salientes`	AVG(ring_duration) por `dialer_source`
Tasa de abandono	`dy_marcador_contactos`	COUNT(abandonadas) / COUNT(originadas)
Volumen de log y tasa de error	`dy_marcador_log`	COUNT, clasificación de errores

Dashboard Grafana con mínimo 7 días de baseline Java antes de iniciar tráfico IPcom.

Paso 0.5 — Topología de Red:

```
GCP VPC: production
■■■ Asterisk 18
■   ■■■ AMI user: dyalogoami → Java marcador
■   ■■■ AMI user: marcador → IPcom Dialer
■■■ java-marcador (JAR existente) — Campañas: TODAS → decreciente
```

```
■■■ ipcom-dialer (Docker .NET AOT) — Campañas: NINGUNA → creciente
■■■ Payara 5: dyalogocore (compartido por ambos)
■■■ AgiAmi (sin cambios)
■■■ Cloud SQL MySQL (compartido)
```

b) Fase 1 — Dry Run: Shadow Solo-Lectura (1 semana)

IPcom Dialer ejecuta toda la lógica del marcador en **modo lectura**:

- Conecta a AMI (`marcador`), suscribe eventos — **solo lectura**.
- Consulta contactos via REST `/trabajoCampana` — **idempotente**.
- Computa decisiones de marcación (contactos, prioridad, canales).
- **NO ejecuta Originate. NO escribe a MySQL. NO llama a `/pdsprerob`.**
- Logea cada decisión: `[DRY-RUN] Would Originate: channel=SIP/xxx, context=xxx`.

Comparación de decisiones:

Métrica de match	Target mínimo
Decision match rate (mismo contacto, mismo ciclo)	> 95%
Priority match rate (prioridad idéntica)	> 95%
Parameter match rate (channel, context, exten, callerID idénticos)	> 95%

Baseline de race conditions: Documentar frecuencia de RC-1 a RC-4 durante dry-run.

Rollback: Detener contenedor IPcom Dialer. Cero impacto en producción.

c) Fase 2 — Shadow en Vivo: Una Campaña (2 semanas)

IPcom Dialer toma UNA campaña de bajo riesgo por tenant (iniciando con 3 tenants).

Criterios de selección:

1. < 50 contactos/día
2. Campaña interna o con SLA tolerante
3. Dialplan simple (sin IVR complejo)
4. Servidor Asterisk único

Procedimiento de switchover por campaña:

Paso	Tiempo	Acción
Notificación	T-5 min	Confirmar campaña limpia (sin llamadas activas, sin MARCANDO pendientes)
Switch	T-0	`UPDATE dy_marcador_dialer_routing SET dialer='ipcom' WHERE campana_id = ?`
Transición	T+0 a T+60s	Java deja de procesar, IPcom toma control (siguiente poll)
Verificación	T+5 min	Confirmar primer Originate exitoso, `dialer_source='ipcom'` en BD
Métricas	T+15 min	Primera comparación de métricas
Validación	T+60 min	Comparación completa

Umbrales de alerta durante shadow:

Métrica	Umbral de alerta	Acción
Tasa de fallo Originate	> 10% vs baseline Java	Investigar; rollback si > 20%
Transiciones de estado inesperadas	Cualquiera	Investigar inmediatamente

Métrica	Umbral de alerta	Acción
Frecuencia RC-1/RC-2	> 2× baseline	Rollback
Quejas de agentes	Cualquiera	Investigar inmediatamente
Caídas de conexión AMI	> 1/hora	Investigar
Errores REST	> 5% tasa de fallo	Rollback
Latencia escrituras BD	> 2× p99 Java	Investigar

Criterios de éxito (deben cumplirse 7 días consecutivos):

- Tasa de conexión dentro de ±5% del baseline Java
- Tasa de abandono dentro de ±5%
- Cero transiciones de estado inesperadas
- Cero incidentes de corrupción de datos
- RC-1/RC-2/RC-3/RC-4 no peores que baseline
- Sin quejas de agentes
- Uptime AMI > 99.9%

d) Fase 3 — Expansión Multi-Campaña (2-4 semanas)

Semana	Tenants	Campañas	% Tráfico Total
Semana 1 (Fase 2)	3	3	~1%
Semana 3	10	15	~5%
Semana 4	30	40	~15%
Semana 5	80	100	~35%
Semana 6	150	200	~60%

Gates de expansión (todos deben pasar):

- Tier anterior estable 5+ días hábiles
- Todos los criterios de Fase 2 cumplidos
- Sin incidentes P1/P2 atribuidos a IPcom Dialer
- Sign-off del equipo de operaciones

Validaciones adicionales en Fase 3:

- **Multi-servidor:** Validar `AsteriskServerPool` con tenants multi-Asterisk
- **Carga pico:** Horarios 10:00-13:00 y 15:00-18:00. Target: 20 canales concurrentes con < 100ms latencia AMI en p99 y cero eventos dropped en `AsyncEventPump`
- **A/B testing:** Campañas similares del mismo tenant (mismo tipo, volumen similar) partidas entre Java e IPcom. Z-test de dos proporciones con $p < 0.05$, mínimo 1,000 contactos por grupo

e) Fase 4 — Cutover Completo (1 semana)

Checklist pre-cutover:

- ■ Los 57 contextos de campaña probados en IPcom Dialer
- ■ Al menos 1 campaña por tenant ejecutada en IPcom Dialer
- ■ Rollback probado en producción (5+ campañas revertidas deliberadamente)
- ■ Equipo on-call entrenado en operación IPcom Dialer
- ■ Comunicación enviada a administradores de tenants

Procedimiento de cutover:

1. Ventana de bajo tráfico (domingo 02:00-06:00 típicamente)

2. `UPDATE dy_marcador_dialer_routing SET dialer='ipcom' WHERE dialer='java'`
3. Monitorear 4 horas
4. Si estable: detener contenedor Java (NO eliminar — mantener para rollback)
5. Si inestable: ejecutar rollback (revertir UPDATE, reiniciar Java)

Post-cutover: Mantener contenedor Java disponible (detenido) por 2 semanas. Descomisionar tras 2 semanas sin incidentes.

f) Procedimientos de Rollback

Escenario	Acción de Rollback	Tiempo de Recuperación
Fase 1 (dry-run)	Detener contenedor IPcom Dialer	0 min (sin impacto)
Fase 2 (una campaña)	UPDATE routing table para campañas afectadas	< 2 min
Fase 3 (multi-campaña)	Batch UPDATE routing table	< 3 min
Fase 4 (cutover completo)	Batch UPDATE + iniciar contenedor Java	< 5 min
Corrupción de datos	Routing rollback + reparación manual desde `dy_marcador_log`	< 5 min tráfico, variable datos

El enfoque de routing table garantiza **rollback sub-5-minutos** en toda fase porque:

- Es un solo `UPDATE` atómico
- Ambos marcadores hacen poll cada 30 segundos
- El Java permanece running durante todas las fases shadow

g) Especificación del Dashboard de Métricas

Panel	Contenido
Throughput	Contactos marcados/hora por `dialer_source` — barras apiladas, buckets 15 min
Distribución de Estados	Pie charts lado a lado: Java vs IPcom (Contestada, No Contesta, Ocupado, Abandonada, Máquina, Error)
Calidad de Llamadas	Tasa de conexión, tasa de abandono, tiempo de ring — promedios móviles 1 hora
Salud del Sistema	Status AMI, latencia eventos, latencia BD (p50/p95/p99), `AsyncEventPump` queue depth y drops, CPU/memoria
Monitor Race Conditions	RC-1 a RC-4 por hora — verde (≤ baseline), amarillo (1-2× baseline), rojo (> 2× baseline)
Estado de Enrutamiento	Tabla: campaña, tenant, dialer asignado, última fecha de switch

h) Registro de Riesgos del Shadow Mode

Riesgo	Probabilidad	Impacto	Mitigación
Nuevas race conditions IPcom↔AgiAmi	Media	Alto	Fase 2 monitorea RC-1 a RC-4 específicamente. Rollback si > 2× baseline
`/trabajoCampana` retorna datos stale	Baja	Medio	Solo un dialer llama por campaña (enforced por routing table)
Permisos AMI de `marcador` ≠ `dyalogoami`	Baja	Alto	Verificado en Fase 0.1 con acciones idénticas
Exceder 20 canales por conteo erróneo	Baja	Alto	IPcom Dialer trackea canales independientemente. Validado en Fase 3

Riesgo	Probabilidad	Impacto	Mitigación
Agotamiento pool MySQL con dos dialers	Media	Medio	Dimensionar pools contando ambos. Monitorear conexiones
Overlap por delay en poll (30s)	Baja	Medio	Contactos en MARCANDO previenen doble marcación
Rate limiting REST con dos callers	Baja	Medio	Solo un dialer por campaña — volumen total no aumenta

i) Duración Total del Programa Shadow Mode

Fase	Duración	Acumulado
Fase 0: Preparación	1-2 semanas	2 semanas
Fase 1: Dry-run	1 semana	3 semanas
Fase 2: Una campaña	2 semanas	5 semanas
Fase 3: Expansión	2-4 semanas	7-9 semanas
Fase 4: Cutover	1 semana	8-10 semanas
Post-cutover monitoring	2 semanas	10-12 semanas

Total programa shadow: 8-10 semanas calendario (puede extenderse si gates no se cumplen). Descomisión Java tras 2 semanas adicionales post-cutover.

---

5. Arquitectura Objetivo en .NET

Propuesta: Monolito Modular Containerizado con Clean Architecture

Justificación de la elección:

Opción	Veredicto	Razón
Monolito modular containerizado	**Recomendado**	Proceso standalone en Docker, K8s-ready, escala horizontalmente por instancia de Asterisk
Clean Architecture	**Aplicar principios**	Separación de concerns, dependency inversion — no la estructura de carpetas dogmática
Microservicios	**No recomendado ahora**	Complejidad operativa innecesaria; migrar a microservicios cuando el volumen lo justifique
Servicios desacoplados	**Fase futura**	Cuando se requiera escalar CRM sync o reporting independientemente

Modelo de Deployment

```
Producción actual (1 servidor):
■■■
```

```
■  ■  AGI: 4573/  ■  ■  - Metrics → OTLP       ■  ■
■  ■       4574   ■  ■  - Health → /healthz     ■  ■
■  ■■■■■■■■■■■■■■■■■■  ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■  ■
■        ■                    ■                  ■
■        ■■■■■ AMI + AGI ■■■■■■■■                  ■
■                                                 ■
■  ■■■■■■■■■■■■■■■■■■  ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■  ■
■  ■ MySQL 8.0    ■  ■  Java AgiAmi (legacy)    ■  ■
■  ■ (compartido) ■  ■  AGI port 4573           ■  ■
■  ■■■■■■■■■■■■■■■■■■  ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■  ■
■■■■■■■■■■■■■■■■■■■
```

Diagrama de Arquitectura

```
■■■
```

Responsabilidad	Implementación
Configuración	`appsettings.json` + `IOptions<T>`
Lifecycle	`IHostedService` / `BackgroundService`
Health checks	ASP.NET Health Checks endpoint

Capa 2: Application Services

Servicio	Responsabilidad	Equivalente Java
`CampaignSchedulerService`	Loop principal, activa/desactiva campañas	`ManejaCampanasMarcacion`
`DialerEngineService`	Orquesta marcación por campaña	`HMarcacionDinamicoCampanaMaster`
`ContactDialerService`	Marca un contacto individual	`HEjecutaMarcacionContacto`
`CallResultProcessorService`	Procesa callbacks de originate	`DyalogoOriginateCallback`
`ContactUpdaterService`	Batch update de contactos	`HProcesadorActualizadorContactos`
`CrmSyncService`	Sincronización con CRM	`OperacionesDatosCRM` + `ConsumeREST`
`ActiveCallVerifierService`	Verificación de llamadas activas	`HVerificadorLlamadasActivas`
`AbandonedCallReinjectionService`	Re-inyecta llamadas abandonadas como contactos outbound	`HiloProcesaLlamadasAbandonadas` (AgiAmi)
`RejectedCallReinjectionService`	Re-inyecta llamadas rechazadas por agentes	`HiloProcesaLlamadasRechazadas` (AgiAmi)
`DialerContestaHumanoAgi`	AGI script: marca contacto como contestada por humano	`EventoMarcadorContestaHumano` (AgiAmi)
`DialerContestaMaquinaAgi`	AGI script: marca contacto como contestadora	`EventoMarcadorContestaMaquina` (AgiAmi)
`DialerAmdResponseAgi`	AGI script: respuesta AMD API	`EnviaRespuestaAMDAPI` (AgiAmi)
`DialerInsertSampleAgi`	AGI script: inserta registro en muestra	`InsertaRegistroMuestraMarcador` (AgiAmi)

Capa 3: Domain

Componente	Responsabilidad
`Campaign`	Entidad de campaña con scheduling y configuración
`Contact`	Contacto con hasta 10 teléfonos y estado de marcación
`Sample`	Muestra que agrupa contactos para una campaña
`DialAttempt`	Value object con resultado de intento de marcación
`RetryPolicy`	Política de reintentos configurable por campaña
`CampaignSchedule`	Reglas de horario y días de ejecución
`OutboundRoute`	Análisis de ruta saliente por número destino

Capa 4: Infrastructure

Componente	Tecnología	Responsabilidad
`ICampaignRepository`	Dapper + MySqlConnector	CRUD campañas y muestras
`IContactRepository`	Dapper + MySqlConnector	Queries y updates de contactos
`ICallLogRepository`	Dapper + MySqlConnector	Logging de llamadas y CDR

Componente	Tecnología	Responsabilidad
`IAmiConnection`	Asterisk.Sdk.Ami	Originate, event subscription
`IAsteriskServer`	Asterisk.Sdk.Live	Estado en tiempo real
`ICrmClient`	`HttpClient`	REST calls al CRM
`ICrmRepository`	Dapper + MySqlConnector	Escritura directa a BD CRM

Integración con Asterisk.Sdk

```
// Registro en DI (Host)
services.AddAsterisk(options =&gt;
{
    options.Ami.Hostname = config["Asterisk:Hostname"];
    options.Ami.Username = config["Asterisk:Username"];
    options.Ami.Password = config["Asterisk:Password"];
    options.Ami.AutoReconnect = true;
});

// Uso en DialerEngineService
public class ContactDialerService(
    IAmiConnection ami,
    IAsteriskServer server,
    IContactRepository contacts,
    ICallLogRepository callLog)
{
    public async Task&lt;DialResult&gt; DialContactAsync(
        Contact contact, Campaign campaign, CancellationToken ct)
    {
        var action = new OriginateAction
        {
            Channel = BuildChannel(contact.CurrentPhone, campaign),
            Context = campaign.DialplanContext,
            Exten = campaign.Extension,
            Priority = 1,
            CallerId = campaign.CallerId,
            Timeout = campaign.TimeoutMs,
            Async = true,
            Variables = BuildVariables(contact, campaign)
        };

        var response = await ami.SendActionAsync(action, ct);
        // Procesar resultado...
    }
}
```

---

6. Estrategia de Migración

Análisis de Opciones

Estrategia	Pros	Contras	Veredicto
Big Bang controlado	Corte limpio, sin deuda de coexistencia, menos complejidad operativa	Requiere validación exhaustiva previa	**Recomendado**
Strangler Fig	Reemplazo progresivo, rollback por componente	Complejidad de coexistencia (flags, dual AGI, dual abandoned handler) supera beneficios para un componente atómico como el marcador	No recomendado

Estrategia	Pros	Contras	Veredicto
Gradual por fases	Reduce riesgo teórico	El marcador es una unidad atómica — medio marcador no es funcional	No aplica
Paralela	Ambos sistemas corren, se compara output	Shadow mode cubre esto sin la complejidad operativa	Shadow mode para validación pre-corte

Estrategia Seleccionada: Reemplazo Completo con Validación en Shadow Mode

Fundamento: El marcador es una unidad funcional atómica. No tiene sentido migrar "medio marcador" porque las campañas dependen de todos los subsistemas (originate, callbacks, AGI, CRM sync) funcionando juntos. La complejidad de mantener dos marcadores simultáneos (flags por campaña, dos procesadores de abandonadas, dos AGI servers) introduce más riesgo del que evita.

Modelo de transición:

```
Fase 1 — Desarrollo y Testing (semanas 1-15):
                                          ■■■■■■■■■■■■■■■■■■■■■■■
■■■■■■■■■■■■■■■■■■■■■■■■AMI■■■■■■■■■■■■■■■■■■■■■■■■■  Java AgiAmi      ■
■  Java Marcador   ■        ■   Asterisk   ■  ■  (producción)     ■
■  (producción)    ■        ■   PBX        ■  ■  17 AGI scripts   ■
■■■■■■■■■■■■■■■■■■■■■        ■■■■■■■■■■■■■■■■■  ■■■■■■■■■■■■■■■■■■■■■■■
        ■
        ■■■■■■■■■■■■■■■■■ MySQL ■■■■■■■■■■■■■■■ (desarrollo local)
                         ▲                    ■■■■■■■■■■■■■■■■■■■■■■
                         ■                    ■  IPcom Dialer    ■
                         ■■■■■■■■■■■■■■■■■■■■■■■■■  (dev + Docker)  ■
                                              ■■■■■■■■■■■■■■■■■■■■■■

Fase 2 — Shadow Mode (semanas 16-17):
■■■■■■■■■■■■■■■■■■■■■■■■AMI■■■■■■■■■■■■■■■■■■■
```

Orden	Módulo	Razón de prioridad
1	Infraestructura (conexión AMI+AGI, DB)	Base para todo lo demás; Asterisk.Sdk ya lo resuelve
2	Modelo de dominio (entidades, value objects)	Mapeo directo de tablas Java a POCOs — incluye entidades de CBXLib usadas
3	Repositorios (DAOs → Dapper)	Queries SQL migran con mínima adaptación — incluye DAOs de CBXLib necesarios
4	**AGI Scripts del marcador**	4 scripts de AgiAmi → `FastAgiServer` de Asterisk.Sdk — desacopla del Java AgiAmi
5	Campaign Scheduler	Loop principal, más visible para validación
6	Dialer Engine (originate + callbacks)	Core funcional, requiere Asterisk de prueba
7	**Abandoned/Rejected reinjection**	Flujo de AgiAmi → incorporar en el marcador .NET
8	Lógica de reintentos y contactos	Complejidad media, reglas de negocio
9	PDS/Predictivo/Robótico	Lógica más compleja, requiere validación exhaustiva
10	Integración CRM	Último por dependencia externa
11	Observabilidad y operación	Transversal, se va construyendo en paralelo

---

7. Inventario de Componentes a Migrar

Clasificación por Tipo y Prioridad

7.1 Lógica de Negocio

Componente Java	Clase(s)	Prioridad	Complejidad	Equivalente .NET
Loop de campañas	`ManejaCampanasMarcacion`	P0	Media	`CampaignSchedulerService : BackgroundService`
Hilo de campaña	`HCampanaMarcacion`	P0	Media	`CampaignWorker` (per-campaign task)
Ejecución de contacto	`HEjecutaMarcacionContacto`	P0	Alta	`ContactDialerService`
PDS principal	`HMarcacionPDSDinamicoPrincipal`	P1	Alta	`PdsDialerService : BackgroundService`
PDS master campaña	`HMarcacionDinamicoCampanaMaster`	P1	Alta	`PdsCampaignOrchestrator`
Analizador de rutas	`AnalizadorRutaSaliente`	P1	Baja	`OutboundRouteResolver`
Utilidades	`UtilidadMarcador`	P2	Baja	Extension methods / helpers
Verificador llamadas	`HVerificadorLlamadasActivas`	P2	Media	`ActiveCallVerifierService`

Componentes de DyalogoCBXAgiAmi a migrar:

Componente Java	Clase(s) (AgiAmi)	Prioridad	Complejidad	Equivalente .NET
AGI contesta humano	`EventoMarcadorContestaHumano`	P0	Baja	`DialerContestaHumanoAgi : BaseAgiScript`
AGI contesta máquina	`EventoMarcadorContestaMaquina`	P0	Baja	`DialerContestaMaquinaAgi : BaseAgiScript`
AGI respuesta AMD	`EnviaRespuestaAMDAPI`	P1	Baja	`DialerAmdResponseAgi : BaseAgiScript`
AGI inserta registro muestra	`InsertaRegistroMuestraMarcador`	P1	Baja	`DialerInsertSampleAgi : BaseAgiScript`
Procesador abandonadas	`HiloProcesaLlamadasAbandonadas`	P0	Media	`AbandonedCallReinjectionService : BackgroundService`
Procesador rechazadas	`HiloProcesaLlamadasRechazadas`	P0	Media	`RejectedCallReinjectionService : BackgroundService`
Singleton abandonadas	`SingletonListaLlamadasAbandonadas`	P0	Baja	`Channel<AbandonedCallInfo>` (bounded)
Singleton rechazadas	`SingletonListaLlamadasRechazadas`	P0	Baja	`Channel<RejectedCallInfo>` (bounded)
Info AMI	`InformacionAMI`	P1	Baja	`IAsteriskServer.ChannelManager` (ya disponible)

Componentes de DyalogoCBXLib a extraer:

Componente Java	Clase(s) (CBXLib)	Prioridad	Complejidad	Equivalente .NET
Analizador rutas	`AnalizadorRutaSaliente`	P1	Media	`OutboundRouteResolver`
Config sistema	`ConfiguracionCBX` (singleton)	P0	Baja	`IOptions<DialerOptions>`
Constantes	`ConstantesCBX`	P0	Baja	`static class DialerConstants`
Utilidades fecha	`FuncionesFecha`	P1	Baja	`DateTimeOffset` + extension methods
HTTP client	`HTTPRequest`	P1	Baja	`IHttpClientFactory`
Conversiones num.	`ConversionesNumericas`	P1	Baja	Extension methods
Cifrado	`EncriptadorPropio` (AES-ECB) + `ManejaEncripcion` (wrapper)	P0	Baja	`LegacyEncryptor` — AES-ECB con key `D7@l0g0*S.A.S109`, reproducción trivial
Enum VoIP	`EnumTecnologiasVOIP`	P0	Baja	`enum VoipTechnology`
Cache agentes	`AgentesCache`	P2	Baja	`ConcurrentDictionary` o `IMemoryCache`
Códigos error	`CodigosError`	P1	Baja	`enum ErrorCode`
Numeración Colombia	`DyDefinicionNumeracionColombia`	P1	Baja	Lookup table o config

7.2 Acceso a Datos

DAO Java	Queries Principales	Prioridad	Equivalente .NET
`DaoMarcadorCampanas`	Campañas activas/inactivas	P0	`ICampaignRepository`
`DaoContactos`	Contactos pendientes por muestra	P0	`IContactRepository`
`DaoMuestras`	Muestras activas por campaña	P0	`ISampleRepository`
`DaoMarcadorLog`	Insert log de llamadas	P0	`ICallLogRepository`
`DAODyLlamadasSalientes`	Insert/update llamadas salientes	P1	`IOutboundCallRepository`
`DaoLlamadas`	Registro general de llamadas	P1	`ICallRepository`
`DaoTroncales`	Consulta troncales disponibles	P1	`ITrunkRepository`
`DaoCTDIA`	CTI data	P2	`ICtiRepository`
`DaoColas`	Colas de Asterisk	P2	`IQueueRepository`
`DAODyVariablesGlobales`	Variables globales del sistema	P2	`IGlobalVariableRepository`
`DAOMarcacionPDSLocal`	Datos PDS local	P2	`IPdsDataRepository`

DAOs de AgiAmi necesarios:

DAO Java (AgiAmi)	Queries Principales	Prioridad	Equivalente .NET
`DAODyMarcadorContactos` (AgiAmi)	Insert contacto abandonado, max prioridad	P0	Incorporar en `IContactRepository`
`DAODyMarcadorMuestrasCampanas` (AgiAmi)	Consulta muestra activa de campaña	P0	Incorporar en `ISampleRepository`
`DAODyCampanas` (AgiAmi)	Config de manejo de abandono por campaña	P0	Incorporar en `ICampaignRepository`
`DAODyLlamadas` (AgiAmi)	Insert llamada con validación	P1	Incorporar en `ICallRepository`
`DaoCTDIALocal` (AgiAmi)	Insert evento CTI	P2	`ICtiRepository`
`DAODyTiemposTimbrando` (AgiAmi)	Insert métrica de ring time	P2	`IRingTimeRepository`
`DAODyFestivos` (AgiAmi)	`esDiafestivo(fecha)`	P2	`IHolidayRepository`

DAOs de CBXLib referenciados:

DAO Java (CBXLib)	Uso por Marcador	Prioridad	Equivalente .NET
`DAODyAgentes` (CBXLib)	Lookup agente por extensión, email	P0	`IAgentRepository` (solo lectura)
`DAODyExtensiones` (CBXLib)	Extensiones por contexto	P1	`IExtensionRepository` (solo lectura)
`DAODyRutasSalientes` (CBXLib)	Rutas para análisis de número	P1	`IOutboundRouteRepository` (solo lectura)
`DAOMarcadorCampanas` (CBXLib)	Definición base de campañas	P0	Incorporar en `ICampaignRepository`
`DAODyListaNegraNumeros` (CBXLib)	Do-Not-Call list	P1	`IDncRepository`

7.3 Integración con Asterisk

Componente Java	Responsabilidad	Prioridad	Reemplazo Asterisk.Sdk
`ConexionAMI`	Conexión dual AMI	P0	`IAmiConnection` (una sola conexión)
`AccionesAsteriskAMI`	`sendOriginateActionAsync()`	P0	`ami.SendActionAsync(OriginateAction)`
`ManejadorEventos`	Listener: Hangup, Abandon, NewState	P0	`server.ChannelManager.ChannelRemoved` + observers
`DyalogoOriginateCallback`	Callback: Success/NoAnswer/Busy/Fail	P0	`OriginateResponse` + async/await
`CanalesDisponiblesColaAMI`	Canales disponibles en cola	P1	`server.QueueManager.GetQueue()`
`HCoreShowChannels`	"core show channels"	P1	`ami.SendActionAsync(CoreShowChannelsAction)`
`InformacionParaMarcador`	Info de canales activos	P1	`server.ChannelManager.GetChannel()`
`Configuracion`	Propiedades AMI	P0	`IOptions<AmiConnectionOptions>`

Integración AGI (de DyalogoCBXAgiAmi):

Componente Java	Responsabilidad	Prioridad	Reemplazo Asterisk.Sdk
`DefaultAgiServer` (port 4573)	FastAGI server Java	P0	`FastAgiServer` de Asterisk.Sdk.Agi (mismo puerto o configurable)
`EventoMarcadorContestaHumano`	AGI: set canal var + update BD	P0	`IDialerAgiScript` implementación
`EventoMarcadorContestaMaquina`	AGI: set canal var + update BD	P0	`IDialerAgiScript` implementación
`EnviaRespuestaAMDAPI`	AGI: AMD result processing	P1	`IDialerAgiScript` implementación
`InsertaRegistroMuestraMarcador`	AGI: insert en muestra	P1	`IDialerAgiScript` implementación
`HiloEscuchadorAMI` (AgiAmi)	AMI events: hangup, hold, agent	P1	`IAsteriskServer` event observers (ya disponible)
`SingletonConexionAmi` (AgiAmi)	Conexión AMI singleton	P0	`IAmiConnection` (una sola instancia DI)

> ***Nota crítica sobre AGI:*** *El dialplan de Asterisk invoca estos scripts vía* `AGI(agi://IP:4573/script)`*. Al migrar el FastAGI server a .NET, el dialplan debe apuntar al nuevo IP:puerto del servicio .NET, o ambos servidores AGI deben correr en puertos diferentes durante la transición.*

7.4 Integración CRM

Componente	Responsabilidad	Prioridad
`OperacionesDatosCRM`	INSERT CONDIA, UPDATE Muestra en BD CRM	P1
`ConsumeREST`	POST a `/bi/gestion/pdsprerob`	P1
`SingletonConexionCRM`	Conexión BD CRM singleton	P1
`HiloSincronizadorCRM`	Hilo de sincronización	P2

7.5 Configuración y Cross-Cutting

Componente	Prioridad	Equivalente .NET
`servicios_asterisk.properties`	P0	`appsettings.json` + `IOptions<T>`
`persistence.xml`	P0	Connection string en `appsettings.json`
`c3p0-config.xml`	P0	`MySqlConnectorPooling` (built-in)
`log4j.properties` (7 appenders)	P0	Serilog con sinks equivalentes
`PoolHilosMarcador` (50+20 threads)	P0	`Channel<T>` + `Task.WhenAll`

---

8. Plan por Fases

Fase 0: Descubrimiento y Levantamiento

Objetivo: Obtener toda la información necesaria para afinar el plan de migración y reducir incertidumbre.

Actividades principales:

#	Actividad	Responsable
0.1	Generar dump del esquema MySQL completo (`dyalogo_telefonia` + `DYALOGOCRM_SISTEMA`)	DBA
0.2	Documentar versión de Asterisk, módulos, contextos y extensiones usados	Infraestructura
0.3	Extraer métricas de producción: campañas/día, contactos/hora, llamadas concurrentes pico	Operaciones
0.4	Revisar proyectos hermanos `DyalogoCBXLib` y `DyalogoCBXAgiAmi`	Desarrollo
0.5	Documentar API contract de `/bi/gestion/pdsprerob` (request/response)	Equipo CRM
0.6	Identificar todos los servicios que leen/escriben en tablas compartidas	Desarrollo
0.7	Revisar scripts de despliegue y configuración de producción	DevOps
0.8	Entrevistar operadores para documentar reglas de negocio no escritas	Product Owner

Entradas: Acceso a código fuente, servidores de producción, equipo funcional. **Entregables:** Documento de levantamiento funcional-técnico, esquema de BD, API contracts, métricas baseline. **Riesgos:** Información dispersa o incompleta, dependencia de personas clave. **Criterios de finalización:** Todos los ítems I1-I10 de la sección de supuestos están resueltos. **Duración estimada:** 1-2 semanas.

Fase 1: Análisis Funcional y Técnico

Objetivo: Mapear al 100% el comportamiento del sistema Java para garantizar paridad funcional.

Actividades principales:

#	Actividad
1.1	Analizar cada clase Java y documentar su responsabilidad exacta

#	Actividad
1.2	Trazar flujos completos: marcación estándar, PDS, predictivo, robótico
1.3	Documentar todos los estados posibles de un contacto y transiciones
1.4	Mapear todas las queries SQL ejecutadas por los DAOs
1.5	Documentar la lógica de reintentos: cuándo, cuántos, con qué delay
1.6	Identificar edge cases: ¿qué pasa si Asterisk se cae? ¿si MySQL no responde?
1.7	Documentar el MDC logging pattern y qué información se traza por llamada
1.8	Crear matriz de trazabilidad: funcionalidad → clase Java → test esperado
1.9	**Analizar los 4 AGI scripts del marcador** en DyalogoCBXAgiAmi: variables de canal, updates a BD, interacción con dialplan
1.10	**Mapear flujo de abandonadas/rechazadas** en AgiAmi: trigger event → agrupación → re-inyección → email alert
1.11	**Identificar dependencias transitivas** de DyalogoCBXLib: qué clases/métodos usa realmente el Marcador
1.12	**Documentar algoritmo de EncriptadorPropio** para replicar en .NET
1.13	**Revisar dialplan de Asterisk**: contextos que invocan AGI scripts del marcador, formato `AGI(agi://host:port/script)`

Entradas: Código fuente Java, documento de levantamiento de Fase 0. **Entregables:** Documento de análisis funcional-técnico, diagramas de flujo, matriz de trazabilidad. **Riesgos:** Lógica oscura en código legacy sin documentación. **Criterios de finalización:** Cada flujo funcional tiene un diagrama y una lista de acceptance criteria. **Duración estimada:** 2-3 semanas.

Fase 2: Diseño de Arquitectura Objetivo

Objetivo: Definir la arquitectura .NET detallada, validarla con el equipo y obtener sign-off.

Actividades principales:

#	Actividad
2.1	Diseñar estructura de proyectos .NET (solution, projects, namespaces)
2.2	Definir interfaces de servicio y contratos de repositorio
2.3	Diseñar modelo de concurrencia: `Channel<T>` queues, `SemaphoreSlim` rate limiters
2.4	Definir estrategia de configuración (`IOptions<T>`, secrets, feature flags)
2.5	Diseñar esquema de logging/métricas/trazas (OpenTelemetry)
2.6	Validar compatibilidad de esquema MySQL con Dapper (tipos, nullables)
2.7	Definir estrategia de pruebas (unit, integration, Asterisk docker)
2.8	Crear ADRs (Architecture Decision Records) para decisiones clave
2.9	Revisión de arquitectura con equipo técnico

Entradas: Análisis de Fase 1, capacidades de Asterisk.Sdk documentadas. **Entregables:** Documento de arquitectura, ADRs, diagrama de componentes, interfaces definidas. **Riesgos:** Over-engineering, decisiones sin validación con realidad de producción. **Criterios de finalización:** ADRs aprobados, interfaces definidas, equipo alineado. **Duración estimada:** 1-2 semanas.

Fase 3: Preparación de Base Técnica en .NET

Objetivo: Crear el esqueleto del proyecto .NET con toda la infraestructura técnica lista.

Actividades principales:

#	Actividad
3.1	Crear solution `IPcom.Dialer.slnx` con proyectos: `IPcom.Dialer.Host`, `IPcom.Dialer.Domain`, `IPcom.Dialer.Application`, `IPcom.Dialer.Infrastructure`
3.2	Configurar `global.json`, `Directory.Build.props`, `Directory.Packages.props`
3.3	Agregar referencia a Asterisk.Sdk (NuGet o project reference)
3.4	Implementar `Program.cs` con Generic Host + `AddAsterisk()`
3.5	Configurar Serilog con sinks equivalentes a log4j (file daily, console, error)
3.6	Implementar health check endpoint
3.7	Configurar Docker compose para desarrollo (MySQL + Asterisk)
3.8	Crear proyecto de tests con xUnit + FluentAssertions + NSubstitute
3.9	Configurar CI pipeline (build + test)
3.10	Implementar POCOs de dominio (26 entidades) y mapeo Dapper

Entradas: Documento de arquitectura aprobado, esquema MySQL. **Entregables:** Solution .NET compilable, Docker compose, CI pipeline verde. **Riesgos:** Incompatibilidades de tipos MySQL-Dapper, configuración AOT. **Criterios de finalización:** `dotnet build` sin warnings, `dotnet test` con tests iniciales passing, Docker compose funcional. **Duración estimada:** 1-2 semanas.

Fase 4: Migración de Integración con Asterisk

Objetivo: Reemplazar `ConexionAMI` + `AccionesAsteriskAMI` + `ManejadorEventos` con Asterisk.Sdk.

Actividades principales:

#	Actividad
4.1	Implementar `IOriginateService` que wrappea `IAmiConnection.SendActionAsync(OriginateAction)`
4.2	Implementar procesamiento de `OriginateResponse` (mapeo a estados: Success/NoAnswer/Busy/Failure)
4.3	Suscribir a eventos AMI via `IAsteriskServer`: `HangupEvent`, `QueueCallerAbandonEvent`, `NewStateEvent`
4.4	Implementar `Channel<DialResult>` para desacoplar callback processing
4.5	Implementar rate limiter con `SemaphoreSlim` para llamadas simultáneas por campaña
4.6	Tests de integración con Asterisk Docker (originate → hangup flow)

#	Actividad
4.7	Validar reconexión automática (simular caída de Asterisk)

Entradas: Asterisk.Sdk configurado, Asterisk Docker. **Entregables:** `IOriginateService` funcional con tests de integración. **Riesgos:** Diferencias de comportamiento entre asterisk-java callbacks y Asterisk.Sdk async/await. **Criterios de finalización:** Originate → callback flow funciona end-to-end en Docker con 100% de cobertura de estados. **Duración estimada:** 2-3 semanas.

Fase 4B: Migración de AGI Scripts y Flujos de AgiAmi

Objetivo: Migrar los 4 AGI scripts del marcador al `FastAgiServer` de Asterisk.Sdk y los flujos de re-inyección de llamadas abandonadas/rechazadas.

Actividades principales:

#	Actividad
4B.1	Implementar `DialerContestaHumanoAgi` — equivalente a `EventoMarcadorContestaHumano`: lee variables de canal (`DY_AGI_*`), actualiza `DyMarcadorContactos.estado = "Contestada"`
4B.2	Implementar `DialerContestaMaquinaAgi` — equivalente a `EventoMarcadorContestaMaquina`: marca contacto como contestadora
4B.3	Implementar `DialerAmdResponseAgi` — equivalente a `EnviaRespuestaAMDAPI`
4B.4	Implementar `DialerInsertSampleAgi` — equivalente a `InsertaRegistroMuestraMarcador`
4B.5	Registrar los 4 scripts en el `FastAgiServer` de Asterisk.Sdk con `SimpleMappingStrategy`
4B.6	Implementar `AbandonedCallReinjectionService` — escucha `QueueCallerAbandonEvent`, agrupa por campaña cada 60s, si campaña tiene `manejoAbandono` activo: INSERT en `DyMarcadorContactos` con prefijo + truncamiento a 10 dígitos
4B.7	Implementar `RejectedCallReinjectionService` — escucha `AgentRingNoAnswerEvent`, misma lógica de re-inyección
4B.8	Implementar envío de email de alerta cuando hay llamadas abandonadas (si campaña tiene `manejoAbandonoEmail` configurado)
4B.9	Tests de integración: AGI script ejecutado desde dialplan de Asterisk Docker
4B.10	Tests de integración: flujo completo abandon → re-inject → dialer picks up contact
4B.11	**Definir estrategia de coexistencia AGI:** configurar dialplan para apuntar scripts de marcador al .NET FastAGI y el resto al Java FastAGI (diferente puerto o routing por script name)

Entradas: Asterisk.Sdk.Agi configurado, análisis de AGI scripts de Fase 1, Asterisk Docker con dialplan de prueba.

Entregables:

- 4 AGI scripts funcionales en .NET registrados en `FastAgiServer`
- `AbandonedCallReinjectionService` y `RejectedCallReinjectionService` funcionales
- Configuración de dialplan para routing de AGI scripts
- Tests de integración passing

Riesgos:

- El dialplan de Asterisk puede estar hardcodeado con IP:puerto del AGI Java
- Variables de canal (`DY_AGI_*`) deben coincidir exactamente con las que el dialplan espera
- Timing: el procesador de abandonadas Java (60s loop) podría competir con el .NET si ambos corren simultáneamente

Criterios de finalización:

- Los 4 AGI scripts responden correctamente desde dialplan de Asterisk Docker
- Flujo abandon→reinjection→dialer funciona end-to-end
- No hay conflicto entre AGI Java y AGI .NET durante coexistencia

Duración estimada: 2 semanas.

> ***Decisión de AGI en producción:*** *Al hacer corte atómico: 1.* ***IPcom Dialer FastAgiServer*** *en puerto* ***4574*** *— procesa los 4 scripts del marcador 2.* ***Java AgiAmi*** *sigue en puerto* ***4573*** *— procesa los 13 scripts restantes (no marcador) 3.* ***Dialplan actualizado en el corte:*** *scripts de marcador →* `AGI(agi://host:4574/script)`*, resto →* `AGI(agi://host:4573/script)` *4. No hay período de coexistencia de dos marcadores — el cambio de dialplan y el corte de servicio son atómicos*

Fase 5: Migración de Reglas de Negocio de Campañas

Objetivo: Migrar toda la lógica de scheduling, selección de contactos y marcación.

Actividades principales:

#	Actividad
5.1	Implementar `CampaignSchedulerService` (loop de 20s, evaluación hora/día)
5.2	Implementar `CampaignWorker` (tarea por campaña activa)
5.3	Implementar `ContactDialerService` (iteración de 10 teléfonos, retry logic)
5.4	Implementar `RetryPolicy` (máximo reintentos, delay, backoff)
5.5	Implementar `OutboundRouteResolver` (análisis de ruta por número — migrar `AnalizadorRutaSaliente` de CBXLib)
5.6	Implementar PDS engine (`PdsDialerService`, aceleración dinámica)
5.7	Implementar modo predictivo (ajuste de llamadas simultáneas por disponibilidad de agentes — usar `IAsteriskServer.AgentManager` en vez de queries AMI directas)
5.8	Implementar modo robótico (marcación masiva sin agentes)
5.9	Implementar anti-thundering-herd (jitter delay formal)
5.10	Implementar `DncService` — consulta `DyListaNegraNumeros` (CBXLib) antes de marcar
5.11	Implementar `EncryptionService` — replicar algoritmo de `EncriptadorPropio` para validar claves de agente
5.12	Unit tests exhaustivos para cada regla de negocio

Entradas: Análisis funcional de Fase 1, `IOriginateService` de Fase 4, AGI scripts de Fase 4B. **Entregables:** Motor de marcación completo con tests unitarios. **Riesgos:** Reglas de negocio implícitas no documentadas, timing differences entre Java threads y .NET tasks, algoritmo de cifrado incompatible. **Criterios de finalización:** Todos los flujos documentados en Fase 1 tienen tests passing. Coverage > 80% en lógica de negocio. **Duración estimada:** 3-4 semanas.

Fase 6: Migración de Persistencia y Configuración

Objetivo: Migrar todos los DAOs a repositorios Dapper y la configuración a `IOptions<T>`.

Actividades principales:

#	Actividad
6.1	Implementar todos los repositorios Dapper (ICampaignRepository, IContactRepository, etc.)
6.2	Migrar queries SQL de Hibernate HQL a SQL nativo para Dapper
6.3	Implementar connection management con `MySqlConnector` pooling
6.4	Implementar `IOptions<DialerOptions>` con validación AOT-safe
6.5	Migrar `servicios_asterisk.properties` a `appsettings.json`
6.6	Implementar secret management para credenciales
6.7	Tests de integración contra MySQL Docker

Entradas: Esquema MySQL, queries extraídas de DAOs Java. **Entregables:** Repositorios Dapper funcionales, configuración migrada. **Riesgos:** Diferencias de comportamiento Hibernate vs. Dapper (lazy loading, cascades, etc.). **Criterios de finalización:** Todos los repositorios tienen tests de integración passing contra MySQL. **Duración estimada:** 2-3 semanas.

Fase 7: Migración de CRM y Pruebas Integrales

Objetivo: Completar integración CRM y ejecutar pruebas end-to-end.

Actividades principales:

#	Actividad
7.1	Implementar `ICrmRepository` (INSERT CONDIA, UPDATE Muestra)
7.2	Implementar `ICrmRestClient` (POST a `/bi/gestion/pdsprerob`)
7.3	Implementar `CrmSyncService` (background sync)
7.4	Pruebas end-to-end: campaña completa desde activación hasta cierre
7.5	Pruebas de regresión: comparar output Java vs .NET con mismos datos
7.6	Pruebas de carga: simular volúmenes de producción
7.7	Pruebas de resiliencia: caída de Asterisk, caída de MySQL, timeout CRM
7.8	Revisión de seguridad: credenciales, SQL injection, input validation
7.9	Pruebas de aceptación funcional con equipo de operaciones

Entradas: Motor de marcación completo, repositorios, CRM integration. **Entregables:** Sistema completo validado, reporte de pruebas. **Riesgos:** Diferencias sutiles de comportamiento entre Java y .NET. **Criterios de finalización:** Todas las pruebas passing, sign-off de operaciones. **Duración estimada:** 2-3 semanas.

Fase 8: Salida Controlada a Producción

Objetivo: Desplegar en producción con mínimo riesgo operativo.

Actividades principales:

#	Actividad
8.1	Desplegar IPcom Dialer en modo shadow (lee campañas, simula marcación sin ejecutar originates)
8.2	Comparar decisiones de marcación: Java real vs IPcom Dialer shadow durante 1-2 semanas
8.3	Corregir discrepancias encontradas en shadow mode
8.4	**Corte:** en ventana de mantenimiento: apagar Java Marcador → actualizar dialplan AGI → iniciar IPcom Dialer
8.5	Monitoreo intensivo primeras 4 horas: latencia de originate, tasa de éxito, AGI callbacks, abandonadas
8.6	Mantener Java como fallback (binario disponible, rollback < 5 min)
8.7	Documentar runbook operativo para equipo de soporte

Entradas: Sistema validado en Fase 7, acceso a producción. **Entregables:** Sistema .NET en producción, runbook operativo. **Riesgos:** Problemas no detectados en testing, impacto en operación. **Criterios de finalización:** IPcom Dialer en producción con 100% de campañas durante 2 semanas sin incidentes P1/P2. **Duración estimada:** 2-3 semanas (shadow mode + corte + observación inicial).

Fase 9: Estabilización y Retiro del Sistema Legado

Objetivo: Confirmar estabilidad y descomisionar el servicio Java.

Actividades principales:

#	Actividad
9.1	Monitorear operación .NET durante 4 semanas post-migración completa
9.2	Resolver issues de estabilización encontrados
9.3	Optimizar configuración basada en métricas reales (pool sizes, timeouts)
9.4	Desactivar servicio Java de producción (mantener artefacto para rollback)
9.5	Limpiar infraestructura Java (JRE, jars, logs, cron jobs)
9.6	Documentar lecciones aprendidas
9.7	Transferencia de conocimiento al equipo de soporte

Entradas: Sistema .NET estable en producción. **Entregables:** Java descomisionado, documentación completa, knowledge transfer. **Riesgos:** Regresiones tardías, pérdida de conocimiento del sistema anterior. **Criterios de finalización:** Java apagado durante 2 semanas sin necesidad de rollback. **Duración estimada:** 2-4 semanas.

---

9. Mapeo Tecnológico Java → .NET

Equivalencias de Stack

Concepto	Java (Actual)	.NET (Objetivo)	Notas
Runtime	JRE 8+	.NET 10 Native AOT	Binario self-contained
Build	Ant (`build.xml`)	MSBuild + `slnx`	`dotnet build`
Dependencias	JARs manuales	NuGet + `Directory.Packages.props`	Versiones centralizadas
Entry Point	`main()` + `Thread.start()`	`IHost` + `BackgroundService`	Generic Host lifecycle
Servicio daemon	`extends Thread`	`BackgroundService.ExecuteAsync()`	CancellationToken graceful shutdown
DI	Singletons manuales	`IServiceCollection` / `IServiceProvider`	Constructor injection
Configuración	`.properties` files	`appsettings.json` + `IOptions<T>`	Validación AOT-safe con `[OptionsValidator]`
Secrets	Hardcoded en XML/properties	User Secrets / Azure Key Vault / env vars	Nunca en código fuente
ORM	Hibernate JPA 2.2	Dapper (micro-ORM)	SQL explícito, AOT-compatible
Connection Pool	C3P0 (min=5, max=100)	MySqlConnector built-in pooling	`Pooling=true;MinPoolSize=5;MaxPoolSize=30`
DB Driver	MySQL Connector/J 5.1.49	MySqlConnector (ADO.NET)	AOT-compatible, async nativo
Logging	Log4j 1.2.16 + MDC	Serilog + enrichers → stdout JSON	Container-native: stdout en producción, Docker/K8s recolecta
Métricas	No existente	`System.Diagnostics.Metrics`	Counters, histograms, gauges
HTTP Client	Manual (HTTPRequest class)	`IHttpClientFactory` + `HttpClient`	Pooling, retry policies, typed clients
JSON	Gson 2.8.6	`System.Text.Json` source-generated	AOT-safe, zero-reflection
Concurrencia	`Thread` + `ExecutorService` (50+20)	`Task` + `Channel<T>` + `SemaphoreSlim`	Async/await nativo
Thread Pool	`Executors.newFixedThreadPool()`	`Channel<T>` (bounded) + consumers	Backpressure integrado
Colecciones concurrentes	`synchronized` blocks	`ConcurrentDictionary<K,V>`	Lock-free reads
Timers	`Thread.sleep()` en loop	`PeriodicTimer`	Async, no bloquea thread
AMI Client	asterisk-java 3.41.0	Asterisk.Sdk.Ami	Zero-copy pipelines, source generators
AMI Events	`ManagerEventListener`	`IObservable<T>` + observers	Reactive, type-safe
AMI Reconnect	Manual (5 intentos, 5s delay)	Auto-reconnect built-in	Configurable en `AmiConnectionOptions`
AGI Server	asterisk-java `DefaultAgiServer` (port 4573)	Asterisk.Sdk.Agi `FastAgiServer`	Pipeline-based, AOT-compatible
AGI Scripts	`extends BaseAgiScript`	`implements IAgiScript`	Async, inyección de dependencias
AGI Mapping	`MappingStrategy` (reflection)	`SimpleMappingStrategy` (explicit)	AOT-safe, sin reflection
Cifrado	`EncriptadorPropio` (custom)	`System.Security.Cryptography`	Replicar algoritmo exacto

Concepto	Java (Actual)	.NET (Objetivo)	Notas
Cache framework	`ICache<T>` + `ObjetoCache<T>` (CBXLib)	`IMemoryCache` / `ConcurrentDictionary`	Built-in en .NET
Tests	No existentes	xUnit + FluentAssertions + NSubstitute	TDD desde el inicio
Caching	`Guava Cache`	`IMemoryCache` / `ConcurrentDictionary`	Built-in en .NET

Mapeo de Patrones de Concurrencia

Patrón Java	Equivalente .NET	Ejemplo
`extends Thread` + `run()`	`BackgroundService` + `ExecuteAsync()`	Campaign scheduler
`ExecutorService.submit(Runnable)`	`Channel<T>.Writer.WriteAsync()` + consumer tasks	Originate queue
`synchronized(this)`	`lock` / `SemaphoreSlim`	Rate limiter
`Thread.sleep(ms)`	`await Task.Delay(ms, ct)`	Jitter delay
`volatile` field	`volatile` / `Interlocked`	Campaign state
Singleton pattern	`services.AddSingleton<T>()`	Active call tracker
`CountDownLatch`	`SemaphoreSlim` / `TaskCompletionSource`	Wait for callback
`ConcurrentHashMap`	`ConcurrentDictionary<K,V>`	Active campaigns

---

10. Diseño de Integración con Asterisk

Distribución de Responsabilidades

```
═══════════════════════════════════    ═══════════════════════════════════
■     Asterisk.Sdk (Librería)     ■    ■  IPcom.Dialer (Aplicación)      ■
═══════════════════════════════════    ═══════════════════════════════════
■                                 ■    ■                                 ■
■  ✓ Conexión TCP AMI             ■    ■  ✓ Qué campañas marcar          ■
■  ✓ Autenticación MD5            ■    ■  ✓ Cuándo marcar (schedule)     ■
■  ✓ Serialización de acciones    ■    ■  ✓ A quién marcar (contactos)   ■
■  ✓ Deserialización de eventos   ■    ■  ✓ Cuántas veces (retries)      ■
■  ✓ Reconexión automática        ■    ■  ✓ Rate limiting por campaña    ■
■  ✓ Event pump async             ■    ■  ✓ Procesamiento de resultados  ■
■  ✓ Estado en tiempo real        ■    ■  ✓ Actualización de BD          ■
■  ✓ Métricas de conexión         ■    ■  ✓ Integración CRM              ■
■  ✓ Channel/Queue/Agent tracking ■    ■  ✓ Lógica PDS/Predictivo        ■
■  ✓ Multi-server pool            ■    ■  ✓ Observabilidad de negocio    ■
■                                 ■    ■                                 ■
■  ✗ NO decide qué marcar         ■    ■  ✗ NO parsea protocolo AMI      ■
■  ✗ NO tiene lógica de campaña   ■    ■  ✗ NO maneja TCP                ■
■  ✗ NO persiste datos            ■    ■  ✗ NO deserializa eventos       ■
═══════════════════════════════════    ═══════════════════════════════════
```

Patrón de Integración

```
// 1. Servicio de marcación (Application layer)
public interface IOriginateService
{
    Task&lt;DialResult&gt; OriginateAsync(DialRequest request, CancellationToken ct);
}
```

```
// 2. Implementación usa Asterisk.Sdk (Infrastructure layer)
public sealed class AsteriskOriginateService(
    IAmiConnection ami,
    IAsteriskServer server,
    ILogger&lt;AsteriskOriginateService&gt; logger) : IOriginateService
{
    public async Task&lt;DialResult&gt; OriginateAsync(DialRequest request, CancellationToken ct)
    {
        var action = new OriginateAction
        {
            Channel = $"SIP/{request.TrunkName}/{request.PhoneNumber}",
            Context = request.Context,
            Exten = request.Extension,
            Priority = 1,
            CallerId = request.CallerId,
            Timeout = request.TimeoutMs,
            Async = true,
            ActionId = request.CorrelationId
        };

        foreach (var (key, value) in request.Variables)
            action.SetVariable(key, value);

        var response = await ami.SendActionAsync(action, ct);

        return response.IsSuccess()
            ? DialResult.Initiated(request.CorrelationId)
            : DialResult.Failed(response.Message);
    }
}

// 3. El ContactDialerService NO conoce AMI ni Asterisk.Sdk
public sealed class ContactDialerService(
    IOriginateService originate,
    IContactRepository contacts,
    ICallLogRepository callLog)
{
    // Lógica de negocio pura, testeable con mock de IOriginateService
}
```

Suscripción a Eventos

```
// En el Host, se suscribe a eventos relevantes
public sealed class AsteriskEventSubscriber(
    IAsteriskServer server,
    ICallResultProcessor processor) : BackgroundService
{
    protected override Task ExecuteAsync(CancellationToken ct)
    {
        // Evento de hangup → procesar resultado de llamada
        server.ChannelManager.ChannelRemoved += (sender, args) =&gt;
        {
            processor.EnqueueHangup(args.Channel);
        };

        // Evento de abandon en cola
        server.QueueManager.CallerLeft += (sender, args) =&gt;
        {
            if (args.Reason == LeaveReason.Abandoned)
                processor.EnqueueAbandon(args.Caller);
        };

        return Task.CompletedTask;
    }
}
```

Recomendaciones de Diseño

1. **Dependency Inversion:** La aplicación define `IOriginateService`; la infraestructura lo implementa con Asterisk.Sdk. Esto permite testear toda la lógica de negocio sin Asterisk real.

2. **Event-driven desacoplado:** Usar `Channel<T>` como buffer entre eventos AMI y procesamiento de negocio. Esto aísla el throughput de eventos del procesamiento de BD.
3. **Idempotencia:** Cada originate debe tener un `ActionId` único (GUID). Si se recibe un duplicado, se ignora. Esto previene doble marcación en caso de reconexión.
4. **Circuit breaker:** Si Asterisk no responde a N originates consecutivos, pausar marcación de la campaña afectada (no de todas).
5. **Testabilidad:** `NSubstitute` para mockear `IAmiConnection` y simular respuestas de Asterisk en unit tests.

---

11. Riesgos y Mitigaciones

11.1 Riesgos Funcionales

#	Riesgo	Probabilidad	Impacto	Mitigación
RF1	Reglas de negocio no documentadas se pierden	Alta	Alto	Análisis exhaustivo en Fase 1; ejecución en shadow mode en Fase 8
RF2	Diferencias sutiles en lógica de reintentos	Media	Alto	Tests de comparación Java vs .NET con mismos datasets
RF3	Timing de marcación diferente (async/await vs threads)	Media	Medio	Benchmarks comparativos; jitter configurable
RF4	~~Estados de contacto inconsistentes durante coexistencia~~	—	—	**ELIMINADO** — no hay coexistencia de marcadores; corte atómico
RF5	~~**Flujo de abandonadas duplicado**~~	—	—	**ELIMINADO** — al apagar Java Marcador, también se desactiva el procesador de abandonadas del marcador Java
RF6	**AGI scripts devuelven variables de canal incorrectas** y el dialplan toma decisiones erróneas	Media	Crítico	Tests de integración AGI exhaustivos en Fase 4B; validar mismas variables `DY_AGI_*`

11.2 Riesgos Técnicos

#	Riesgo	Probabilidad	Impacto	Mitigación
RT1	Incompatibilidad Dapper con tipos MySQL específicos	Baja	Medio	Pruebas tempranas de mapeo en Fase 3
RT2	AOT trimming elimina código necesario	Baja	Medio	Asterisk.Sdk ya validado; tests de publicación AOT
RT3	Memory leak en long-running service	Media	Alto	Monitoring con dotnet-counters; stress tests
RT4	Deadlock en nuevo modelo de concurrencia	Baja	Alto	Revisión de código, uso de `Channel<T>` sin locks

#	Riesgo	Probabilidad	Impacto	Mitigación
RT5	~~**Algoritmo de EncriptadorPropio** no reproducible en .NET~~	~~Media~~	~~Medio~~	**MITIGADO** — AES-ECB estándar, reproducible con `System.Security.Cryptography.Aes`. Key y modo documentados en sección 4.3
RT6	**Dependencias transitivas de CBXLib** no identificadas (clases que se usan indirectamente)	Media	Medio	Análisis estático de imports en Fase 1; compilar incrementalmente

11.3 Riesgos Operativos

#	Riesgo	Probabilidad	Impacto	Mitigación
RO1	Downtime durante corte	Media	Alto	Ventana de mantenimiento en horario de baja actividad; rollback < 5 min
RO2	Equipo no familiarizado con .NET	Media	Medio	Capacitación previa; pair programming
RO3	Rollback necesario en producción	Baja	Medio	Java siempre disponible como fallback durante Fase 8

11.4 Riesgos de Compatibilidad con Asterisk

#	Riesgo	Probabilidad	Impacto	Mitigación
RA1	Versión de Asterisk con AMI no estándar	Baja	Alto	Asterisk.Sdk basado en asterisk-java 3.42.0; validar versión
RA2	Contextos/extensiones custom en dialplan	Media	Medio	Documentar en Fase 0; mismos contextos aplican
RA3	Diferencia en timeout de originate	Baja	Medio	Parametrizar; mismos valores que Java
RA4	Eventos AMI diferentes por versión de Asterisk	Baja	Medio	Asterisk.Sdk soporta 215 eventos; validar en staging
RA5	**Dialplan hardcodea IP:puerto del AGI Java** (4573) — cambiar a IPcom Dialer (4574) requiere editar dialplan de producción	Alta	Alto	Documentar dialplan en Fase 0; hacer cambio atómico con rollback script — el cambio es parte del corte
RA6	**Dos FastAGI servers post-corte** (IPcom Dialer=4574 para marcador, Java AgiAmi=4573 para resto)	Baja	Medio	Documentar en runbook; post-corte es el estado estable, no transitorio

11.5 Riesgos de Rendimiento

#	Riesgo	Probabilidad	Impacto	Mitigación
RP1	Mayor latencia de originate en .NET vs Java	Baja	Medio	Asterisk.Sdk usa Pipelines (más rápido); benchmark
RP2	Pool de conexiones MySQL insuficiente	Baja	Medio	Monitorear con métricas; ajustar pool size
RP3	GC pauses en long-running service	Baja	Bajo	Native AOT tiene GC optimizado; monitorear
RP4	Backpressure en Channel<T> bajo carga	Baja	Medio	Bounded channels con drop policy; métricas

11.6 Riesgos de Transición Productiva

#	Riesgo	Probabilidad	Impacto	Mitigación
RT1	~~Dos sistemas marcando al mismo contacto~~	—	—	**ELIMINADO** — corte atómico, un solo sistema activo
RT2	~~Datos de log divididos entre dos sistemas~~	—	—	**ELIMINADO** — corte atómico
RT3	Monitoreo insuficiente del nuevo sistema	Media	Alto	Dashboards ready antes de Fase 8
RT4	**Cambio de dialplan AGI falla** y los scripts del marcador no responden	Media	Crítico	Script de rollback de dialplan pre-probado; ambos cambios (dialplan + servicio) en una ventana atómica
RT5	~~**Campañas con manejo de abandono** procesan duplicados~~	—	—	**ELIMINADO** — un solo procesador activo

---

12. Plan de Pruebas

Tipos de Pruebas

Tipo	Herramientas	Alcance	Cuándo
Unitarias	xUnit + FluentAssertions + NSubstitute	Lógica de negocio, servicios, domain	Continuo (TDD)
Integración BD	xUnit + Testcontainers (MySQL)	Repositorios Dapper, queries	Fases 3, 6
Integración Asterisk	xUnit + Docker (Asterisk)	Originate, eventos, reconnect	Fases 4, 7
End-to-End	xUnit + Docker Compose completo	Flujo campaña completa	Fase 7
Regresión	Comparación output Java vs .NET	Mismos inputs → mismos resultados	Fase 7
Carga/Concurrencia	BenchmarkDotNet + custom load gen	N campañas × M contactos simultáneos	Fase 7
Resiliencia	Chaos testing (kill containers)	Caída de Asterisk/MySQL/CRM	Fase 7
Aceptación funcional	Manual con equipo de operaciones	Flujos de producción reales	Fases 7, 8

Tipo	Herramientas	Alcance	Cuándo
Shadow testing	Log comparison	Decisiones Java real vs .NET simulado	Fase 8

Ambientes de Prueba

Ambiente	Componentes	Propósito
Local/Dev	Docker Compose: MySQL + Asterisk + .NET	Desarrollo y unit/integration tests
Staging	Réplica de producción con datos sanitizados	End-to-end, carga, aceptación
Shadow	.NET lee producción, no ejecuta originates	Validación de decisiones de marcación
Producción piloto	.NET con 1-2 campañas de bajo volumen	Validación final antes de rollout

Métricas de Calidad Requeridas

Métrica	Umbral
Cobertura de código (lógica de negocio)	> 80%
Tests de integración Asterisk passing	100%
Tests de regresión Java vs .NET	> 99% paridad
Tiempo de originate p99	≤ Java + 10%
Zero P1/P2 bugs en shadow mode	2 semanas continuas

---

13. Plan de Despliegue

Estrategia de Salida a Producción

```
Semana 1-2: Shadow Mode
■■■ IPcom Dialer desplegado junto a Java Marcador
■■■ Lee campañas y contactos de MySQL (read-only)
■■■ Simula decisiones de marcación (NO originate)
■■■ Compara decisiones con Java real (logs de divergencia)
■■■ Corrige discrepancias encontradas

Semana 3: Corte (Big Bang controlado)
■■■ Ventana de mantenimiento en horario de baja actividad
■■■ Apagar Java Marcador
■■■ Actualizar dialplan de Asterisk (AGI scripts marcador → IPcom Dialer FastAGI)
■■■ Iniciar IPcom Dialer en modo producción
■■■ Monitoreo intensivo primeras 4 horas
■■■ Rollback: revertir dialplan + reiniciar Java (&lt; 5 minutos)

Semana 4-6: Estabilización
■■■ Monitoreo continuo
■■■ Java Marcador apagado pero binario disponible como fallback
■■■ Resolver issues de estabilización
■■■ Confirmación de retiro definitivo del Java Marcador
```

> ***Nota sobre el esquema de BD:*** *No se modifica ninguna tabla. IPcom Dialer opera sobre el mismo esquema MySQL que el marcador Java. Esto incluye las tablas* `dy_marcador_*` *en* `dyalogo_telefonia`*, las tablas CRM en* `DYALOGOCRM_SISTEMA`*, y las vistas de* `asterisk`*. Los nombres de tabla mantienen el prefijo* `dy_` *del sistema legacy.*

Validaciones Previas al Despliegue

- ■ Todos los tests passing en CI
- ■ Imagen Docker construida y publicada en registry (`docker build` exitoso)
- ■ Publicación AOT exitosa dentro del contenedor (0 trim warnings)
- ■ Health check endpoint responde 200 (`/healthz`)
- ■ Readiness probe responde 200 (`/ready`) — conexión AMI + MySQL OK
- ■ Conexión AMI exitosa a Asterisk de producción desde contenedor
- ■ Conexión MySQL exitosa a BD de producción desde contenedor
- ■ Logs JSON visibles en `docker logs` / sistema de recolección
- ■ Métricas reportando a Prometheus/dashboard de monitoreo
- ■ Runbook operativo revisado por equipo de soporte
- ■ Variables de entorno documentadas y configuradas

Validaciones Posteriores al Despliegue

- ■ Campañas activas se detectan correctamente
- ■ Originates se ejecutan y callbacks se procesan
- ■ Contactos se actualizan en BD
- ■ Logs contienen contexto correcto (campaña/contacto/llamada)
- ■ CRM recibe actualizaciones
- ■ Métricas de tasa de éxito comparables a Java
- ■ Sin errores P1/P2 en primera hora

Plan de Rollback

Paso	Acción	Tiempo
1	Apagar IPcom Dialer: `docker compose stop ipcom-dialer`	< 30 segundos
2	Revertir dialplan de Asterisk: `asterisk -rx "dialplan reload"` con config original	< 1 minuto
3	Reiniciar Java Marcador: `docker compose start dyalogo-marcador` (o `systemctl start` si bare-metal)	< 1 minuto
4	Verificar que campañas activas se retoman en Java	< 2 minutos
Total rollback		**< 5 minutos**

> *El esquema de BD no cambia entre Java y .NET, por lo que el rollback es puramente de servicio — no hay migración de datos que revertir.*

---

14. Recomendaciones de Observabilidad

Logging Estructurado (Serilog) — Container-Native

Principio: En entornos containerizados (Docker/K8s), los logs van a **stdout/stderr en formato JSON**. No se escriben archivos de log dentro del contenedor. La recolección, agregación y retención la maneja la infraestructura

(Docker log driver, Fluentd, Loki, EFK stack).

Sink	Entorno	Configuración
Console (JSON)	**Producción (Docker/K8s)**	`WriteTo.Console(new JsonFormatter())` → stdout. Docker/K8s recolecta automáticamente
Console (plain text)	Desarrollo local	`WriteTo.Console()` — legible para humanos
File (opcional)	Debug local / staging	`/var/log/ipcom/dialer/debug-{Date}.log` — solo si se monta volumen explícitamente

> ***Por qué no archivos de log en producción:*** *- Los contenedores son efímeros — al reiniciar se pierden los logs - Docker captura stdout/stderr automáticamente (`docker logs`) - K8s expone logs via `kubectl logs` y los envía a Loki/EFK - Los archivos de log dentro del contenedor consumen espacio del filesystem overlay y degradan rendimiento - Rotación de logs es responsabilidad de la infraestructura, no de la aplicación*

Enrichers obligatorios (equivalente a MDC de log4j):

```
Log.ForContext("TenantId", campaign.TenantId)
   .ForContext("CampaignId", campaign.Id)
   .ForContext("ContactId", contact.Id)
   .ForContext("CallId", callId)
   .ForContext("AgentId", agentId)
   .Information("Originate sent to {PhoneNumber}", phone);

// En producción (Docker/K8s) el output es JSON a stdout:
// {"@t":"2026-03-07T14:30:00Z","@l":"Information","TenantId":5,
//  "CampaignId":42,"ContactId":1234,"CallId":"abc-123",
//  "PhoneNumber":"573001234567","@m":"Originate sent to 573001234567"}
```

Métricas (System.Diagnostics.Metrics)

Métrica	Tipo	Descripción
`ipcom.dialer.campaigns.active`	ObservableGauge (tag: tenant)	Campañas activas por tenant
`ipcom.dialer.contacts.dialed`	Counter (tag: tenant)	Contactos marcados por tenant
`ipcom.dialer.contacts.dialed_by_result`	Counter (tag: result, tenant)	Por resultado y tenant
`ipcom.dialer.originate.duration`	Histogram	Tiempo desde originate hasta callback (ms)
`ipcom.dialer.originate.active`	ObservableGauge	Originates en curso
`ipcom.dialer.originate.queue_depth`	ObservableGauge	Items en cola de marcación
`ipcom.dialer.retries.count`	Counter	Reintentos ejecutados
`ipcom.dialer.crm.sync_duration`	Histogram	Tiempo de sincronización CRM (ms)
`ipcom.dialer.crm.errors`	Counter	Errores de integración CRM
`ipcom.dialer.db.query_duration`	Histogram (tag: query)	Tiempo de queries a MySQL

Trazabilidad

Evento	Información Mínima
Campaña activada	campaignId, nombre, hora inicio, simultaneas
Campaña desactivada	campaignId, nombre, razón, contactos procesados
Contacto seleccionado	contactId, campaignId, teléfono seleccionado, intento #

Evento	Información Mínima
Originate enviado	callId, contactId, channel, callerID, timeout
Callback recibido	callId, contactId, resultado, duración
Reintento programado	contactId, teléfono siguiente, delay
CRM actualizado	contactId, endpoint, httpStatus, duración
Error de conexión AMI	hostname, error, intento de reconexión #
Error de BD	query, error, duración

Alertas Sugeridas

Alerta	Condición	Severidad
AMI desconectado	Sin conexión AMI > 30 segundos	P1 - Crítica
Tasa de error > 50%	`failure / (success + failure)` > 0.5 en 5 min	P1 - Crítica
MySQL no responde	Query timeout > 10 consecutivos	P1 - Crítica
Cola de originates saturada	`queue_depth` > 80% capacidad	P2 - Alta
CRM sync fallando	5 errores consecutivos	P2 - Alta
Campaña sin actividad	Campaña activa sin originates en 5 min	P3 - Media
Memory > 80%	Working set > 80% del límite	P3 - Media
GC pause > 500ms	Gen2 collection > 500ms	P3 - Media

Auditoría de Campañas y Marcación

Registro	Tabla	Campos Mínimos
Ejecución de campaña	`dy_marcador_log`	timestamp, campaignId, action, result
Intento de marcación	`dy_marcador_log`	timestamp, contactId, phone, callId, result
Cambio de estado contacto	`dy_marcador_contactos`	contactId, estado_anterior, estado_nuevo, timestamp
Resultado de originate	`dy_respuestas_originate`	callId, actionId, result, duration
Efectividad de campaña	`dy_marcador_efectividad_campanas`	campaignId, periodo, total, exitosas, %

---

15. Backlog Inicial de Trabajo

Épicas

Épica	Descripción	Fase
E1	Levantamiento y análisis del sistema Java	0-1
E2	Arquitectura y diseño .NET	2
E3	Infraestructura técnica base	3
E4	Integración Asterisk (AMI) vía SDK	4
E4B	Migración AGI scripts y flujos de AgiAmi	4B

Épica	Descripción	Fase
E5	Motor de marcación de campañas	5
E6	Persistencia y configuración	6
E7	Integración CRM	7
E8	Pruebas y validación	7
E9	Despliegue y transición	8
E10	Estabilización y retiro	9

Historias Técnicas Priorizadas

E3 - Infraestructura Base (P0)

#	Historia	Estimación
HT-01	Crear solution .NET con estructura de proyectos	2h
HT-02	Configurar Generic Host + BackgroundService	4h
HT-03	Integrar Asterisk.Sdk con `AddAsterisk()`	2h
HT-04	Configurar Serilog: JSON a stdout (producción), plain text console (desarrollo). Enrichers: TenantId, CampaignId, ContactId, CallId	4h
HT-05	Implementar POCOs de dominio: 26 del Marcador + entidades compartidas de CBXLib (DyAgentes, DyCampanas, DyExtensiones, DyTroncales, DyListaNegraNumeros, etc.) + entidades de AgiAmi (DyFestivos, DyTiemposTimbrando, etc.)	12h
HT-06	Crear Dockerfile multi-stage (build AOT + runtime alpine) + Docker Compose (MySQL + Asterisk + IPcom Dialer)	6h
HT-07	Crear proyecto de tests + CI pipeline	4h

E4 - Integración Asterisk (P0)

#	Historia	Estimación
HT-08	Implementar `IOriginateService` con Asterisk.Sdk	8h
HT-09	Implementar procesamiento de `OriginateResponse`	8h
HT-10	Suscribir a HangupEvent para tracking de llamadas	4h
HT-11	Implementar rate limiter por campaña (`SemaphoreSlim`)	4h
HT-12	Tests de integración originate con Asterisk Docker	8h
HT-13	Validar reconexión automática AMI	4h

E4B - Migración AGI y Flujos AgiAmi (P0)

#	Historia	Estimación
HT-14	Implementar `DialerContestaHumanoAgi` — AGI script: lee vars canal, update contacto a "Contestada"	4h
HT-15	Implementar `DialerContestaMaquinaAgi` — AGI script: marca contacto como contestadora	4h
HT-16	Implementar `DialerAmdResponseAgi` — AGI script: respuesta AMD	4h
HT-17	Implementar `DialerInsertSampleAgi` — AGI script: insert registro muestra	4h
HT-18	Registrar 4 scripts en `FastAgiServer` con `SimpleMappingStrategy`	2h
HT-19	Implementar `AbandonedCallReinjectionService` — escucha `QueueCallerAbandonEvent`, agrupa 60s, insert contacto	8h
HT-20	Implementar `RejectedCallReinjectionService` — escucha `AgentRingNoAnswerEvent`, re-inyección	8h
HT-21	Implementar envío de email de alerta por abandonadas (replicar lógica de `Correo.enviar()` de AgiAmi)	4h
HT-22	Implementar `LegacyEncryptor` — AES-ECB con key `D7@l0g0*S.A.S109` + fallback key2 `{1..16}`, Base64 encoding. Key en `IOptions<EncryptionOptions>` (no hardcodeada)	2h
HT-23	Tests de integración AGI scripts con Asterisk Docker (dialplan → AGI → DB update)	8h
HT-24	Tests de integración flujo abandon → reinjection → dialer picks up	4h
HT-25	Definir y documentar estrategia de coexistencia AGI (puerto 4574 vs 4573)	2h

E5 - Motor de Marcación (P0)

#	Historia	Estimación
HT-26	Implementar `CampaignSchedulerService` (loop 20s)	8h
HT-27	Implementar evaluación de horario/día de campaña	4h
HT-28	Implementar `ContactDialerService` (10 teléfonos, retries)	16h
HT-29	Implementar `RetryPolicy` configurable	8h
HT-30	Implementar `OutboundRouteResolver` (migrar `AnalizadorRutaSaliente` de CBXLib)	6h
HT-31	Implementar PDS dialer engine	16h

#	Historia	Estimación
HT-32	Implementar modo predictivo (usar `AgentManager` de Asterisk.Sdk.Live para disponibilidad)	16h
HT-33	Implementar modo robótico	8h
HT-34	Implementar jitter delay anti-thundering-herd	2h
HT-35	Implementar active call tracker	4h
HT-36	Implementar `DncService` — consulta `DyListaNegraNumeros` de CBXLib	4h
HT-37	Implementar `HolidayService` — consulta `DyFestivos` de AgiAmi	2h
HT-37B	Implementar `ScheduledCallbackService` — polling cada 40s de contactos con `agendado=true AND agenda_fecha_hora<=NOW()`, priorización en cola (agendados primero), reset de intentos a -1	6h
HT-37C	Implementar lectura dual de config: `dy_marcador_campanas` (siempre) + `CAMPAN` CRM (solo tipos 6/7/8 para aceleración y AMD)	4h

E6 - Persistencia (P1)

#	Historia	Estimación
HT-38	Implementar `ICampaignRepository` (Dapper) — incluye queries de CBXLib `DAOMarcadorCampanas` y AgiAmi `DAODyCampanas`	10h
HT-39	Implementar `IContactRepository` (Dapper) — incluye insert de abandonadas de AgiAmi	10h
HT-40	Implementar `ISampleRepository` (Dapper)	4h
HT-41	Implementar `ICallLogRepository` (Dapper)	4h
HT-42	Implementar `IOutboundCallRepository` (Dapper)	4h
HT-43	Implementar `IAgentRepository` (solo lectura, de CBXLib)	4h
HT-44	Implementar `IDncRepository` (lista negra, de CBXLib)	2h
HT-45	Implementar `IHolidayRepository` (festivos, de AgiAmi)	2h
HT-46	Migrar connection string a `IOptions<T>` + secrets	4h
HT-47	Tests de integración repositorios vs MySQL	8h

E7 - CRM (P1)

#	Historia	Estimación
HT-48	Implementar `ICrmRepository` (INSERT CONDIA, UPDATE Muestra)	8h
HT-49	Implementar `ICrmRestClient` (POST `/bi/gestion/pdsprerob`)	4h
HT-50	Implementar `CrmSyncService` background	4h
HT-51	Tests de integración CRM	4h

E8 - Pruebas (P1)

#	Historia	Estimación
HT-52	Pruebas end-to-end: campaña completa incluyendo AGI callbacks y abandon reinjection	16h
HT-53	Pruebas de regresión: comparación Java vs .NET	8h
HT-54	Pruebas de carga con múltiples campañas	8h
HT-55	Pruebas de resiliencia (chaos testing)	8h
HT-56	Pruebas de aceptación funcional con operaciones	8h
HT-57	Pruebas de shadow mode: comparar decisiones IPcom Dialer vs Java con mismos datos de producción	4h

E9 - Despliegue (P1)

#	Historia	Estimación
HT-58	Configurar deployment Docker de producción: imagen optimizada, compose de producción, env vars, health/readiness probes	6h
HT-59	Implementar shadow mode (lee campañas, simula marcación, no ejecuta originates)	8h
HT-60	~~Implementar flag `sistema` en BD~~ — **ELIMINADO** (corte atómico, no coexistencia)	0h
HT-61	Crear script de actualización de dialplan (AGI routing: scripts marcador → IPcom Dialer FastAGI) + script de rollback	4h
HT-62	Crear dashboards de monitoreo (incluir métricas AGI y abandoned/rejected)	8h
HT-63	Escribir runbook operativo (procedimiento de corte, rollback < 5 min)	4h
HT-64	Script de rollback: revertir dialplan + reiniciar Java Marcador	2h

Dependencias entre Historias

```
HT-01 → HT-02 → HT-03 → HT-08 (Infra → Asterisk AMI)
HT-05 → HT-38..HT-45 (POCOs → Repositorios)
HT-08 → HT-14..HT-18 (AMI → AGI scripts, necesitan conexión AMI para eventos)
HT-08 → HT-19..HT-20 (AMI → Abandoned/Rejected, escuchan eventos AMI)
```

```
HT-18 → HT-23 (AGI registrado → tests de integración AGI)
HT-06 → HT-12 (Docker → tests Asterisk)
HT-38..HT-45 → HT-26 (Repos → Campaign scheduler)
HT-14..HT-25 → HT-26..HT-37 (AGI + Repos → Motor de marcación)
HT-28 → HT-52 (ContactDialer → E2E tests)
HT-48 + HT-49 → HT-50 (CRM repos → CRM sync)
HT-61 → HT-57 (Dialplan script → test shadow mode)
```

Orden Recomendado de Ejecución

```
Sprint 1: HT-01..HT-07 (Infraestructura base)
Sprint 2: HT-08..HT-13 (Asterisk AMI) + HT-38..HT-42 (Repos core, en paralelo)
Sprint 3: HT-14..HT-25 (AGI scripts + Abandoned/Rejected) + HT-43..HT-47 (Repos adicionales, en paralelo)
Sprint 4: HT-26..HT-37 (Motor de marcación completo: scheduler, PDS, predictivo, robótico, DNC, festivos)
Sprint 5: HT-48..HT-51 (CRM) + HT-58..HT-64 (Despliegue + scripts de corte, en paralelo)
Sprint 6: HT-52..HT-57 (Pruebas integrales + shadow mode)
```

---

16. Cronograma Tentativo

Estimación de Alto Nivel por Fases

Fase	Duración	Semanas	Dependencia
F0: Descubrimiento (+ dialplan, EncriptadorPropio, CBXLib deps)	1-2 sem	S1-S2	—
F1: Análisis funcional (+ AGI scripts, flujos AgiAmi, deps CBXLib)	2-3 sem	S2-S4	F0
F2: Diseño arquitectura (+ coexistencia AGI dual-port)	1-2 sem	S4-S5	F1
F3: Base técnica .NET	1-2 sem	S5-S6	F2
F4: Integración Asterisk AMI	2-3 sem	S6-S8	F3
F4B: Migración AGI + Abandoned/Rejected	2 sem	S8-S10	F4
F5: Motor de marcación	3-4 sem	S10-S13	F4B
F6: Persistencia	2-3 sem	S8-S10	F3 (paralelo con F4/F4B)
F7: CRM + Pruebas	2-3 sem	S13-S15	F5, F6
F8: Shadow mode + Corte + Estabilización inicial	2-3 sem	S15-S17	F7
F9: Estabilización y retiro Java	2-4 sem	S17-S20	F8

Diagrama de Gantt Simplificado

```
Semana:  1  2  3  4  5  6  7  8  9  10 11 12 13 14 15 16 17 18 19 20
F0:      ■■■■
F1:         ■■■■■■■
F2:               ■■■■
F3:                  ■■■■
F4:                     ■■■■■■■
F4B:                          ■■■■                    (AGI + Abandoned)
F5:                               ■■■■■■■■■■■■■■
F6:                     ■■■■■■■■■■                      (paralelo con F4-F4B)
F7:                                         ■■■■■■■
F8:                                               ■■■■■■■ (shadow + corte)
F9:                                                     ■■■■■■■■■
```

```
■■■
```

7. **Multi-tenant confirmado** — filtro por `id_huesped` en todas las queries, métricas y logs segmentados por tenant

La estrategia recomendada es **reemplazo completo con validación en shadow mode**, construyendo IPcom Dialer completo como contenedor Docker (imagen AOT ~15 MB sobre Alpine), validándolo contra el Java en shadow mode, y realizando un corte único. El Java se mantiene como fallback durante la estabilización (rollback < 5 min). Total estimado: **18-20 semanas** desde descubrimiento hasta estabilización.

> ***Cambio v2.1 respecto a v2.0:*** *Se incorporan requisitos de containerización (Docker desde día 1, K8s futuro), logging container-native (stdout JSON, no archivos), multi-tenancy confirmada (modelo SaaS), versión de Asterisk (18, plan migrar a últimas), dimensionamiento inicial (10-30 campañas), y variables de entorno para configuración. Se agregan ADR-015 a ADR-019 y los anexos D (dimensionamiento) y E (variables de entorno).*

> ***Cambio v3.0 respecto a v2.3:*** *Se incorpora* ***mapa completo del ecosistema Dyalogo/IPcom*** *en nueva sección 4.13, basado en análisis exhaustivo de las 23 aplicaciones Java, 7 servicios Node.js, aplicación PHP/CRM, infraestructura Terraform/GCP, y 368 tablas MySQL. Hallazgos clave: (1) 15 servicios Java desplegados + 6 Node.js activos + 4 Cloud Run + PHP CRM multi-módulo; (2)* `dyalogocore` *es el hub central — genera dialplan, gestiona AMI, chat, config; (3) 279 tenants activos, 120+ extensiones SIP, 155+ colas ACD; (4) Infraestructura en GCP us-central1, 2 instancias MySQL 8.0 (8GB RAM cada una); (5) 15+ credenciales hardcodeadas identificadas (riesgo de seguridad); (6) IPcom Dialer reemplaza SOLO 1 de 23 servicios Java — impacto controlado; (7) Grafo completo de dependencias inter-servicios documentado.*

> ***Cambio v2.3 respecto a v2.2:*** *Se incorpora análisis completo de la* ***configuración de producción de Asterisk*** *(*`/etc/asterisk/`*) en nueva sección 4.12. Hallazgos críticos: (1) Solo 1 script AGI usado en producción (*`AGIClasificaAM.agi` *en* `172.18.0.2:5000`*), no 4 como se infirió del código Java; (2) 57 contextos de marcador activos con 2 patrones claros (35 sin AMD, 22 con AMD); (3) 12 variables de canal que IPcom Dialer debe setear en cada originate; (4) AMD es nativo de Asterisk — no requiere implementación en .NET; (5) Credenciales AMI reales (4 usuarios), networking NAT/TLS en GCP; (6)* `chan_sip` *activo (pjsip deshabilitado); (7) Límite global de canales marcador = 20. Se actualiza Docker Compose de desarrollo con contextos de producción reales. Items de información: 16/17 resueltos.*

> ***Cambio v3.1 respecto a v3.0:*** *Se incorpora* ***análisis de configuración operacional de producción*** *(*`/etc/dyalogo/`*) en nueva sección 4.14. Hallazgos: (1)* `parametros_generales.properties` *con 36 propiedades reales — solo 8 relevantes para IPcom Dialer; (2)* `blendActivo=true` *en producción — IPcom Dialer DEBE implementar modo blend; (3) Payara 5 con 3 WARs (core, cbx, public_front) — marcador es independiente; (4) Colector ETL con* `callbackAbandonadas=true` *— depende del formato de tablas* `dy_llamadas_*`*; (5)* `procesadorFlechas` *(Node.js) alimenta muestras del marcador — sin cambios requeridos; (6) Pool MySQL en Java = 10 conexiones (MUY bajo) — IPcom Dialer usa 50; (7) GCS bucket* `ipcom360-comunicaciones` *para archivos públicos; (8) Script generador de vistas BI itera 279 tenants activos. Anexo E actualizado con valores REALES de producción y mapeo explícito desde propiedades Java.*

> ***Cambio v4.0 respecto a v3.1:*** *Se incorpora* ***análisis exhaustivo de*** `dyalogocore` *en nueva sección 4.15 — el componente más crítico del ecosistema (104 clases Java, 50+ endpoints REST, 5 EJBs singleton). Hallazgos fundamentales: (1) dyalogocore GENERA dinámicamente los 57 contextos* `DyCampanaMarcador_*.conf` *desde la BD → escribe archivos .conf → ejecuta* `dialplan reload` *via AMI; (2) Controla ciclo de vida completo de agentes (login/logout/pausa) con QueueAdd/Remove/Pause actions y delays de 1000-6500ms; (3) Mantiene conexión AMI dual:* `dyalogoami` *para acciones +* `dyamievt` *para escucha de 11 eventos; (4) Sistema de blend activo (*`blendActivo=true`*) con 5 clases dedicadas al tracking de agentes inbound/outbound; (5) Caché centralizado (*`EJBSingletonCacheDatos`*) con refresco cada 10 min; (6) 7 códigos de acción para dialplan (ninguna, extensión, cola, audio, IVR, externo, encuestas); (7) AMD configurado per-campaña desde BD.* ***VEREDICTO: NO migrar dyalogocore*** *— tiene 22 responsabilidades además del marcador, el costo sería 10x mayor, y la coexistencia es viable. IPcom Dialer debe respetar el dialplan generado, leer* `ActividadActual` *de BD, y NO duplicar operaciones de agentes. Se documentan 8 acciones concretas de coexistencia.*

> ***Cambio v6.0 respecto a v5.0:*** *Se incorporan* ***análisis P2*** *en secciones 4.18 y 4.19 — los 2 gaps finales de información.* ***Sección 4.18 (Volúmenes de Datos):*** *Escala real de producción: 352 agentes, 42 campañas activas, 20 canales máx globales, ~57K intentos/día de capacidad, 168 colas ACD, pool MySQL de solo 10 conexiones (bottleneck confirmado), batch default de 10 registros (`CAMPAN_MaxRegDinam_b`), ciclo PDS de 60s, ~235K registros acumulados en `dy_llamadas_salientes`, `dy_marcador_contactos` sin AUTO_INCREMENT (crecimiento ilimitado).* ***Sección 4.19 (Shadow Mode):*** *Procedimiento completo de validación en 4 fases + preparación: (0) infraestructura — columna `dialer_source`, tabla `dy_marcador_dialer_routing`, dashboard Grafana con 7 días baseline; (1) dry-run solo-lectura con targets de match rate > 95%; (2) una campaña viva por 3 tenants con 7 criterios de éxito durante 7 días consecutivos; (3) expansión progresiva de ~1% a ~60% del tráfico con gates de expansión y A/B testing estadístico; (4) cutover completo en ventana de bajo tráfico. Rollback sub-5-minutos garantizado en toda fase via tabla de enrutamiento atómica. Duración total del programa: 8-10 semanas calendario + 2 semanas post-cutover. Los 9 gaps de información (P0+P1+P2) están* ***TODOS CERRADOS****.*

> ***Cambio v5.0 respecto a v4.0:*** *Se incorporan* ***análisis P0 y P1*** *en secciones 4.16 y 4.17 — los 7 gaps de información más críticos del proyecto.* ***Sección 4.16 (P0):*** *(1) Algoritmo PDS completo con fórmulas de pacing por tipo (6=PDS, 7=Predictivo, 8=Robot), 6 mejoras numeradas, AHT cache 2min, tope 3 llamadas/agente; (2) Flujo de Originate completo — corrección: solo 7 variables de canal (no 12), las otras 5 las setea el dialplan; 7 códigos de respuesta (1-5 + 101/102); 3 tablas escritas por callback (`dy_marcador_contactos`, `dy_marcador_log`, `dy_llamadas_salientes`); trunk selection con patrón matching y desborde; (3) AgiAmi escribe en `dy_marcador_contactos` desde 3 code paths — 4 race conditions identificadas (RC-1 a RC-4), especialmente inserciones de abandonadas cada 60s.* ***Sección 4.17 (P1):*** *(4) Marcador NO usa stored procedures — escribe SQL directo; 15 SPs encontrados en DDL, todos ejecutados por dyalogocore o colector; (5) Callbacks agendados via `HLlamadasAgenda` cada 40s — prioridad sobre contactos normales, reset de intentos, limpieza de agenda en callback; (6) Blend es pause a nivel de CAMPAÑA ENTERA (no per-agente) — dual toggle global+per-campaña, AMI `queue show` + BD `ActividadActual`, 2.5s espera por llamada en cola; (7) 7 endpoints REST identificados — los 3 obligatorios: `/agentes/tareas/trabajoCampana`, `/bi/gestion/pdsprerob`, `/agentes/tareas/listarPDSPre`; auth hardcodeada `local/local`; marcador también escribe directo a `DYALOGOCRM_SISTEMA` via JDBC.*

Primeros Pasos Concretos a Ejecutar

1. ~~**Obtener dump del esquema MySQL**~~ **COMPLETADO** — 4 DDLs analizados (telefonia 241KB, CRM 343KB, general 28KB, asterisk 18KB). 10 hallazgos documentados en sección 4.4
2. **Inmediato:** ~~Revisar proyectos hermanos~~ **COMPLETADO** — ver sección 2 "Proyectos Hermanos"
3. ~~**Obtener dialplan de Asterisk**~~ **COMPLETADO** — 57 contextos `DyCampanaMarcador_*` analizados, 2 patrones (con/sin AMD), 1 script AGI real (`AGIClasificaAM.agi`), 12 variables de canal. Ver sección 4.12
4. ~~**Obtener fuente de `EncriptadorPropio`**~~ **COMPLETADO** — AES-ECB con key `D7@l0g0*S.A.S109`, reproducción trivial en .NET (ver sección 4.3)
5. **Semana 1-2:** Entrevistar al equipo de operaciones para documentar reglas de negocio no escritas
6. **Semana 2:** Crear solution `IPcom.Dialer.slnx` con `AddAsterisk()` (AMI + AGI) y Docker Compose funcional
7. **Semana 3:** Primer originate de prueba + primer AGI script respondiendo desde .NET contra Asterisk Docker

---

18. Anexos

A. Información que Debo Levantar del Sistema Actual Antes de Iniciar

#	Información	Fuente	Prioridad	Bloquea Fase
1	~~Esquema MySQL (`dyalogo_telefonia`)~~	~~DBA~~	~~P0~~	**RESUELTO** — `ddl_telefonia.sql` analizado

#	Información	Fuente	Prioridad	Bloquea Fase
2	~~Esquema MySQL (`DYALOGOCRM_SISTEMA`)~~	~~DBA~~	~~P0~~	**RESUELTO** — `ddl_crm.sql` analizado
3	~~Versión de Asterisk~~	~~Infra~~	~~P0~~	**RESUELTO** — Asterisk 18, plan migrar a últimas versiones
4	~~Contextos y extensiones del dialplan~~	~~Infra~~	~~P0~~	**RESUELTO** — Contextos dinámicos `DyCampanaMarcador_<id>`, generados por dyalogocore. Ver sección 4.11
5	~~API contract de `/bi/gestion/pdsprerob`~~	~~CRM~~	~~P1~~	**RESUELTO** — POST JSON, auth local/local, 7 campos. Ver sección 4.9
6	~~Volúmenes~~	~~Ops~~	~~P0~~	**RESUELTO** — 10-30 campañas simultáneas inicialmente. Modelo SaaS con crecimiento esperado
7	~~Contenido de `DyalogoCBXLib` y `DyalogoCBXAgiAmi`~~	~~Código~~	~~P0~~	**RESUELTO**
8	~~Scripts de deployment~~	~~DevOps~~	~~P1~~	**RESUELTO** — Deployment será Docker containerizado. Futuro Kubernetes
9	~~Otros servicios que usan las mismas tablas MySQL~~	~~Dev~~	~~P0~~	**RESUELTO** — 7 servicios concurrentes identificados. Matriz de escritores en sección 4.10
10	~~Reglas de negocio de reintentos~~	~~PO~~	~~P0~~	**RESUELTO** — Algoritmo completo en sección 4.6. 2 reintentos/tel, max 3 globales, 2,875 reglas por campaña×respuesta
11	~~Configuración de `servicios_asterisk.properties` en producción~~	~~DevOps~~	~~P0~~	**RESUELTO** — Propiedades extraídas del código Java. Mapeadas a variables de entorno en Anexo E
12	~~Variables de entorno o JVM flags usadas en producción~~	~~DevOps~~	~~P1~~	**RESUELTO** — JVM flags no aplican en .NET AOT. Config via env vars documentada en Anexo E
13	~~**Dialplan de Asterisk:** contextos con `AGI(agi://host:4573/EventoMarcador*)`~~	~~Infra~~	~~P0~~	**RESUELTO** — 57 contextos `DyCampanaMarcador_<id>` analizados de producción. Solo 1 script AGI real (`AGIClasificaAM.agi` en `172.18.0.2:5000`). 2 patrones: con/sin AMD. Ver sección 4.12

#	Información	Fuente	Prioridad	Bloquea Fase
14	~~**Algoritmo de** `EncriptadorPropio`~~	~~Código CBXLib~~	~~P0~~	**RESUELTO** — AES-ECB, key documentada en sección 4.3
15	~~**Campañas con** `manejoAbandono` **activo**~~	~~DBA + Ops~~	~~P1~~	**RESUELTO** — De 57 campañas en producción, 22 usan AMD. `AccMaq` siempre es `null` (no hay acción especial para máquina). Solo 1 script AGI (`AGIClasificaAM.agi`). Ver sección 4.12
16	~~`LibreriaPersistencia.jar`~~	~~Código~~	~~P1~~	**RESUELTO** — 9 clases, `DAOImpOperaciones` (969 líneas). Boilerplate Hibernate. Solo replicar audit trail y bulk save. Ver sección 4.7
17	~~**Config SMTP** (`dyalogo_config_smtp.properties`)~~	~~DevOps~~	~~P2~~	**RESUELTO** — Archivo en `/Dyalogo/conf/dyalogo_config_smtp.properties`. Propiedades: `auth`, `ttls`, `dominio`, `password` (cifrado AES), `puerto`, `servidorSmtp`, `usuario`. Usado por `DyalogoCBXAgiAmi` (no por el marcador). Password descifrado con `EncriptadorPropio`. IPcom Dialer puede usar `System.Net.Mail` con config via env vars

B. Quick Wins

#	Quick Win	Esfuerzo	Valor	Fase
QW1	Crear Docker Compose para desarrollo local (MySQL + Asterisk)	4h	Alto	F3
QW2	Mapear entidades JPA a POCOs .NET (Marcador + CBXLib compartidas + AgiAmi)	12h	Alto	F3
QW3	Primer originate funcional con Asterisk.Sdk	4h	Alto	F4
QW3B	**Primer AGI script respondiendo desde .NET** FastAgiServer	4h	Alto	F4B
QW4	Migrar queries SQL de DAOs (Marcador + AgiAmi + CBXLib) a archivos `.sql`	6h	Medio	F1
QW5	Eliminar credenciales hardcodeadas desde el día 1	2h	Alto	F3

#	Quick Win	Esfuerzo	Valor	Fase
QW6	Configurar health check endpoint	1h	Medio	F3
QW7	Dashboard de métricas básicas (campañas activas, originates/min)	4h	Alto	F4

C. Decisiones Arquitectónicas Clave

ADR #	Decisión	Opciones Evaluadas	Elegida	Razón
ADR-001	**Topología de deployment**	Worker Service vs. ASP.NET Host vs. Console App	Worker Service	Lifecycle management, health checks, sin overhead HTTP innecesario
ADR-002	**ORM / Data Access**	EF Core vs. Dapper vs. ADO.NET puro	Dapper	AOT-compatible, SQL explícito, control total, mínimo overhead
ADR-003	**Base de datos**	Mantener MySQL vs. Migrar a PostgreSQL	Mantener MySQL	Minimizar riesgo; migración de BD es otra épica completa
ADR-004	**Modelo de concurrencia**	Thread pool fijo vs. Channel<T> vs. TPL Dataflow	Channel\<T\>	Backpressure nativo, bounded, async-first, más simple que Dataflow
ADR-005	**Rate limiting**	Token bucket vs. SemaphoreSlim vs. System.Threading.RateLimiting	SemaphoreSlim	Ya probado en Asterisk.Sdk, simple, sin dependencia extra
ADR-006	**Integración CRM**	Direct DB vs. REST API vs. Message queue	Mantener ambos (DB + REST)	Paridad con Java; rediseño CRM es scope separado
ADR-007	**Logging**	Microsoft.Extensions.Logging vs. Serilog vs. NLog	Serilog	Structured logging, enrichers, múltiples sinks, community support
ADR-008	**Estrategia de transición**	Strangler Fig vs. Big Bang vs. Big Bang controlado con shadow mode	Big Bang controlado con shadow mode	El marcador es atómico — coexistencia parcial introduce más riesgo del que evita; rollback < 5 min
ADR-009	**Callback handling**	Polling vs. Event subscription vs. TaskCompletionSource	Event subscription (IObservable)	Ya implementado en Asterisk.Sdk, reactive, no polling
ADR-010	**Secrets management**	Environment vars vs. User Secrets vs. Vault	Environment vars + User Secrets (dev)	Simple, Docker-compatible; Vault si se requiere en producción
ADR-011	**AGI post-corte**	Mismo puerto vs. Puertos diferentes vs. Proxy	Puertos diferentes (IPcom Dialer=4574, Java AgiAmi=4573)	IPcom Dialer sirve 4 scripts de marcador; Java AgiAmi sigue sirviendo 13 scripts no-marcador; rollback cambiando dialplan
ADR-012	**Extracción de CBXLib**	Migrar toda la librería vs. Extraer solo lo necesario vs. Wrapper	Extraer solo lo necesario (~25%)	CBXLib sigue usada por otros módulos Java; migrar todo es innecesario

ADR #	Decisión	Opciones Evaluadas	Elegida	Razón
ADR-013	**Cifrado de claves**	Replicar AES-ECB vs. Migrar a AES-GCM vs. bcrypt vs. Bridge Java	Replicar AES-ECB exacto (key=`D7@l0g0*S.A.S109`, ECB, PKCS7, Base64)	Preserva compatibilidad con datos cifrados en producción; key mover a secret manager; a futuro evaluar AES-GCM
ADR-014	**Esquema de BD**	Modificar esquema vs. Mantener intacto vs. Vistas de abstracción	Mantener esquema intacto	El marcador es una pieza de un sistema más grande; modificar tablas `dy_*` afecta CRM, AgiAmi, Web y otros módulos. Los POCOs .NET mapean directamente a las tablas legacy
ADR-015	**Deployment**	Bare-metal + systemd vs. Docker Compose vs. Kubernetes	Docker Compose (inicial), Kubernetes (futuro)	Container-first desde día 1. Imagen AOT sobre `runtime-deps:alpine`. K8s-ready por diseño (stateless, env vars, probes)
ADR-016	**Logging en producción**	File sinks vs. stdout JSON vs. Sidecar	stdout JSON (Serilog `JsonFormatter`)	Logs a stdout es el estándar en Docker/K8s. Docker log driver y K8s capturan automáticamente. Sin archivos de log dentro del contenedor
ADR-017	**Multi-tenancy**	BD separada por tenant vs. Schema separado vs. Filtro por columna	Filtro por columna (`id_huesped`/`id_proyecto`)	Mantiene compatibilidad con esquema existente. Todas las queries incluyen `WHERE id_proyecto = @tenantId`. A futuro evaluar BD por tenant si el volumen lo justifica
ADR-018	**Compatibilidad Asterisk**	Solo Asterisk 18 vs. 18+ vs. Latest only	Asterisk 18, 20, 21+	Asterisk.Sdk basado en AMI estándar. No usar features específicas de una versión. Testear contra 18 (producción actual) y 20/21 (objetivo de migración)
ADR-019	**Escalamiento horizontal**	Single instance vs. Instance per Asterisk vs. Shared instance pool	1 instancia IPcom Dialer por servidor Asterisk	Cada contenedor se conecta a un Asterisk. `AsteriskServerPool` de Asterisk.Sdk permite multi-server en futuro. K8s escala con replicas = N servidores Asterisk

D. Dimensionamiento Inicial (10-30 campañas)

Recurso	Valor inicial	Justificación
`Channel<DialRequest>` capacity	500	~15 contactos/campaña × 30 campañas = 450 peak
`SemaphoreSlim` por campaña	configurable (default 10)	`cantidad_llamadas_simultaneas` de `dy_marcador_campanas`
MySQL connection pool	`MinPoolSize=5; MaxPoolSize=50`	30 campañas × ~1.5 conexiones activas promedio

Recurso	Valor inicial	Justificación
Worker tasks concurrentes	30	1 `CampaignWorker` task por campaña activa
Health check interval	10s	K8s liveness probe
Readiness check	AMI connected + MySQL reachable	K8s readiness probe — no recibe tráfico AGI hasta estar listo
Memoria contenedor	256 MB limit, 128 MB request	AOT es muy eficiente; monitorear y ajustar
CPU contenedor	1 core limit, 0.5 core request	Async I/O — no es CPU-bound

E. Variables de Entorno (Docker / K8s ConfigMap)

Mapeo desde `parametros_generales.properties` (produccion) a env vars de IPcom Dialer:

Variable	Ejemplo (prod)	Origen en `parametros_generales`	Obligatoria	Descripción
`ASTERISK__AMI__HOSTNAME`	`127.0.0.1`	`direccionIpAmi`	Si	Host del servidor Asterisk
`ASTERISK__AMI__PORT`	`5038`	`puertoAMI`	No (default)	Puerto AMI
`ASTERISK__AMI__USERNAME`	`dyalogoami`	`usuario`	Si	Usuario AMI
`ASTERISK__AMI__PASSWORD`	*(K8s Secret)*	`contrasena` (`dyalogo*`)	Si	Password AMI (K8s Secret)
`ASTERISK__AGI__PORT`	`4574`	*(nuevo — no existe en Java)*	No (default)	Puerto FastAGI
`CONNECTIONSTRINGS__TELEFONIA`	`Server=127.0.0.1;Database=dyalogo_telefonia;Uid=dymarcador;...`	`direccionIpBd` + credenciales del marcador	Si	BD telefonía
`CONNECTIONSTRINGS__CRM`	`Server=127.0.0.1;Database=DYALOGOCRM_SISTEMA;Uid=dymarcador;...`	`direccionIpBd`	Si	BD CRM
`CONNECTIONSTRINGS__GENERAL`	`Server=127.0.0.1;Database=dyalogo_general;Uid=dymarcador;...`	`direccionIpBd`	Si	BD general
`CONNECTIONSTRINGS__ASTERISK`	`Server=127.0.0.1;Database=asterisk;Uid=dymarcador;...`	`direccionIpBd`	Si	BD asterisk
`DIALER__TENANTID`	`5`	*(parametro de ejecucion)*	Si	ID del tenant (huesped) que gestiona esta instancia
`DIALER__CAMPAIGNPOLLINTERVALSECONDS`	`20`	*(nuevo — Java usa thread continuo)*	No (default)	Intervalo del scheduler
`DIALER__MAXCONCURRENTCAMPAIGNS`	`30`	*(nuevo — Java no tiene limite)*	No (default)	Máximo campañas simultáneas
`DIALER__BLENDMODE`	`true`	`blendActivo`	No (default false)	Habilitar blend inbound+outbound
`ENCRYPTION__KEY`	*(K8s Secret)*	*(hardcodeado en `EncriptadorPropio`)*	Si	Key AES (K8s Secret)

Variable	Ejemplo (prod)	Origen en `parametros_generales`	Obligatoria	Descripción
`SERILOG__MINIMUMLEVEL__DEFAULT`	`Information`	*(nuevo — Java usa Log4j)*	No (default)	Nivel de log
`CRM__RESTBASEURL`	`https://127.0.0.1:8080/dyalogocore`	`ipServicioCore` + `intPuertoCore`	Si	URL base del CRM/Core REST
`CRM__APITOKEN`	*(K8s Secret)*	`tokenAPIInterno`	Si	Token interno para REST API
`DIALER__PUBLICIP`	`34.63.181.35`	`publicIp`	No	IP publica del Asterisk (para SIP headers)
`DIALER__SIPTECHNOLOGY`	`SIP`	`tecnologiaUsuarios`	No (default SIP)	Tecnologia SIP (SIP/PJSIP)

Documento generado como parte del plan de migración tecnológica. Sujeto a revisión y actualización conforme se complete el levantamiento de información de las Fases 0 y 1.